

macOS 开发环境完全指南

涵盖 VS Code、iTerm2、Git、GitHub CLI、.gitignore、Python venv、uv、Conda、Mamba、Docker、Homebrew、Zsh、SSH、Make、pyenv、nvm、tmux、Vim、CUDA、PyTorch、Jupyter、MLOps、Prompt Engineering、AI Agent 等开发工具的极其详细使用方法与命令参考。

目录

- 一、Homebrew macOS 包管理器
- 二、iTerm2 终端利器
 - 2.5 tmux 终端复用器深入指南
 - 2.6 Vim / Neovim 基础求生指南
- 三、Zsh Shell 与 Oh My Zsh
- 四、SSH 远程连接与密钥管理
- 五、VS Code 快捷键与使用技巧
- 六、Git 版本控制完全指南
 - 6.15 现代 Git 辅助工具
- 七、Python 虚拟环境 venv 完全指南
- 八、Conda 环境管理完全指南
- 九、Python 多版本管理 pyenv
- 十、Node.js 版本管理 nvm
- 十一、Docker 容器化完全指南
- 十二、Make 与 Makefile 构建自动化
 - 12.5 just——现代命令运行器
- 十三、常用系统工具
 - 13.1 rsync
 - 13.2 sed
 - 13.3 awk
 - 13.4 cron / launchd
 - 13.5 进程与端口管理
 - 13.6 curl / HTTP API 测试
 - 13.7 日志查看与文本处理
 - 13.8 管道、重定向与环境变量
 - 13.9 SQLite
 - 13.10 命令行基础命令速查
 - 13.11 现代化 CLI 工具集
 - 13.12 Nginx 本地开发环境
 - 13.13 CI/CD 本地测试 (act)
- 十四、数据库客户端与工具
- 十五、深度学习开发环境
- 十六、CUDA 与并行计算基础

- 十七、数据科学与 MLOps 工具
 - 十八、Prompt Engineering 提示词工程
 - 十九、AI Agent 与 AI 辅助开发
 - 二十、云 GPU 训练完整 workflow
-

学习路径建议

本文档按学习逻辑编排——从环境搭建到日常工具再到 AI 开发。初学者可从第一章顺序阅读，有经验者可按需跳转。

路径一：新手入门（刚拿到 Mac，从零搭建开发环境）

- ① Homebrew（第一章） → 安装一切软件的基础
- ② iTerm2（第二章） → 每天都要用的终端
- ③ Zsh（第三章） → 让终端更好用
- ④ SSH（第四章） → 连接服务器 / 配置 GitHub
- ⑤ VS Code（第五章） → 主力编辑器
- ⑥ Git（第六章） → 版本控制，写代码第一天就要用

路径二：进阶开发（已入门，需要专业化环境管理）

- ⑦ Python venv + uv（第七章） → 项目级包隔离
- ⑧ Conda + Mamba（第八章） → 科学计算 / 跨语言依赖
- ⑨ pyenv（第九章） → 多 Python 版本共存
- ⑩ nvm（第十章） → 多 Node.js 版本共存
- ⑪ Docker（第十一章） → 应用容器化与部署
- ⑫ Make + just（第十二章） → 构建自动化
- ⑬ 系统工具 + CLI（第十三章） → 日志、管道、curl、Nginx
- ⑭ 数据库客户端（第十四章） → PostgreSQL / Redis / MongoDB

路径三：深度学习与 GPU 计算

- ⑮ 深度学习开发环境（第十五章） → PyTorch / Jupyter / HuggingFace
- ⑯ CUDA 与并行计算（第十六章） → GPU 编程 / OpenMP
- ⑰ 数据科学与 MLOps（第十七章） → DVC / MLflow / 模型部署
- ⑱ 云 GPU 训练工作流（第二十章） → 远程 GPU 全流程实战

路径四：AI 驱动的开发

速查场景

你要做什么	看哪一章
装软件	第一章 Homebrew
记不住快捷键	第二章 iTerm2 / 第三章 Zsh / 第五章 VS Code
Git 操作忘了	第六章 Git (含 .gitignore、GitHub CLI、Lazygit)
管理 Python 环境	第七章 venv + 第八章 Conda + 第九章 pyenv
部署项目	第十一章 Docker
SSH 连不上 / 配置免密	第四章 SSH
远程服务器持久会话	第二章 tmux (2.5)
Vim 基本操作	第二章 Vim 求生指南 (2.6)
文本批量处理	第十三章 sed/awk
写定时任务	第十三章 cron/launchd
调 API 接口	第十三章 curl / WebSocket / gRPC
操作数据库	第十四章 PostgreSQL / Redis / MongoDB
深度学习环境搭建	第十五章 PyTorch / Jupyter / HuggingFace
GPU 编程 / CUDA	第十六章 CUDA 与并行计算
数据版本控制	第十七章 DVC
实验与模型管理	第十七章 MLflow
模型部署	第十七章 FastAPI / ONNX
让 AI 写出更好的代码	第十八章 Prompt Engineering
AI 辅助编程工具	第十九章 Copilot / Cursor / Claude Code
远程 GPU 训练	第二十章 云 GPU 训练 workflow
本地 CI/CD 调试	第十三章 act (13.13)

一、Homebrew macOS 包管理器

1.1 简介与安装

Homebrew 是 macOS 上最流行的包管理器（也被称为「macOS 缺失的包管理器」）。它允许你通过命令行简便地安装、更新和管理各种开源软件和命令行工具，自动处理依赖关系。Homebrew 将软件安装在自己的目录中（默认 `/opt/homebrew` for Apple Silicon 或 `/usr/local` for Intel），然后将其符号链接到系统路径中，因此不会污染系统目录。

```
# 安装 Homebrew (官方安装脚本)
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 安装完成后, 按照提示将 Homebrew 添加到 PATH (Apple Silicon Mac)
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"

# 验证安装
brew doctor
```

1.2 核心概念

- **Formula**（配方）：一个 Ruby 脚本，定义了如何从源码编译安装一个软件包。例如 `python`、`git` 都是 Formula。
- **Cask**（木桶）：Homebrew 的一个扩展，用于安装 macOS 原生 GUI 应用（如 Google Chrome、VS Code、Docker Desktop）。Cask 安装的是编译好的二进制应用，而非从源码编译。
- **Tap**（仓库源）：Homebrew 的第三方 Formula 仓库。除了官方的 `homebrew/core`，你可以添加社区维护的 Tap 来安装官方仓库中没有的软件。
- **Keg-only**：某些 Formula 被标记为 `keg-only`，意味着它们不会自动链接到 `/opt/homebrew/bin`，避免与系统自带版本冲突（例如 macOS 自带的 Python 和 Homebrew 安装的 Python）。
- **Bottle**（预编译包）：为特定 macOS 版本预编译好的二进制包，安装时不需要从源码编译，速度极快。

1.3 常用命令

```
# --- 安装软件 ---

# 安装命令行工具 (Formula)
brew install <包名>
brew install git python node

# 安装 GUI 应用 (Cask)
```

```
brew install --cask <应用名>
brew install --cask visual-studio-code google-chrome
```

安装指定版本

```
brew install <包名>@<版本号>
brew install python@3.11
```

从源码编译安装（而非使用预编译的 Bottle）

```
brew install --build-from-source <包名>
```

安装时显示详细输出

```
brew install --verbose <包名>
```

--- 搜索与查找 ---

搜索可用的软件包

```
brew search <关键词>
```

搜索并显示描述信息

```
brew search --desc <关键词>
```

查看软件包的详细信息（版本、依赖、安装路径等）

```
brew info <包名>
```

查看已安装包的信息

```
brew info --installed <包名>
```

查看软件包的依赖关系

```
brew deps <包名>
```

以树状图显示依赖

```
brew deps --tree <包名>
```

查看哪些包依赖了指定包

```
brew uses <包名>
```

查看本地 Homebrew 仓库中的所有可用包

```
brew formulae
brew casks
```

--- 更新与升级 ---

更新 Homebrew 自身和所有 Formula 仓库的索引

```
brew update
```

升级所有已安装的包

```
brew upgrade
```

升级指定包

```
brew upgrade <包名>
```

查看哪些包可以升级（不实际升级）

```
brew outdated
```

清理旧版本包和缓存


```
brew cleanup

# 模拟清理（查看将删除什么但不实际删除）
brew cleanup --dry-run

# --- 查看与管理 ---

# 列出所有已安装的 Formula
brew list
# 或
brew list --formula

# 列出所有已安装的 Cask
brew list --cask

# 查看已安装包的安装路径
brew --prefix <包名>

# 查看已安装包的文件列表
brew list <包名>

# 查看包的依赖是否都正确安装
brew missing

# 检查 Homebrew 系统配置问题
brew doctor

# 查看 Homebrew 配置
brew config

# 查看 Homebrew 环境变量
brew --env

# --- 卸载 ---

# 卸载命令行工具
brew uninstall <包名>
# 或
brew remove <包名>

# 卸载 GUI 应用
brew uninstall --cask <应用名>

# 卸载并删除所有相关依赖（如果这些依赖不再被其他包使用）
brew uninstall --ignore-dependencies <包名>
brew autoremove # 自动删除所有不再需要的依赖

# --- 服务管理 ---

# 列出所有通过 Homebrew 安装的服务（如数据库、Web 服务器）
brew services list

# 启动服务并设置开机自启
brew services start <服务名>
```

```
# 停止服务
brew services stop <服务名>

# 重启服务
brew services restart <服务名>

# 仅运行服务（不设置开机自启）
brew services run <服务名>

# ---- 仓库管理 ----

# 添加第三方 Tap（仓库源）
brew tap <用户名>/<仓库名>

# 列出已添加的 Tap
brew tap

# 移除 Tap
brew untap <用户名>/<仓库名>

# ---- 锁定版本 ----

# 锁定包版本（防止在 brew upgrade 时被升级）
brew pin <包名>

# 解锁包版本
brew unpin <包名>

# 查看已锁定的包
brew list --pinned
```

1.4 Homebrew Bundle

brew bundle 允许你通过一个 Brewfile 来声明式管理所有 Homebrew 包，实现环境的快速复现。

```
# 从当前目录的 Brewfile 安装所有列出的包
brew bundle

# 导出当前已安装的包到 Brewfile
brew bundle dump

# 清理 Brewfile 中未列出的包（与 Brewfile 保持一致）
brew bundle cleanup

# 检查 Brewfile 状态
brew bundle check

# 指定 Brewfile 路径
brew bundle --file=/path/to/Brewfile
```

1.5 推荐安装的软件列表

命令行工具

brew install git	# 版本控制 (比 macOS 自带的版本更新)
brew install gh	# GitHub CLI, 在终端中直接操作 GitHub
brew install node	# Node.js 运行时
brew install python	# Python 解释器
brew install uv	# 极速 Python 包管理器 (Rust 实现, 替代 pip)
brew install wget	# 文件下载工具
brew install curl	# HTTP 请求工具
brew install jq	# JSON 命令行处理工具
brew install ripgrep	# 极速文本搜索工具 (替代 grep)
brew install fd	# 极速文件搜索工具 (替代 find)
brew install fzf	# 模糊搜索工具
brew install tmux	# 终端复用器
brew install htop	# 交互式进程查看器
brew install tree	# 目录树状显示
brew install neofetch	# 系统信息展示
brew install imagemagick	# 图像处理工具集
brew install ffmpeg	# 音视频处理工具

深度学习与数据科学

PyTorch 请使用 pip/conda 安装 (详见第十五章), Homebrew 不提供 PyTorch

brew install jupyterlab	# JupyterLab 交互式开发环境
-------------------------	----------------------

TensorBoard 请使用 pip install tensorboard (Homebrew 不提供)

数据库客户端

brew install postgresql@16	# PostgreSQL 客户端与服务器
brew install redis	# Redis 客户端与服务器
brew install mongosh	# MongoDB Shell (mongosh)
brew install mysql-client	# MySQL 客户端

现代化 CLI 替代品 (提升日常命令行体验)

brew install bat	# 语法高亮的 cat 替代品
brew install git-delta	# 更好看的 git diff viewer (命令为 delta)
brew install lazygit	# Git 终端 UI (TUI)
brew install zoxide	# 智能目录跳转 (cd 替代)
brew install direnv	# 目录级环境变量自动加载
brew install just	# 现代命令运行器 (Make 替代)
brew install starship	# 跨 Shell 美化提示符
brew install lsd	# 美化版 ls (图标 + 颜色)
brew install duf	# 美化版 df (磁盘使用)
brew install bottom	# 图形化进程/资源监控 (btm)
brew install gdu	# 美化版 du (磁盘使用分析)
brew install hyperfine	# 命令行基准性能测试
brew install tokei	# 代码行数统计 (比 cloc 更快)
brew install watchexec	# 文件变更时自动执行命令

API 与网络调试工具

brew install websocat	# WebSocket 命令行客户端
brew install grpcurl	# gRPC 命令行客户端
brew install graphql	# GraphQL 命令行客户端 (或 npm install -g graphql)
brew install mitmproxy	# HTTP/HTTPS 中间人调试代理
brew install httpie	# 更友好的 curl 替代品

MLOps 工具

brew install dvc	# 数据版本控制 (Data Version Control)
brew install act	# 本地运行 GitHub Actions
brew install mlflow	# ML 实验追踪与模型管理

编辑器

brew install neovim	# 现代化的 Vim 重构版本
brew install vim	# 经典 Vim 编辑器 (比系统自带版本新)

本地服务

brew install nginx	# 高性能 HTTP 服务器 / 反向代理
--------------------	-----------------------

GUI 应用

brew install --cask visual-studio-code	# VS Code 编辑器
brew install --cask iterm2	# iTerm2 终端模拟器
brew install --cask google-chrome	# Google Chrome 浏览器
brew install --cask docker	# Docker Desktop
brew install --cask postman	# API 调试工具
brew install --cask rectangle	# 窗口管理工具
brew install --cask obsidian	# 笔记与知识管理
brew install --cask notion	# 协作笔记
brew install --cask slack	# 团队通讯
brew install --cask discord	# 语音和文字聊天
brew install --cask spotify	# 音乐
brew install --cask vlc	# 视频播放器
brew install --cask the-unarchiver	# 解压工具
brew install --cask alfred	# 效率启动器
brew install --cask raycast	# 效率启动器 (Alfred 替代)
brew install --cask warp	# 现代化的终端
brew install --cask db-browser-for-sqlite	# SQLite 图形化浏览器

二、iTerm2 终端利器

2.1 窗口与面板管理

功能	快捷键	详细说明
垂直拆分面板	Cmd + D	将当前终端面板左右拆分为两个独立的终端会话。每个面板都是完全独立的 Shell 环境，互不影响。可以无限嵌套拆分。
水平拆分面板	Cmd + Shift + D	将当前终端面板上下拆分为两个独立的终端会话。
切换面板	Cmd + [/ Cmd +]	在拆分的多个终端面板之间按顺序切换焦点。Cmd + [向左或向上切换，Cmd +] 向右或向下切换。
方向键切换面板	Cmd + Option + ↑/↓/←/→	按照物理方向（上下左右）切换面板焦点。比顺序切换更直观。
最大化/恢复面板	Cmd + Shift + Enter	将当前面板临时最大化占据整个窗口，方便查看长输出。再次按恢复原布局。
关闭当前面板	Cmd + W	关闭当前获得焦点的终端面板。面板中的进程也会被终止。
新建标签页	Cmd + T	创建一个新的终端标签页（Tab）。标签页是独立的，不参与面板拆分。
切换标签页	Cmd + ←/→ 或 Cmd + 数字键	在不同标签页之间切换。Cmd + 1 切换到第一个标签页，依此类推。
关闭当前标签页	Cmd + W（无拆分的标签页）	关闭当前的标签页。
移动标签页	Cmd + Shift + ←/→	将当前标签页在标签栏中左右移动位置。
重新打开关闭的标签页	Cmd + Z（在关闭后立即按）	撤销关闭标签页操作。

2.2 文本操作与搜索

功能	快捷键	详细说明
终端内搜索	Cmd + F	在当前终端面板的输出缓冲区中搜索特定字符串或正则表达式。搜索范围覆盖当前面板的所有可滚动历史输出，不限于可见区域。
搜索	Cmd + G	跳转到搜索结果的下一处匹配。

下一个		
搜索上一个	Cmd + Shift + G	跳转到搜索结果的上一处匹配。
清屏	Cmd + K	清除当前终端面板的所有屏幕输出和回滚缓冲区。比在终端中输入 <code>clear</code> 命令更彻底—— <code>clear</code> 只是将当前可见内容上移，仍可回滚查看；Cmd + K 会清空整个回滚历史。
清空至行首	Ctrl + U	在终端命令行中，删除当前输入行中光标之前的所有字符。
清空至行尾	Ctrl + K	在终端命令行中，删除当前输入行中光标之后的所有字符。
删除前一个词	Ctrl + W	在终端命令行中，向前删除一个完整的词（以空格分隔）。
光标移动到行首	Ctrl + A	将光标移动到当前命令行的开头。在修改长命令时极其常用。
光标移动到行尾	Ctrl + E	将光标移动到当前命令行的末尾。
撤销终端操作	Ctrl + -	在终端中撤销上一次的文本编辑操作。
复制	Cmd + C（选中即复制）	iTerm2 默认开启了「选中即复制」功能，用鼠标或键盘选中文本后自动复制到剪贴板，无需额外按快捷键。
粘贴	Cmd + V	将剪贴板内容粘贴到终端。
选择文本	Shift + 方向键	用键盘逐字符选择终端中的文本。Option + Shift + 方向键 按词选择。
智能选择	双击（选词）、三击（选整行）、四击（选整个命令输出）	根据点击次数智能扩展选中范围。在 URL、路径、文件名等特殊文本上双击会选中完整的 URL 或路径。

2.3 高级配置与技巧

Hotkey Window (热键窗口) :

这是 iTerm2 最强大的功能之一。在 Preferences → Keys → Hotkey 中设置一个全局快捷键 (推荐 Option + Space)，可以在任何桌面、任何应用之上以一个半透明浮动窗口的形式快速调出或隐藏 iTerm2 终端，无需切换到桌面或 Dock。对于需要频繁执行终端命令的开发者来说，这极大地减少了上下文切换成本。

Profile 配置文件:

在 Preferences → Profiles 中可以为不同用途创建专门的终端配置文件，每个配置文件可以设置独立的：

- 外观：字体、字号、配色方案、透明度、背景图片
- 行为：启动时执行的命令、工作目录
- 终端模拟：字符编码、回滚行数（建议设置为「无限回滚」以便查看完整编译日志）
- 按键映射：自定义快捷键映射
- 高级：语义历史、触发器（根据输出自动高亮、标记、触发动作）

常用 Shell 集成:

在 iTerm2 菜单中选择 Install Shell Integration 可以为当前 Shell 安装集成功能，包括：Toolbelt（在输出中点击文件名直接跳转）、命令执行时间标记、上一个命令的返回码着色提示、命令自动完成等。

Tmux 集成:

iTerm2 对 tmux 有原生集成支持。使用 `tmux -CC` 命令启动 tmux 会话时，iTerm2 会将 tmux 的窗口和面板映射为原生的 iTerm2 标签页和面板，同时保留 tmux 的会话持久化能力。

复制模式 (Copy Mode):

按 Cmd + Shift + C 进入复制模式，可以像在文本编辑器中一样使用键盘快捷键（Vim 风格或 Emacs 风格）在终端输出中导航、选择、搜索和复制文本，完全脱离鼠标。

即时重播 (Instant Replay):

按 Cmd + Option + B 可以进入即时重播模式，回放最近终端输出的历史画面，就像视频回放一样。适合回溯查看快速闪过的重要输出。

密码管理器:

iTerm2 内置了密码管理器，可以为需要密码的 SSH 连接、sudo 命令等自动填充密码。密码存储在 macOS 钥匙串中。

2.4 常用终端命令

命令	说明
<code>clear</code>	清除当前屏幕可见内容（可回滚查看）
<code>reset</code>	完全重置终端会话状态，修复终端显示异常
<code>history</code>	显示当前 Shell 会话的命令历史记录
<code>!n</code>	执行历史记录中编号为 <code>n</code> 的命令
<code>!!</code>	重新执行上一条命令
<code>!\$</code>	引用上一条命令的最后一个参数
<code>ctrl + r</code>	在 Shell 中进入反向增量搜索模式，输入关键词可实时匹配历史命令
<code>open .</code>	在 Finder 中打开当前目录
<code>open <文件名></code>	用默认关联应用打开文件
<code>pbcopy / pbpaste</code>	与 macOS 系统剪贴板交互—— <code>pbcopy</code> 将管道输入复制到剪贴板， <code>pbpaste</code> 将剪贴板内容输出到终端
<code>say "文本"</code>	让 Mac 朗读指定文本
<code>mdfind</code>	Spotlight 命令行搜索工具，比 <code>find</code> 更快地按文件名搜索
<code>caffeinate</code>	防止 Mac 进入休眠状态，适合长时间运行的任务

2.5 *tmux* 终端复用器深入指南

tmux (Terminal Multiplexer) 是最强大的终端生产力工具之一。它允许你在一个终端窗口中创建多个会话、窗口和面板，即使关闭终端或断开 SSH 连接，会话中的程序仍然在后台继续运行。对于需要长时间运行训练任务、同时管理多台服务器的开发者来说，*tmux* 是必不可少的工具。

核心概念三层次：

- **Session（会话）**：顶层容器，一个 *tmux* 会话包含多个窗口。你可以同时运行多个互不干扰的会话。典型用法：一个项目一个会话。
- **Window（窗口）**：相当于终端中的「标签页」，一个会话可以有多个窗口，每个窗口占据整个屏幕。
- **Pane（面板）**：对窗口进行拆分，一个窗口可以有多个面板，每个面板都是一个独立的 Shell。

```
# 安装
brew install tmux
```

会话管理：

```
# 创建新的命名会话
tmux new-session -s <会话名>
```



```
# 简写
tmux new -s myproject

# 列出所有会话
tmux list-sessions
# 简写
tmux ls

# 连接到（恢复）已有会话
tmux attach-session -t <会话名>
# 简写
tmux attach -t myproject
tmux a -t myproject

# 断开当前会话（session 继续在后台运行）
# 快捷键：Prefix + d（先按 Ctrl+b，松开后再按 d）

# 杀死指定会话
tmux kill-session -t <会话名>

# 重命名会话
tmux rename-session -t <旧名> <新名>
# 快捷键：Prefix + $

# 切换会话（不离开 tmux）
# 快捷键：Prefix + s（显示会话列表，方向键选择）

# 创建后台会话并执行命令（适合脚本化）
tmux new-session -d -s mysession "python train.py"
```

前缀键 (Prefix Key):

tmux 的所有快捷键都需要先按「前缀键」（默认 Ctrl + B），松开后再按功能键。例如 Ctrl+B 然后 % 表示垂直拆分面板。

功能	快捷键	说明
垂直拆分面板	Prefix + %	左右拆分当前面板
水平拆分面板	Prefix + "	上下拆分当前面板
切换面板	Prefix + ↑/↓/←/→	方向键切换焦点面板
关闭面板	Prefix + x (然后按 y 确认)	关闭当前面板
切换面板全屏	Prefix + z	当前面板临时最大化/恢复
新建窗口	Prefix + c	创建新窗口
切换窗口	Prefix + 数字键	切换到第 N 个窗口
切换上一个/下一个窗口	Prefix + p / Prefix + n	顺序切换窗口
窗口列表	Prefix + w	可视化窗口列表
重命名窗口	Prefix + ,	重命名当前窗口
分离会话	Prefix + d	离开 tmux 但保持后台运行
滚动模式	Prefix + [	进入滚动/复制模式，可用方向键或 PageUp/Down 回看历史输出，按 Q 退出
搜索历史	Prefix + [然后 Ctrl+S	在滚动模式中向前搜索
帮助	Prefix + ?	显示所有快捷键列表

常用 **tmux** 配置 (~/.tmux.conf) :

```
# 将前缀键从 Ctrl+B 改为 Ctrl+A (更顺手, 类 GNU Screen)
set -g prefix C-a
unbind C-b
bind C-a send-prefix

# 使用 vi 风格的按键在滚动模式和复制模式中操作
setw -g mode-keys vi

# 从 1 开始编号窗口 (而非从 0 开始)
set -g base-index 1
setw -g pane-base-index 1

# 鼠标支持 (点击切换面板、拖动调整面板大小、滚轮回看历史)
set -g mouse on

# 增加回滚行数
set -g history-limit 50000
```

```
# 更直观的面板切换 (Alt + 方向键, 不需要前缀)
bind -n M-Left select-pane -L
bind -n M-Right select-pane -R
bind -n M-Up select-pane -U
bind -n M-Down select-pane -D

# 重新加载配置
bind r source-file ~/.tmux.conf \; display "Config reloaded!"
```

iTerm2 原生 tmux 集成:

iTerm2 对 tmux 有原生支持——使用 `tmux -CC` 启动会话时, tmux 的窗口和面板会被映射为原生的 iTerm2 标签页和面板:

```
# 在 iTerm2 中以集成模式创建新会话
tmux -CC new-session -s mysession

# 在 iTerm2 中以集成模式连接到已有会话
tmux -CC attach -t mysession
```

集成模式下仍然保留 tmux 的会话持久化能力, 但操作方式完全变成了 iTerm2 原生体验。非常适合不习惯 tmux 快捷键但又需要会话持久化的用户。

推荐插件:

- **tmux-resurrect**: 保存和恢复 tmux 会话状态 (窗口布局、工作目录等), 重启后一键恢复
- **tmux-continuum**: 自动保存和恢复 tmux 会话
- **tmux-yank**: 增强复制功能, 自动复制到 macOS 系统剪贴板

插件管理器推荐使用 TPM (Tmux Plugin Manager):

```
git clone https://github.com/tmux-plugins/tpm ~/.tmux/plugins/tpm
# 在 ~/.tmux.conf 中添加插件配置
# 按 Prefix + I 安装插件, Prefix + U 更新插件
```

2.6 Vim / Neovim 基础求生指南

对于开发者来说, 掌握 Vim 的基础操作是必要的——远程服务器上编辑配置文件、在终端中快速修改代码、使用 Git 的交互式 rebase, 这些场景都绕不开 Vim。

为什么需要会一点 Vim:

- `git commit / git rebase -i` 默认打开 Vim
- SSH 到远程服务器编辑文件, Vim 几乎是唯一可靠的选择
- 掌握了 Vim 后, 可以在 VS Code、iTerm2 等几乎所有编辑器中启用 Vim 模式

安装:

```
# Neovim (推荐, 现代化 Vim 重构版)
brew install neovim
# 创建别名方便使用
alias vim="nvim"
alias vi="nvim"
```

Vim 的四种模式：

模式	进入方式	用途
Normal（普通模式）	按 Esc	默认模式，用于导航和执行编辑命令
Insert（插入模式）	i、a、o 等	正常输入文本
Visual（可视模式）	v、V、Ctrl+V	选中文本块
Command（命令模式）	:	执行保存、退出、搜索替换等命令

最小求生命令集——记住这些就够了：

```
# --- 打开/保存/退出 ---
vim filename      # 打开文件
:q                # 退出
:q!               # 不保存强制退出
:w               # 保存
:wq 或 :x         # 保存并退出
ZZ                # 保存并退出 (Normal 模式快捷键)

# --- 进入编辑模式 ---
i                # 光标前插入
a                # 光标后插入
o                # 下方新开一行插入
O                # 上方新开一行插入

# --- 导航 (Normal 模式) ---
h/j/k/l          # 左/下/上/右移动 (替代方向键, 减少手指移动)
w                # 跳到下一个词首
b                # 跳到上一个词首
0                # 跳到行首
$                # 跳到行尾
gg               # 跳到文件开头
G                # 跳到文件末尾
行号 + G         # 跳到指定行 (如 25G 跳到第 25 行)
Ctrl+U / Ctrl+D  # 向上/向下翻半页
Ctrl+B / Ctrl+F  # 向上/向下翻整页

# --- 编辑 ---
x                # 删除光标处字符
dd               # 删除整行
yy               # 复制整行
p                # 粘贴到光标后
P                # 粘贴到光标前
u                # 撤销
```

```

Ctrl+R          # 重做
.               # 重复上一次操作

# --- 搜索替换 ---
/pattern        # 向前搜索 (n=下一个, N=上一个)
?pattern        # 向后搜索
:%s/old/new/g   # 全文替换
:%s/old/new/gc  # 全文替换 (每次确认)
:5,20s/old/new/g # 在第 5-20 行替换

# --- 可视模式 ---
v              # 按字符选择
V              # 按行选择
Ctrl+V        # 按列 (块) 选择
# 选中后可以用 d 删除、y 复制、> 缩进等

# --- 窗口分割 ---
:split         # 水平分割 (或 :sp)
:vsplit        # 垂直分割 (或 :vsp)
Ctrl+W + 方向键 # 切换分割窗口
Ctrl+W + q     # 关闭当前窗口

```

Neovim 基本配置 (~/.config/nvim/init.lua) :

```

-- 行号显示
vim.opt.number = true
vim.opt.relativenumber = true

-- 缩进设置
vim.opt.tabstop = 4
vim.opt.shiftwidth = 4
vim.opt.expandtab = true

-- 搜索设置
vim.opt.ignorecase = true
vim.opt.smartcase = true

-- 鼠标支持
vim.opt.mouse = 'a'

-- 系统剪贴板 (macOS 使用 unnamed, Linux 使用 unnamedplus)
vim.opt.clipboard = 'unnamed'

-- 启用语法高亮
vim.cmd('syntax on')

```

在 VS Code 中使用 Vim 键绑定:

安装 VS Code 扩展 `vscodevim.vim` 即可在 VS Code 中获得 Vim 的编辑体验——编辑时使用 Vim 快捷键，同时保留 VS Code 的所有现代 IDE 功能（智能补全、文件导航等）。这是很多开发者的理想组合。

三、Zsh Shell 与 Oh My Zsh

3.1 简介

从 macOS Catalina (10.15) 开始，Zsh (Z Shell) 取代 Bash 成为 macOS 的默认 Shell。Zsh 与 Bash 高度兼容（绝大多数 Bash 脚本可以直接在 Zsh 中运行），但提供了大量增强功能：更好的自动补全、拼写纠正、路径展开、主题支持、插件系统等。

Oh My Zsh 是 Zsh 最流行的社区配置框架，提供了数百个插件和主题。安装 Oh My Zsh 可以将 Zsh 的配置成本从数小时降到几分钟，让你立即享受到一套精心调校的终端体验。

3.2 安装 Oh My Zsh

```
# 安装 Oh My Zsh
sh -c "$(curl -fsSL
https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"

# 配置文件位置
# ~/.zshrc - Zsh 主配置文件 (Oh My Zsh 会修改此文件)
# ~/.oh-my-zsh/ - Oh My Zsh 安装目录
# ~/.oh-my-zsh/themes/ - 主题目录
# ~/.oh-my-zsh/plugins/ - 插件目录
```

3.3 主题配置

在 ~/.zshrc 中修改 ZSH_THEME 变量来切换主题：

```
# Oh My Zsh 默认主题
ZSH_THEME="robbyrussell"

# 推荐的流行主题
ZSH_THEME="agnoster"           # 需要安装 Powerline 字体，显示 Git 分支和状态
ZSH_THEME="powerlevel10k/powerlevel10k" # 极受欢迎的现代主题，高度可定制
```

Powerlevel10k 安装（推荐）：

```
# 克隆到 Oh My Zsh 自定义主题目录
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git ${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k

# 在 ~/.zshrc 中设置
ZSH_THEME="powerlevel10k/powerlevel10k"

# 安装推荐字体 (MesloLGS NF)
# 下载并安装 MesloLGS NF 字体，然后在终端模拟器中设置为默认字体

# 重启终端，会自动运行配置向导
```

3.4 插件配置

Oh My Zsh 自带大量官方插件，社区也提供了海量第三方插件。插件通过修改 ~/.zshrc 中的 `plugins` 数组来启用：

```
plugins=(
  git
  docker
  docker-compose
  python
  vscode
  zsh-autosuggestions
  zsh-syntax-highlighting
  z
  extract
  web-search
  copypath
  copyfile
  dirhistory
  history
  colored-man-pages
)
```

常用插件详解：

插件名	类型	功能说明
git	内置	提供大量 Git 别名（如 <code>gst = git status</code> 、 <code>ga = git add</code> 、 <code>gcmsg = git commit -m</code> 等数十个）和 Git 信息展示
docker	内置	Docker 命令自动补全
docker-compose	内置	docker-compose 命令自动补全和别名
python	内置	Python 相关别名和自动补全
vscode	内置	VS Code 相关别名（ <code>vsc = code .</code> 、 <code>vsc = code --goto</code> 等）
z	内置	智能目录跳转——根据访问频率和历史， <code>z <部分目录名></code> 即可跳转到最匹配的目录，无需输入完整路径
extract	内置	通用解压命令—— <code>extract <文件名></code> 自动识别压缩格式并解压（支持 tar、zip、gz、bz2、rar、7z 等）
web-search	内置	从终端直接搜索—— <code>google <关键词></code> 、 <code>github <关键词></code> 、 <code>stackoverflow <关键词></code> 等
copypath	内置	copypath 复制当前目录绝对路径到剪贴板
copyfile	内置	copyfile <文件名> 复制文件内容到剪贴板
dirhistory	内置	<code>Alt + ←/→</code> 在目录访问历史中前后跳转
history	内置	提供历史命令操作别名（ <code>h = history</code> 、 <code>hs = 搜索历史</code> 等）
colored-man-pages	内置	为 man 帮助手册页面添加语法着色，可读性大幅提升
zsh-autosuggestions	第三方	根据历史命令自动建议补全（灰色文字显示建议），按 <code>→</code> 接受。极其提升效率的插件
zsh-syntax-highlighting	第三方	命令行实时语法高亮——有效命令显示为绿色，无效命令显示为红色，字符串和路径分别着色

安装第三方插件：


```
# zsh-autosuggestions
git clone https://github.com/zsh-users/zsh-autosuggestions ${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zsh-autosuggestions

# zsh-syntax-highlighting
git clone https://github.com/zsh-users/zsh-syntax-highlighting.git ${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zsh-syntax-highlighting
```

3.5 Zsh 原生快捷键

快捷键	功能
Ctrl + A	移动光标到行首
Ctrl + E	移动光标到行尾
Ctrl + U	删除光标前所有字符
Ctrl + K	删除光标后所有字符
Ctrl + W	向后删除一个词
Ctrl + Y	粘贴刚才删除的内容
Ctrl + R	反向增量搜索历史命令
Ctrl + T	交换光标前后两个字符的位置
Esc + Backspace	向后删除一个词
Esc + D	向前删除一个词
Esc + F	向前移动一个词
Esc + B	向后移动一个词
Tab	自动补全（Zsh 的补全比 Bash 强大得多，支持模糊匹配和菜单选择）
Ctrl + X Ctrl + E	在默认编辑器中编辑当前命令行

3.6 .zshrc 常用自定义配置

```
# 启用大小写不敏感的自动补全
zstyle ':completion:*' matcher-list 'm:{a-z}={A-Z}'

# 补全结果使用颜色高亮
```

```
zstyle ':completion:*' list-colors "${(s..)LS_COLORS}"

# 在补全菜单中使用方向键选择
zstyle ':completion:*' menu select

# 历史记录设置
HISTSIZE=10000                                # 内存中保存的历史命令数量
SAVEHIST=10000                                # 保存到历史文件中的命令数量
HISTFILE=~/.zsh_history                       # 历史文件位置
setopt SHARE_HISTORY                          # 多个终端会话间共享历史
setopt HIST_IGNORE_DUPS                       # 忽略重复的历史命令
setopt HIST_IGNORE_SPACE                     # 以空格开头的命令不保存到历史

# 常用别名
alias ll="ls -la"
alias la="ls -A"
alias l="ls -CF"
alias cls="clear"
alias ..="cd .."
alias ...="cd ../.."
alias ....="cd ../../.."
alias zshrc="code ~/.zshrc"
alias reload="source ~/.zshrc"
alias python="python3"
alias pip="pip3"
alias ip="ipconfig getifaddr en0"             # 查看本机局域网 IP
alias pubip="curl -s ifconfig.me"            # 查看本机公网 IP

# Git 别名
alias gs="git status"
alias ga="git add"
alias gc="git commit"
alias gp="git push"
alias gl="git pull"
alias gd="git diff"
alias glog="git log --oneline --graph"
alias gco="git checkout"
alias gb="git branch"
```

四、SSH 远程连接与密钥管理

4.1 SSH 基本概念

SSH (Secure Shell) 是一种加密的网络协议，用于在不安全的网络中安全地访问远程计算机和执行命令。对开发者而言，SSH 是连接远程服务器、推送代码到 GitHub/GitLab、管理云服务器器的基础技能。

SSH 的核心概念：

- 非对称加密：SSH 使用公钥/私钥对进行身份验证。公钥放在服务器上（你锁的门），私钥保存在本地（你的钥匙）。私钥永远不能分享给任何人。
- 密钥类型：常见的 SSH 密钥算法包括 `ed25519`（推荐，更高安全性，更小体积）、`rsa`（传统选择，兼容性最好）、`ecdsa`（中等选择）。
- `known_hosts` 文件：存储已连接过的服务器的公钥指纹，用于在下次连接时验证服务器身份，防止中间人攻击。
- `authorized_keys` 文件：服务器上存放允许登录的客户端公钥列表，一行一个公钥。
- **SSH Agent**：在本地运行的服务，缓存解密后的私钥，避免每次使用 SSH 时都需要输入私钥的密码短语。

4.2 生成 SSH 密钥

```
# 生成 Ed25519 密钥（推荐，现代且安全）
ssh-keygen -t ed25519 -C "your_email@example.com"

# 生成 4096 位 RSA 密钥（兼容性最好，某些老旧系统可能不支持 Ed25519）
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"

# 指定密钥文件路径（避免覆盖已有密钥）
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_github -C "your_email@example.com"

# 生成时不设置密码短语（不推荐，安全性降低）
ssh-keygen -t ed25519 -C "your_email@example.com" -N ""

# 查看密钥指纹
ssh-keygen -lf ~/.ssh/id_ed25519.pub

# 更改已有私钥的密码短语
ssh-keygen -p -f ~/.ssh/id_ed25519

# 从私钥中提取公钥（如果丢失了公钥文件）
ssh-keygen -y -f ~/.ssh/id_ed25519 > ~/.ssh/id_ed25519.pub
```

密钥文件说明：

- `id_ed25519` — 私钥（绝密，权限应为 `600`，即仅所有者可读写）

- `id_ed25519.pub` — 公钥（可以安全分享，权限通常为 644）
- `id_ed25519` 也可用于导出 OpenSSH 公钥格式和 PEM 格式，在不同系统间转换

SSH 目录与文件权限（安全要求极其严格）：

SSH 对文件权限非常敏感，权限设置不当会导致 SSH 拒绝使用密钥并回退到密码认证。正确的权限设置如下：

```
# 修复权限（如果遇到 SSH 密钥权限问题）
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519
chmod 644 ~/.ssh/id_ed25519.pub
chmod 600 ~/.ssh/authorized_keys
chmod 644 ~/.ssh/known_hosts
chmod 600 ~/.ssh/config

# SSH 目录：仅所有者可读写执行
# 私钥文件：仅所有者可读写
# 公钥文件：所有者可读写，其他人可读
# 服务器端授权密钥文件
# 已知主机文件
# SSH 配置文件（如果包含敏感信息）
```

如果不修复权限，可能会遇到以下错误：

- `Permissions 0644 for 'id_ed25519' are too open` — 私钥权限太过宽松
- `Bad owner or permissions on ~/.ssh/config` — SSH 配置文件权限不正确
- 静默回退到密码认证，即使密钥已正确配置

4.3 SSH 配置文件

`~/.ssh/config` 文件允许你为不同的远程主机设置不同的连接参数（主机别名、用户名、端口、密钥文件等），避免每次连接时输入冗长的命令行参数。

```
# 基本配置结构
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519_github
    IdentitiesOnly yes

Host myserver
    HostName 192.168.1.100
    User admin
    Port 2222
    IdentityFile ~/.ssh/id_ed25519
    ServerAliveInterval 60
    ServerAliveCountMax 3

Host dev-*
    User developer
    IdentityFile ~/.ssh/id_ed25519_dev
    ForwardAgent yes
    Compression yes

Host *
    AddKeysToAgent yes
    UseKeychain yes
```

IdentityFile ~/.ssh/id_ed25519

常用 SSH 配置参数说明：

参数	说明
Host	定义主机别名（支持通配符*）
HostName	实际的服务器地址（域名或 IP）
User	登录用户名
Port	SSH 端口（默认 22）
IdentityFile	指定使用的私钥文件路径
IdentitiesOnly yes	仅使用配置文件指定的密钥，不尝试 SSH Agent 中的其他密钥
AddKeysToAgent yes	自动将私钥添加到 SSH Agent
UseKeychain yes	在 macOS 钥匙串中存储密钥密码短语
ForwardAgent yes	允许从远程服务器使用本地的 SSH Agent（跳板机场景）
ServerAliveInterval	保活信号发送间隔（秒），防止长时间无操作时连接断开
ServerAliveCountMax	保活信号无响应的最大次数，超过后断开连接
Compression yes	启用压缩（在慢网络下有助于提升速度）
StrictHostKeyChecking	主机密钥检查策略（yes=严格, no=不检查, accept-new=自动接受新主机）
ConnectTimeout	连接超时时间（秒）
ProxyJump 或 ProxyCommand	通过跳板机（Bastion Host）连接目标服务器

4.4 SSH Agent 管理

```
# 启动 SSH Agent (macOS 通常在登录时自动启动)
eval "$(ssh-agent -s)"

# 将私钥添加到 Agent
ssh-add ~/.ssh/id_ed25519

# 添加时在钥匙串中存储密码短语 (macOS 特有)
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
# 旧版 macOS 使用
ssh-add -K ~/.ssh/id_ed25519

# 列出已加载的密钥
```

```
ssh-add -l

# 列出已加载密钥的公钥
ssh-add -L

# 删除所有已加载的密钥
ssh-add -D

# 删除指定密钥
ssh-add -d ~/.ssh/id_ed25519

# 锁定/解锁 Agent (需要设置密码)
ssh-add -x
ssh-add -X
```

4.5 常用 SSH 命令

```
# 基本的远程连接
ssh user@hostname

# 使用指定端口连接
ssh -p 2222 user@hostname

# 使用指定私钥连接
ssh -i ~/.ssh/id_ed25519 user@hostname

# 在远程服务器上执行单个命令 (不打开交互 Shell)
ssh user@hostname "ls -la /var/www"

# 在远程服务器上执行本地脚本
ssh user@hostname "bash -s" < local_script.sh

# 通过跳板机连接目标服务器
ssh -J user@bastion_host user@target_host

# 或使用 ProxyJump 配置
ssh -J user@bastion-host:2222 user@target-host

# 本地端口转发 (将本地端口流量通过 SSH 隧道转发到远程)
ssh -L 本地端口:目标主机:目标端口 user@ssh-server
# 例: 将本地 8080 端口转发到 remote-server 的 80 端口
ssh -L 8080:remote-server:80 user@ssh-server

# 远程端口转发 (将远程端口流量转发到本地)
ssh -R 远程端口:本地主机:本地端口 user@ssh-server

# 动态端口转发 (SOCKS 代理)
ssh -D 1080 user@ssh-server

# 测试 SSH 连接 (不执行命令)
ssh -T user@hostname
```

```
# 详细输出模式（用于调试连接问题）
ssh -vvv user@hostname

# 复制公钥到远程服务器（一步设置免密登录）
ssh-copy-id user@hostname

# 或手动复制
cat ~/.ssh/id_ed25519.pub | ssh user@hostname "mkdir -p ~/.ssh && cat >>
~/.ssh/authorized_keys"
```

4.6 SCP 与 SFTP

```
# --- SCP（安全复制） ---

# 从本地复制文件到远程
scp local_file.txt user@hostname:/remote/path/

# 从远程复制文件到本地
scp user@hostname:/remote/path/file.txt ./local/path/

# 复制整个目录（递归）
scp -r local_directory/ user@hostname:/remote/path/

# 指定端口
scp -P 2222 file.txt user@hostname:/remote/path/

# 压缩传输（在慢网络上加速）
scp -C large_file.txt user@hostname:/remote/path/

# 保留文件属性（权限、时间戳等）
scp -p file.txt user@hostname:/remote/path/

# --- SFTP（SSH 文件传输协议） ---

# 连接到远程
sftp user@hostname

# SFTP 交互命令：
# ls - 列出远程目录
# cd - 切换远程目录
# ll - 列出本地目录
# lcd - 切换本地目录
# get - 下载文件
# put - 上传文件
# mget - 批量下载（支持通配符）
# mput - 批量上传（支持通配符）
# mkdir - 创建远程目录
# rm - 删除远程文件
# rmdir - 删除远程目录
# pwd - 显示远程当前目录
```

```
# lpwd - 显示本地当前目录
# exit - 退出
```

4.7 GitHub SSH 配置

```
# 测试 GitHub SSH 连接
ssh -T git@github.com

# 如果成功, 输出应为:
# Hi <username>! You've successfully authenticated, but GitHub does not provide shell
access.

# 如果之前使用 HTTPS 克隆的仓库, 可以切换为 SSH
git remote set-url origin git@github.com:username/repository.git
```


五、VS Code 快捷键与使用技巧

5.1 命令与导航

功能	快捷键	详细说明
命令面板 (Command Palette)	Cmd + Shift + P	调出所有 VS Code 命令的入口。可以执行任何操作：打开设置、安装扩展、运行任务、切换主题、格式化文档等。输入 > 后直接键入命令关键词即可模糊匹配。输入 @ 可以跳转到当前文件内的符号，输入 : 可以跳转到指定行号。
快速打开文件 (Quick Open)	Cmd + P	在当前工作区内全局搜索文件名并极速跳转。支持模糊匹配，输入文件名的部分字母即可定位。在已打开的文件之间切换时，按 Cmd + P 后直接按 Enter 可在最近使用的两个文件间快速切换。
全局查找与替换	Cmd + Shift + F	在整个项目目录中搜索特定字符串或正则表达式。搜索结果会显示在左侧搜索面板中，支持区分大小写、全词匹配、正则表达式三种搜索模式。在搜索结果中可以直接批量替换，支持逐个确认替换或全部替换。
全局搜索符号	Cmd + T	在整个工作区中搜索类、函数、变量等符号定义，而不只是文件名。比 Cmd + P 更精确，直接定位到代码逻辑。
转到定义	F12	跳转到当前光标所在符号的定义位置。对于阅读陌生代码库极其有用。
查看定义 (Peek)	Option + F12	在浮窗中预览定义内容，而不离开当前文件。适合快速查看函数签名或变量类型。
转到引用	Shift + F12	查看当前符号在项目中的所有引用位置。在重构代码时非常实用，可以确认修改影响范围。
前进/后退导航	Ctrl + - / Ctrl + Shift + -	在浏览过的位置之间前后跳转，类似于浏览器的前进后退按钮。
切换侧边栏	Cmd + B	隐藏或显示左侧资源管理器（文件树），最大化代码编辑区域。对于小屏幕笔记本尤为重要。
切换面板 (Panel)	Cmd + J	显示或隐藏底部面板（终端、输出、问题、调试控制台等）。
切换终端	Ctrl + ``	在编辑器中直接打开或切换到集成终端。终端会在当前文件所在目录打开。
快速打开最近项目	Cmd + R (在 Quick Open 中)	在 Cmd + P 面板中输入 Cmd + R 可以快速切换到最近打开过的项目。
聚焦资源管理器	Cmd + Shift + E	将键盘焦点切换到左侧文件资源管理器（文件树），之后可以用方向键导航文件列表，Enter 打开文件，Space 预览文件。
聚焦源代码管理	Ctrl + Shift + G	将焦点切换到源代码管理面板（Git 面板），快速查看变更文件列表、暂存或提交。

聚焦扩展面板	Cmd + Shift + X	打开并聚焦到扩展市场面板，快速搜索、安装和管理扩展。
聚焦运行与调试	Cmd + Shift + D	打开并聚焦运行与调试面板，快速选择调试配置并启动调试。
聚焦终端面板	Ctrl + `	打开或切换到集成终端面板，如果终端已打开则在上次活动的终端和编辑器之间切换焦点。
切换编辑器标签页	Cmd + Shift + [/ Cmd + Shift +]	在已打开的编辑器标签页之间按顺序前后切换。也可使用 Ctrl + Tab 按最近使用顺序切换。
关闭所有编辑器	Cmd + K Cmd + W	一次性关闭当前编辑器组中所有打开的标签页。
快速打开 Settings	Cmd + ,	打开 VS Code 的设置界面（GUI 版或 JSON 版）。在命令面板中选择 Preferences: Open User Settings (JSON) 可直接编辑 JSON 配置文件。
键盘快捷键参考	Cmd + K Cmd + S	打开完整的键盘快捷键列表，支持搜索和自定义重新绑定。这是探索和学习 VS Code 快捷键的最佳入口。

5.2 编辑器操作

功能	快捷键	详细说明
拆分编辑器	Cmd + \	将当前编辑器窗口垂直拆分为左右两个，便于同时查看和编辑两个文件。可以多次拆分，最多可同时显示多个编辑器窗口。
切换编辑器组	Cmd + 1/2/3...	快速将焦点切换到第 N 个编辑器组。当同时打开多个拆分窗口时非常高效。
关闭当前编辑器	Cmd + W	关闭当前活动的编辑器标签页。
重新打开已关闭文件	Cmd + Shift + T	撤销关闭操作，重新打开最近关闭的编辑器标签页。可以多次执行以恢复多个文件。
移动编辑器到新组	Ctrl + Cmd + →/←	将当前文件标签页移动到右侧或左侧的编辑器组中。
禅模式 (Zen Mode)	Cmd + K Z	进入全屏无干扰模式，隐藏所有 UI 元素只保留编辑器。按两次 Esc 退出。适合需要极度专注的编码场景。
格式化文档	Shift + Option + F	根据项目配置的格式化工具（如 Prettier、Black、clang-format）自动格式化整个文档。
格式化选中代码	Cmd + K Cmd + F	仅格式化当前选中的代码片段。
折叠/展开代码块	Cmd + Shift + [/ Cmd + Shift +]	折叠或展开当前光标所在的最内层代码块。在浏览大型文件时非常有用。
折叠/展开所有代码块	Cmd + K Cmd + 0 / Cmd + K Cmd + J	一键折叠或展开文件中所有可折叠的代码区域（函数、类、条件语句等）。
切换自动换行	Option + Z	开启或关闭编辑器的自动换行功能。处理长文本或 Markdown 时建议开启。

5.3 多光标与批量编辑

功能	快捷键	详细说明
多光标选择 (向下)	Cmd + D	选中当前光标所在的词，然后持续按此快捷键可依次选中下方下一个相同的词，形成多个光标进行同步批量修改。如果某个匹配项不想选中，按 Cmd + K Cmd + D 可以跳过该项。
多光标选择 (全部)	Cmd + Shift + L	一次性选中文件中所有与当前选中词相同的匹配项，立即建立全文件批量编辑的多光标。
垂直多光标	Cmd + Option + ↑/↓	在当前光标位置向上或向下添加一个新的光标，适合在连续多行的相同列位置进行编辑。
任意位置多光标	Option + 鼠标点击	在任意位置通过鼠标点击添加或移除光标，完全自由地选择需要同步编辑的位置。
选中相同列的多行	Shift + Option + 鼠标拖拽	在列模式下拖拽选中一个矩形区域，适合处理对齐的表格数据或批量缩进调整。
选中下一行/ 上一行	Cmd + Shift + ↓/ ↑	在不使用鼠标的情况下，用键盘在下一行或上一行的相同列位置添加光标。
撤销最后一个光标	Cmd + U	撤销最近一次添加的多光标操作。

5.4 代码编辑

功能	快捷键	详细说明
行注释/ 取消注释	Cmd + /	快速注释或取消注释当前行。选中多行时会对所有选中行执行注释切换。注释符号会根据文件语言自动选择（如 //、#、-- 等）。
块注释	Shift + Option + A	添加或移除块注释（如 /* */、""、<!-- --> 等）。
移动代码 行	Option + ↑/ ↓	将当前行或选中的代码块整体上移或下移一行。在调整代码逻辑顺序时极其方便。
复制代码 行	Option + Shift + ↑/↓	将当前行或选中的代码块复制到上方或下方。相当于原地复制后移动。
删除当前 行	Cmd + Shift + K	删除光标所在的整行（不需要先选中整行）。
在下方插 入空行	Cmd + Enter	在当前行的下方插入一个空行，光标移动到新行。无论光标是否在行尾都能插入。
在上方插 入空行	Cmd + Shift + Enter	在当前行的上方插入一个空行。
缩进/取 消缩进	Cmd +] / Cmd + [	增加或减少当前行或选中代码块的缩进层级。
选中当前 行	Cmd + L	选中光标所在的整行。连续按可以扩展选中到下一行。
选择到词 尾/词首	Shift + Option + →/ ←	按词为单位扩展选中范围。
跳转到匹 配括号	Cmd + Shift + \	在成对的括号（{}、[]、()等）之间跳转光标位置。
触发建议	Ctrl + Space	手动触发代码补全建议列表，在没有自动触发时使用。
参数提示	Cmd + Shift + Space	在函数调用括号内触发参数签名提示，显示函数参数列表和类型。

5.5 调试快捷键

功能	快捷键	详细说明
启动/继续	F5	启动调试会话，或在暂停后继续执行。如果没有配置 <code>launch.json</code> ，VS Code 会自动尝试检测并启动适合当前语言的调试器。
单步跳过	F10	执行当前行代码，不进入函数调用内部。适用于不需要深入查看的函数调用。
单步进入	F11	进入当前行所调用的函数内部。适用于需要追踪函数执行逻辑的情况。
单步跳出	Shift + F11	执行完当前函数的剩余代码并返回到调用方。当你已经获得了足够的信息，想快速返回时使用。
切换断点	F9	在当前行设置或移除断点。断点是调试的核心，程序执行到断点行时会自动暂停。
条件断点	右键 → Add Conditional Breakpoint	设置一个表达式，只有当表达式结果为 <code>true</code> 时才触发断点暂停。避免在循环中反复暂停。
日志点 (Logpoint)	右键 → Add Logpoint	不暂停程序执行，只在调试控制台输出一条日志消息。相当于在代码中插入临时 <code>console.log</code> 而不修改源代码。

5.6 扩展推荐

- **GitLens**: 超级 Git 增强工具，显示每行代码的提交历史、作者、时间，支持行内 blame 注释、文件历史对比、代码热图等功能。对于理解团队代码的演进过程至关重要。
- **GitHub Copilot**: AI 代码补全助手，根据上下文和注释自动生成代码建议。支持多行补全、聊天式问答、代码解释、测试生成等。按 Tab 接受建议，Option +] 查看下一个建议。使用 Cmd + I 打开内联聊天，直接对代码进行自然语言修改。
- **Prettier**: 代码格式化工具，支持 JavaScript、TypeScript、JSON、HTML、CSS、Markdown、YAML 等前端主流格式。建议在设置中开启「保存时自动格式化」选项。
- **ESLint**: JavaScript/TypeScript 代码质量检查工具，即时标注代码中的潜在问题和风格违规。
- **Python / Pylance**: Python 语言支持套装，提供语法高亮、智能补全、类型检查、自动导入、代码导航等功能。
- **Remote - SSH**: 通过 SSH 远程连接到服务器或虚拟机，在 VS Code 中像编辑本地文件一样操作远程文件。
- **Dev Containers**: 在 Docker 容器中打开项目，确保所有开发者使用完全一致的开发环境。
- **Markdown All in One**: Markdown 编辑增强，提供快捷键、自动目录生成、预览等功能。
- **Better Comments**: 为注释添加颜色分类，区分 TODO、FIXME、疑问注释等。
- **Error Lens**: 将编译错误和警告直接内联显示在代码行末尾，无需悬停查看。

六、Git 版本控制完全指南

6.1 核心概念

Git 是一种分布式版本控制系统，每个开发者都拥有完整的代码仓库历史副本。理解以下核心概念是掌握 Git 的前提：

三个工作区域：

1. **工作区 (Working Directory)**：你当前正在电脑上编辑代码的目录。对文件的任何修改都首先发生在这里。
2. **暂存区 (Staging Area / Index)**：一个中间层，用于暂存你准备提交的修改。对应 `git add` 操作。可以将暂存区理解为一个「购物车」或「待办清单」——你可以精心挑选哪些修改要包含在下一次提交中，而不是一股脑地把所有改动都提交。
3. **本地仓库 (Local Repository)**：存储所有历史版本快照的地方，位于项目根目录的隐藏文件夹 `.git` 中。对应 `git commit` 操作。每次提交都会创建一个包含完整项目快照的新对象。

文件的四种状态：

- **未追踪 (Untracked)**：新创建的文件，从未被 `git add` 过，Git 不了解这个文件。
- **已修改 (Modified)**：已被 Git 追踪的文件发生了修改，但尚未暂存。
- **已暂存 (Staged)**：已通过 `git add` 放入暂存区，等待被提交。
- **已提交 (Committed)**：已经安全地保存到本地仓库中，成为一个版本快照。

核心概念：

- **Repository (仓库)**：存放项目代码与完整版本历史记录的地方。分为本地仓库（电脑上的 `.git` 目录）和远程仓库（GitHub/GitLab 等服务器上的 repo）。
- **Commit (提交)**：代码的一个版本快照。每个提交都有一个唯一的 40 位 SHA-1 哈希值作为标识（通常使用前 7 位缩写），包含作者、时间戳和提交说明。
- **Branch (分支)**：一个指向某个提交的可移动指针。分支允许你在不影响主线代码的情况下进行开发。默认分支通常是 `main` 或 `master`。
- **HEAD**：一个特殊的指针，指向当前所在的分支（或某个特定的提交，称为“分离头指针”状态）。HEAD 指向的位置就是你当前工作目录所对应的代码版本。
- **Remote (远程)**：托管在服务器上的仓库副本，用于协作和备份。默认远程名称是 `origin`。
- **Push (推送)**：将本地仓库的提交上传到远程仓库。
- **Pull (拉取)**：从远程仓库下载最新提交并合并到当前分支。实际上是 `git fetch + git merge` 的组合操作。
- **Fetch (获取)**：从远程仓库下载最新的提交和分支信息，但不自动合并到当前工作分支。
- **Merge (合并)**：将两个分支的历史记录合并在一起。
- **Rebase (变基)**：将当前分支的提交「移植」到另一个分支的顶部，使提交历史更加线性整洁。
- **Conflict (冲突)**：当两个分支对同一文件的同一行进行了不同的修改时，Git 无法自动决定使用哪个版本，需要人工介入解决。

6.2 初始配置（仅需一次）

在你的 Mac 上第一次使用 Git 时，必须先告诉 Git 你的身份标识。这些信息会被嵌入到每一次提交中，作为提交者的身份凭据：

```
# 设置全局用户名（出现在每次提交日志中）
git config --global user.name "Your Name"

# 设置全局邮箱（必须与 GitHub 账户邮箱匹配，否则贡献统计不会计入你的账户）
git config --global user.email "your_email@example.com"

# 设置默认分支名称（新仓库初始化时的默认分支名，GitHub 推荐使用 main）
git config --global init.defaultBranch main

# 设置默认文本编辑器（用于撰写提交说明、解决冲突等交互场景）
git config --global core.editor "code --wait"

# 启用命令行彩色输出（让 diff、status、log 等命令的输出更易读）
git config --global color.ui auto

# 设置自动转换行尾符（macOS/Linux 使用 LF，Windows 使用 CRLF）
git config --global core.autocrlf input

# 查看当前所有全局配置
git config --global --list

# 查看某个特定配置项的值
git config --global user.name
```

配置的三个级别：

- --system：系统级，作用于当前计算机所有用户，存储在 /etc/gitconfig
- --global：用户级，作用于当前用户的所有仓库，存储在 ~/.gitconfig
- --local：仓库级（默认），仅作用于当前仓库，存储在 .git/config

6.3 基础 workflow

步骤 A：克隆远程仓库 (Clone)

将 GitHub 或其他 Git 托管平台上的现有项目下载到本地。克隆会下载完整的仓库历史，并自动将 origin 设置为该远程仓库的别名：

```
# 通过 HTTPS 克隆（需要用户名和密码/Token）
git clone https://github.com/username/repository.git

# 通过 SSH 克隆（需要先配置 SSH Key）
git clone git@github.com:username/repository.git

# 克隆到指定目录名（而非使用默认的仓库名）
```



```
git clone <仓库URL> <自定义目录名>

# 浅克隆：仅克隆最近 N 次提交历史（减少下载量，适合只需最新代码的场景）
git clone --depth 1 <仓库URL>

# 克隆指定分支
git clone -b <分支名> <仓库URL>
```

步骤 B：检查状态 (Status)

在修改代码后，随时检查当前文件状态——哪些文件被修改了、哪些在暂存区、哪些未追踪：

```
# 查看详细的文件状态
git status

# 以精简模式查看（每行一个文件，适合脚本处理）
git status --short

# 输出格式：左列表示暂存区状态，右列表示工作区状态
# ?? = 未追踪, M = 已修改, A = 新添加, D = 已删除, R = 重命名

# 查看状态并显示分支信息（包含与远程分支的领先/落后比较）
git status --branch
```

步骤 C：暂存更改 (Add)

将工作区中修改过的文件添加到暂存区，为提交做准备。暂存区让你可以精确控制每次提交包含哪些修改：

```
# 添加所有发生更改的文件（包括新增、修改、删除）
git add .

# 添加指定文件或目录
git add <文件名>
git add src/

# 以交互模式选择性地暂存（可以只暂存文件的部分修改，而非整个文件）
git add --patch <文件名>
# 交互选项：y=暂存这个块，n=不暂存这个块，s=拆分为更小的块，e=手动编辑

# 添加所有已追踪文件的修改（不包括新文件）
git add --update
# 或简写
git add -u

# 添加所有文件（包括未追踪的），但不包括 .gitignore 中忽略的文件
git add --all
```

步骤 D：提交更改 (Commit)

将暂存区中的所有内容保存到本地仓库，创建一个新的版本快照：

```
# 最基本的提交（会打开默认编辑器让你输入提交说明）
git commit
```

```
# 附带简短提交说明的提交（适合单行信息）
git commit -m "修复了数据解析模块的空指针异常"

# 附带详细提交说明的提交（第一行为标题，空一行后为详细描述）
git commit -m "修复数据解析空指针" -m "当输入数据为 null 时增加空值检查，返回默认值而非抛出异常"

# 提交所有已追踪文件的修改（跳过 git add 步骤，但不包括新文件）
git commit -a -m "提交说明"

# 修改上一次提交（追加漏掉的文件或修改提交说明）
# 注意：这会改写提交历史，如果已推送则不要对已推送的提交使用此命令
git commit --amend -m "新的提交说明"
git commit --amend --no-edit # 保持原提交说明，只追加暂存区的修改
# 忘记暂存某个文件时：
git add <遗漏的文件>
git commit --amend --no-edit
```

提交说明的最佳实践：

- 使用祈使句（"修复"而非"已修复"或"修复了"）
- 第一行不超过 50 个字符（作为邮件标题行）
- 第一行与详细描述之间空一行
- 详细描述说明做了什么、为什么这样做（而非怎么做的——代码已经说明了怎么做的）
- 使用约定式提交格式（Conventional Commits）：`<type>: <description>`
 - feat: — 新功能
 - fix: — 修复 Bug
 - docs: — 文档更新
 - style: — 代码格式（不影响功能）
 - refactor: — 重构
 - perf: — 性能优化
 - test: — 测试相关
 - chore: — 构建过程或辅助工具的变动

步骤 E：推送到远程 (Push)

将本地仓库的提交同步到远程仓库（GitHub/GitLab 等），实现云端备份和协作共享：

```
# 推送当前分支到远程仓库的同名分支
git push

# 推送指定分支到远程
git push origin main

# 推送并设置上游分支（之后可以直接使用 git push 而不需要指定分支名）
git push --set-upstream origin <分支名>
# 或简写
git push -u origin <分支名>
```

```
# 强制推送（覆盖远程历史，极危险操作，仅在确认无人基于此分支工作时使用）
git push --force

# 更安全的强制推送（仅在你的远程分支与本地已知状态一致时才推送，避免覆盖他人的推送）
git push --force-with-lease

# 推送所有本地分支到远程
git push --all origin

# 推送本地标签到远程
git push --tags

# 删除远程分支
git push origin --delete <分支名>
```

步骤 F：同步远程更新 (Pull)

在开始工作前或需要获取他人最新代码时，从远程仓库拉取并合并更新：

```
# 拉取远程更新并自动合并到当前分支
git pull

# 指定拉取的远程和分支
git pull origin main

# 使用变基方式拉取（保持提交历史线性整洁，推荐在个人分支上使用）
git pull --rebase

# 仅获取远程更新但不合并
git fetch origin
# 之后可以手动查看差异再决定如何合并
git diff main origin/main
git merge origin/main
```

6.4 查看历史与比较差异

功能	命令	详细说明
查看完整提交历史	<code>git log</code>	按时间倒序显示所有提交，包含完整 SHA-1 哈希值、作者、日期和提交说明。按 q 退出分页浏览。
单行精简日志	<code>git log --oneline</code>	每个提交仅一行，显示缩写哈希和提交说明标题。适合快速浏览版本历史。
图形化分支历史	<code>git log --oneline --graph --all</code>	以 ASCII 图形直观展示所有分支的提交历史和合并关系。--all 显示所有分支，--graph 显示分支图谱。
显示每次提交的修改统计	<code>git log --stat</code>	在每次提交信息后列出被修改的文件和增删行数统计。
显示具体修改内容	<code>git log -p</code>	显示每次提交的完整差异内容（patch）。适合深入审查某次提交的代码变更。
限制显示数量	<code>git log -n 5</code>	只显示最近 5 次提交。
按作者过滤	<code>git log --author="用户名"</code>	只显示特定作者的提交。支持部分匹配和正则表达式。
按时间过滤	<code>git log --since="2024-01-01" --until="2024-12-31"</code>	只显示指定时间范围内的提交。
按提交信息过滤	<code>git log --grep="关键词"</code>	只显示提交说明中包含指定关键词的提交。
按代码内容搜索	<code>git log -S "代码片段"</code>	搜索所有历史中引入或删除了特定代码片段的提交。在追踪某个函数何时被引入时非常有用。
查看某个文件的修改历史	<code>git log -- <文件名></code>	只查看某个特定文件的所有提交历史。
查看某个文件的逐行修改者	<code>git blame <文件名></code>	显示文件中每一行是谁、在哪次提交中最后修改的。-L 参数可以指定行号范围查看。
查看工作区未暂存的修改	<code>git diff</code>	比较工作区和暂存区之间的差异。显示自上次 git add 以来你对文件做了哪些修改。
查看暂存区待提交的修改	<code>git diff --staged</code> 或 <code>git diff --cached</code>	比较暂存区和最近一次提交之间的差异。显示即将被提交的内容。
查看两个提交之间的差异	<code>git diff <commit1> <commit2></code>	比较任意两个提交之间的代码差异。可以用分支名、标签名或哈希值指定提交。
查看两个分支的差异	<code>git diff main..feature</code>	比较 feature 分支相对于 main 分支的所有代码变更。
查看引入了某次提交的分支	<code>git branch --contains <commit></code>	列出所有包含指定提交的分支。
查看某次提交的详细信息	<code>git show <commit></code>	显示某次提交的完整信息，包括提交说明和代码差异。

6.5 分支管理

分支是 Git 最强大的特性之一。它相当于游戏中的「平行宇宙」或「存档位」——你可以在新分支上放心大胆地尝试新功能或进行实验性修改，如果搞砸了，直接删掉分支即可，主干代码丝毫无损。

```
# 查看所有本地分支（当前分支前有 * 号标记）
git branch

# 查看所有分支（包括已删除但在远程仓库中仍然存在的远程追踪分支）
git branch --all

# 查看每个分支的最新提交（同时显示提交说明）
git branch --verbose

# 查看哪些分支已经合并到当前分支（可以安全删除）
git branch --merged

# 查看哪些分支尚未合并到当前分支
git branch --no-merged

# 创建新分支（但不会切换到它）
git branch <新分支名>

# 创建并切换到新分支（推荐方式）
git switch -c <新分支名>
# 旧版 Git 使用: git checkout -b <新分支名>

# 切换分支
git switch <分支名>
# 旧版 Git 使用: git checkout <分支名>

# 切换到上一个所在的分支
git switch -

# 基于指定提交创建分支
git switch -c <新分支名> <commit-hash>

# 重命名当前分支
git branch -m <新分支名>

# 删除本地分支（安全删除：仅当分支已合并时才删除）
git branch -d <分支名>

# 强制删除本地分支（即使未合并也会删除）
git branch -D <分支名>

# 合并分支（将指定分支合并到当前分支）
# 假设当前在 main 分支，执行下命令会将 dev 分支的代码合并进 main
git merge <分支名>

# 合并时禁用快进（始终创建合并提交，保留分支历史痕迹）
git merge --no-ff <分支名>

# 合并时压缩所有提交为一个（保持主分支历史整洁）
```

```

git merge --squash <分支名>
git commit -m "合并 feature 分支的功能"

# 变基操作（将当前分支的提交重新应用到目标分支的顶端）
git rebase main
# 使用场景：你在 feature 分支上开发时，main 分支有了新提交，
# 使用 rebase 可以将你的提交「移动」到 main 的最新提交之后

# 交互式变基（可以压缩、编辑、重排、删除提交）
git rebase -i HEAD~n # n 为要处理的最近 n 次提交数量
# 交互选项：
# pick = 保留该提交
# reword = 保留但修改提交说明
# edit = 保留但暂停以修改提交内容
# squash = 合并到上一个提交并保留说明
# fixup = 合并到上一个提交但丢弃说明
# drop = 删除该提交

```

分支策略：

- main / master — 主分支，始终保持可部署状态
- develop — 开发分支，所有功能分支的集成目标
- feature/xxx — 功能分支，从 develop 分出，完成后合并回 develop
- hotfix/xxx — 热修复分支，从 main 分出，修复紧急 Bug 后同时合并回 main 和 develop
- release/xxx — 发布分支，从 develop 分出，准备发布版本

6.6 撤销与重置操作

Git 提供了多层级的「后悔药」，从撤销工作区修改到回滚整个提交历史：

```

# --- 工作区级别 ---

# 撤销工作区中某个文件的修改（恢复到最近一次提交的状态）
git restore <文件名>
# 旧版 Git 使用：git checkout -- <文件名>

# 撤销工作区中所有文件的修改
git restore .

# --- 暂存区级别 ---

# 将误添加到暂存区的文件移出（保留工作区的修改内容）
git restore --staged <文件名>
# 旧版 Git 使用：git reset HEAD <文件名>

# 将所有文件移出暂存区
git restore --staged .

# --- 提交级别 ---

```

```
# 撤销最近一次提交，保留工作区和暂存区的修改（就像没提交过一样）
git reset --soft HEAD~1

# 撤销最近一次提交，保留工作区修改但清空暂存区（默认行为）
git reset --mixed HEAD~1
# 等价于 git reset HEAD~1

# 彻底撤销最近一次提交（丢弃所有相关修改，危险操作）
git reset --hard HEAD~1

# 撤销到任意指定的提交
git reset --hard <commit-hash>

# --- 安全撤销（推荐用于已推送的提交） ---

# 创建一个新的提交来反向撤销某个历史提交的修改（不会改写历史，适合已推送的提交）
git revert <commit-hash>

# 撤销最近一次提交并自动创建新的 revert 提交
git revert HEAD

# --- 恢复已删除的内容 ---

# 查看所有操作历史（包括已删除的提交和分支），找到丢失提交的哈希值
git reflog

# 恢复到 reflog 中的某个状态
git checkout <reflog-entry>
# 或创建新分支指向该状态
git switch -c recovered-branch <reflog-entry>

# --- 暂存工作现场 ---

# 将当前未提交的修改暂存起来（保存到栈中），恢复干净的工作区
git stash

# 暂存时附带描述信息
git stash push -m "WIP: 正在重构数据层，临时切换修复紧急 Bug"

# 查看暂存列表
git stash list

# 恢复最近一次暂存的内容（保留 stash 记录）
git stash apply

# 恢复最近一次暂存的内容并删除该 stash 记录
git stash pop

# 恢复指定的 stash
git stash apply stash@{2}

# 删除指定的 stash
git stash drop stash@{2}
```

```
# 清空所有 stash
git stash clear

# 暂存包括未追踪文件
git stash --include-untracked
```

6.7 标签管理

标签是一个指向特定提交的不可变引用，通常用于标记发布版本号：

```
# 列出所有标签
git tag

# 按模式过滤标签
git tag --list "v1.0.*"

# 创建轻量标签（只是一个指向提交的指针，无额外信息）
git tag v1.0.0

# 创建附注标签（包含标签作者、日期和说明，推荐用于正式发布）
git tag -a v1.0.0 -m "正式发布 1.0.0 版本"

# 为历史提交打标签
git tag -a v0.9.0 <commit-hash> -m "回溯标记 0.9.0 版本"

# 查看标签详细信息
git show v1.0.0

# 推送标签到远程
git push origin v1.0.0      # 推送单个标签
git push origin --tags      # 推送所有标签

# 删除本地标签
git tag -d v1.0.0

# 删除远程标签
git push origin --delete v1.0.0
```

6.8 远程仓库管理

```
# 查看已配置的远程仓库
git remote --verbose

# 添加远程仓库
git remote add <远程名> <URL>

# 修改远程仓库 URL
```



```
git remote set-url <远程名> <新URL>

# 重命名远程仓库
git remote rename <旧名> <新名>

# 删除远程仓库
git remote remove <远程名>

# 查看远程仓库详细信息
git remote show origin

# 清理本地的远程追踪分支（删除远程已删除的分支的本地引用）
git remote prune origin

# 获取所有远程仓库的更新（不合并）
git fetch --all
```

6.9 GitHub CLI (gh) 完全指南

GitHub CLI（命令行工具 gh）是 GitHub 官方推出的命令行工具，让你可以在终端中直接完成创建仓库、管理 Issue 和 Pull Request、查看 CI/CD 状态等操作，无需打开浏览器。

```
# 安装 (通过 Homebrew)
brew install gh

# 登录认证 (选择 GitHub.com 然后选择 HTTPS 或 SSH, 通过浏览器完成 OAuth)
gh auth login

# 查看当前认证状态
gh auth status

# 登出
gh auth logout
```

仓库管理:

```
# 从当前目录创建 GitHub 仓库 (交互式)
gh repo create

# 指定名称、描述、可见性和远程源创建仓库
gh repo create <仓库名> --description "描述" --public --source=. --remote=origin

# 从模板创建仓库
gh repo create <仓库名> --template <owner>/<template-repo>

# 克隆仓库 (自动使用已配置的协议)
gh repo clone <owner>/<repo>

# 在浏览器中打开当前仓库的 GitHub 页面
gh repo view --web
```

```
# 查看仓库基本信息
gh repo view

# Fork 仓库
gh repo fork <owner>/<repo>

# Fork 并自动克隆到本地
gh repo fork <owner>/<repo> --clone

# 列出你拥有的或参与的仓库
gh repo list

# 删除 GitHub 上的仓库
gh repo delete <owner>/<repo> --confirm
```

Issue 管理:

```
# 创建 Issue (会打开编辑器)
gh issue create

# 快速创建 Issue
gh issue create --title "Bug: 登录页面崩溃" --body "## 复现步骤..." --label "bug,high-priority"

# 列出当前仓库的 Issue
gh issue list

# 按标签、状态过滤
gh issue list --label "bug" --state open

# 查看 Issue 详情 (包括评论)
gh issue view <issue编号>

# 在浏览器中打开指定 Issue
gh issue view <issue编号> --web

# 关闭/重新打开 Issue
gh issue close <issue编号>
gh issue reopen <issue编号>

# 给 Issue 添加评论
gh issue comment <issue编号> --body "已确认问题, 正在修复中"

# 搜索 Issue
gh search issues "内存泄漏" --repo <owner>/<repo>
```

Pull Request 管理:

```
# 创建 PR (自动使用当前分支作为源, 默认分支作为目标)
gh pr create

# 快速创建 PR
```

```
gh pr create --title "Feat: 添加用户认证模块" --body "实现了 JWT 认证流程" --base main --head feature/auth

# 设置为草稿 PR
gh pr create --draft

# 列出 PR
gh pr list

# 查看 PR 详情
gh pr view <PR编号>

# 查看 PR 的 CI 检查状态
gh pr checks <PR编号>

# 在浏览器中打开 PR
gh pr view <PR编号> --web

# Checkout 某个 PR 到本地
gh pr checkout <PR编号>

# 合并 PR (支持 merge、squash、rebase 三种策略)
gh pr merge <PR编号> --merge
gh pr merge <PR编号> --squash
gh pr merge <PR编号> --rebase

# 关闭 PR
gh pr close <PR编号>

# 查看 PR 的代码差异
gh pr diff <PR编号>

# 对 PR 进行 Review (批准、评论、请求修改)
gh pr review <PR编号> --approve
gh pr review <PR编号> --comment --body "建议优化错误处理逻辑"
gh pr review <PR编号> --request-changes

# 查看所有待 Review 的 PR
gh pr list --search "review-requested:@me"
```

Actions / CI/CD:

```
# 查看最近的 Workflow 运行记录
gh run list

# 查看指定 Workflow 的运行记录
gh run list --workflow "CI"

# 查看运行详情
gh run view <run-id>

# 在浏览器中查看运行
gh run view <run-id> --web
```

```
# 查看运行日志
gh run view <run-id> --log

# 重新运行 Workflow
gh run rerun <run-id>

# 监听 Workflow 运行直到完成
gh run watch <run-id>

# 查看 Workflow 的配置文件列表
gh workflow list

# 手动触发 Workflow
gh workflow run <workflow名称>
```

其他常用命令：

```
# 查看当前仓库的 GitHub Actions 状态概要
gh status

# 创建和管理 Gist
gh gist create <文件> --public
gh gist list
gh gist edit <gist-id>
gh gist delete <gist-id>

# 管理 Release
gh release create v1.0.0 --title "1.0.0 正式发布" --notes "新增功能和 Bug 修复"
gh release list
gh release download v1.0.0

# 管理 Secrets (仓库机密变量)
gh secret set <密钥名> --body "<密钥值>"
gh secret list
gh secret remove <密钥名>

# 设置 Git 协议偏好
gh config set git_protocol ssh

# 查看别名
gh alias list

# 设置别名
gh alias set co "pr checkout"

# 使用 GitHub Copilot CLI 扩展
gh extension install github/gh-copilot
```

.gitignore 是一个纯文本文件，放在 Git 仓库的根目录下。在这个文件中，你可以定义一系列规则，告诉 Git 在执行 git add 和 git status 时应当忽略哪些文件和文件夹。被忽略的文件不会被 Git 追踪，不会出现在 git status 的输出中，也不会被提交到仓库。

为什么需要 .gitignore:

- 避免将编译产物（二进制文件、.class、.dll 等）提交到仓库——这些文件应通过源代码构建生成
- 避免将包含密码、API 密钥等敏感信息的本地配置文件提交到仓库
- 避免将操作系统自动生成的文件（macOS 的 .DS_Store、Windows 的 Thumbs.db）提交到仓库
- 避免将依赖包目录（node_modules/、venv/）提交到仓库——依赖信息应在包管理文件中声明
- 保持仓库清洁，让 git status 的输出只显示真正重要的文件变更

6.11 语法规则

语法符号	作用说明	示例	匹配结果
#	注释行，Git 会忽略该行	# 这是注释	该行被忽略，仅供人阅读
*	通配符，匹配任意多个字符（包括零个）	*.log	忽略所有以 .log 结尾的文件
?	通配符，匹配单个字符	test?.txt	忽略 test1.txt、testA.txt，但不忽略 test12.txt
**	匹配任意层级的目录	**/logs	忽略任何层级的 logs 目录
/ (结尾)	指明这是一个目录而非文件	build/	忽略名为 build 的整个文件夹及其所有内容
/ (开头)	仅匹配相对于 .gitignore 文件所在目录（通常是根目录）的路径	/TODO.md	只忽略根目录下的 TODO.md，不忽略 src/TOD0.md
!	反向排除——不要忽略匹配此模式的文件	!important.log	即使前面定义了 *.log，这个文件依然会被 Git 追踪
[abc]	匹配方括号中的任意一个字符	file[0-9].txt	匹配 file0.txt 到 file9.txt

重要规则:

- 空行不匹配任何文件，作为分隔符便于阅读
- 行尾空格会被忽略，除非用 \ 转义
- .gitignore 文件的规则只能作用于尚未被 Git 追踪的文件。如果文件已经被 git add 或 git commit 追踪过，再加入 .gitignore 是无效的
- 可以在子目录中创建额外的 .gitignore 文件，其规则仅作用于该子目录及其下级目录
- 规则的优先级：子目录的 .gitignore > 根目录的 .gitignore > .git/info/exclude > 全局配置

6.12 综合模板

```
# =====
# 1. macOS 系统文件
# =====
# Mac 系统自动生成的文件夹自定义属性文件
.DS_Store
.AppleDouble
.LSOverride
# 缩略图缓存
._*
# 图标文件
Icon
# .DS_Store 即使在子目录也不例外
**/.DS_Store

# =====
# 2. 编辑器与 IDE
# =====
# VS Code 工作区设置（如果团队需要共享工作区设置，则不忽略此目录）
.vscode/
*.code-workspace
# JetBrains IDE 系列
.idea/
*.iml
# Vim 交换文件
*.swp
*.sw0
*~
# Emacs
\#*\#
.*#*

# =====
# 3. Python 相关
# =====
# 编译后的字节码文件
__pycache__/
*.py[cod]
*.pyo
# 虚拟环境目录
.venv/
venv/
env/
ENV/
# 包分发构建产物
*.egg-info/
dist/
build/
*.egg
# 单元测试与覆盖率报告
.pytest_cache/
.coverage
htmlcov/
```

```
.tox/
# Jupyter Notebook 检查点
.ipynb_checkpoints/
# mypy 类型检查缓存
.mypy_cache/
# 环境变量文件（可能包含密钥）
.env
.env.local
.env.*.local
# Poetry 锁文件（库项目忽略，应用项目保留）
# poetry.lock

# =====
# 4. C# / .NET 相关
# =====
# 编译输出目录
bin/
obj/
# NuGet 包目录
packages/
*.nupkg
# Visual Studio 用户设置
*.suo
*.user
*.userosscache
*.sln.docstates
# VS Code 的 C# 扩展产物
.vs/

# =====
# 5. JavaScript / TypeScript / Node.js
# =====
# 依赖包目录
node_modules/
# 构建输出
dist/
build/
.next/
.nuxt/
# 锁文件（库项目忽略，应用项目保留）
# package-lock.json
# yarn.lock
# 环境变量
.env
.env.local
.env.development.local
.env.test.local
.env.production.local
# 日志文件
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# =====
# 6. Java
```

```
# =====
# 编译产物
*.class
*.jar
*.war
# Maven
target/
# Gradle
.gradle/
build/
# IDE
*.iml
.idea/

# =====
# 7. C / C++
# =====
# 编译产物
*.o
*.obj
*.out
*.exe
*.app
# 静态和动态库
*.a
*.so
*.dylib
# CMake 构建目录
build/
cmake-build-*/

# =====
# 8. 通用项目文件
# =====
# 压缩文件
*.zip
*.tar.gz
*.rar
*.7z
# 日志文件
*.log
# 临时文件
tmp/
temp/
*.tmp
*.temp
# 操作系统缩略图缓存
Thumbs.db
ehthumbs.db
# 备份文件
*.bak
*.backup
*.old
```


6.13 全局 .gitignore

除了项目级别的 .gitignore，你还可以设置一个全局的 .gitignore 文件，作用于本机所有 Git 仓库。这对于在系统级别忽略 .DS_Store 等文件非常实用：

```
# 创建全局 .gitignore 文件
touch ~/.gitignore_global

# 配置 Git 使用这个全局文件
git config --global core.excludesfile ~/.gitignore_global

# 在全局文件中添加需要忽略的内容（例如 macOS 系统文件）
echo ".DS_Store" >> ~/.gitignore_global
```

6.14 常见问题：文件已被追踪怎么办

.gitignore 只能忽略那些还没有被 Git 追踪（Tracked）的新文件。如果你之前不小心已经把 bin/ 目录或者 .DS_Store 文件 git commit 到仓库里了，此时再把它们加到 .gitignore 中是无效的——Git 会继续追踪它们的修改。

解决方法：清除本地 Git 缓存后重新添加

```
# 第一步：从 Git 的索引缓存中删除所有文件的追踪状态
# （不会删除你电脑上的实体文件，只清除 Git 的追踪记录）
git rm -r --cached .

# 第二步：重新添加所有文件
# 此时 Git 会严格按照 .gitignore 的规则来决定追踪哪些文件
git add .

# 第三步：提交这次刷新索引的更改
git commit -m "chore: 更新 .gitignore 配置并清理缓存"
```

6.15 现代 Git 辅助工具

除了 gh 命令行工具外，还有几个能极大提升 Git 日常使用体验的辅助工具。

Lazygit——终端 Git 图形界面：

Lazygit 是一个用 Go 编写的终端 UI（TUI）Git 客户端，让你在终端中用键盘快速完成 Git 操作而无需记忆大量命令。

```
# 安装
brew install lazygit

# 在 Git 仓库中启动
lazygit
```

```
# 常用快捷键（在 Lazygit 界面中）：
# Tab - 在面板间切换（状态/暂存/提交/分支）
# Space - 暂存/取消暂存当前文件
# a - 暂存所有更改
# c - 打开提交面板
# P - Push（大写）
# p - Pull（小写）
# / - 搜索
# q - 退出
# ? - 显示帮助
# z - 展开/折叠（stash）
# B - 分支面板
# [ / ] - 切换面板
# X - 打开菜单
# Ctrl+S - 打开过滤面板（搜索文件）
```

delta——更好看的 git diff:

delta 是一个用 Rust 编写的 git diff 查看器，提供语法高亮、行号、并排视图等增强功能。

```
# 安装
brew install git-delta
# 注意：Homebrew 中包名为 git-delta，命令为 delta

# 在 ~/.gitconfig 中配置
# [core]
#     pager = delta
# [interactive]
#     diffFilter = delta --color-only
# [delta]
#     navigate = true           # 使用 n/N 跳转到下一个/上一个文件
#     light = false            # 暗色主题
#     side-by-side = true      # 并排视图
#     line-numbers = true      # 显示行号
# [merge]
#     conflictstyle = diff3
# [diff]
#     colorMoved = default

# 快速配置（推荐）：
# 在 ~/.gitconfig 中添加：
# [core]
#     pager = delta
# [interactive]
#     diffFilter = delta --color-only

# delta 也支持直接比较两个文件
delta file1.txt file2.txt
```

git-open——在浏览器中打开仓库：

```
# 安装
brew install git-open

# 在 Git 仓库中运行（打开当前分支的 GitHub/GitLab 页面）
git open

# 打开指定文件/行号
git open --issue      # 打开 Issue 页面
git open --pull       # 打开 PR 页面
```

git-standup——查看每日工作记录：

```
# 安装
brew install git-standup

# 查看今天的提交记录（自动按作者过滤）
git standup

# 查看指定日期范围的记录
git standup "2 days ago"
git standup "last week"

# 查看所有作者的提交
git standup -a
```

tig——终端的 Git 仓库浏览器：

```
# 安装
brew install tig

# 基本用法
tig                # 浏览提交历史
tig <文件名>       # 浏览文件的修改历史
tig blame <文件名> # 查看行的修改归属
# tig 内快捷键：j/k 上下移动，Enter 查看详情，q 返回/退出
```

七、Python 虚拟环境 venv 完全指南

7.1 核心概念

venv 是 Python 3.3 及以后版本内置的标准库模块，用于创建轻量级的虚拟 Python 环境。每个虚拟环境都是一个独立的目录，包含该项目专属的 Python 解释器副本（或链接）和一套独立的第三方包安装空间。

为什么需要虚拟环境：

- 依赖隔离：项目 A 需要 Django 2.2，项目 B 需要 Django 4.0，两个版本的 Django 无法在同一系统 Python 中共存。虚拟环境让每个项目拥有独立的包空间，各不干扰。
- 可复现性：通过 `requirements.txt` 冻结项目依赖的确切版本，确保在其他机器上能复现相同的运行环境。
- 避免系统污染：不在系统级 Python 中安装包，保持系统 Python 清洁。macOS 自带的系统 Python 被系统工具依赖，随意修改可能导致系统问题。
- 权限问题：在系统 Python 中使用 `pip install` 可能需要 `sudo` 权限，而虚拟环境中的安装完全不需要管理员权限。
- 安全清理：不再需要的虚拟环境可以直接删除整个目录，不会留下任何残留文件。

venv 的隔离维度（也是其局限性）：

-  隔离 Python 第三方包（通过 pip 安装的包）
-  不隔离 Python 解释器版本——虚拟环境必须是系统已安装的 Python 版本
-  不隔离系统级 C/C++ 库（某些 Python 包依赖的系统库）
-  不隔离操作系统环境

7.2 创建虚拟环境

```
# 最基础的创建命令（在当前目录下创建名为 .venv 的虚拟环境目录）
python3 -m venv .venv

# 创建指定名称的虚拟环境
python3 -m venv myenv

# 创建虚拟环境但不在其中安装 pip（极少数情况下使用）
python3 -m venv --without-pip .venv

# 允许虚拟环境访问系统已安装的 Python 包（不推荐，会降低隔离性）
python3 -m venv --system-site-packages .venv

# 即使目标目录已存在也强制创建（会覆盖原有内容）
python3 -m venv --clear .venv

# 指定使用特定版本的 Python 解释器创建虚拟环境
# （前提是该版本已在系统中安装）
```

```
python3.11 -m venv .venv
python3.12 -m venv .venv

# 创建虚拟环境时不创建脚本链接（用于部署场景）
python3 -m venv --without-scripts .venv
```

虚拟环境目录结构：

```
.venv/
├── bin/           # 可执行文件：python、pip、activate 等
├── include/       # C 头文件
├── lib/           # 安装的第三方 Python 包
│   └── python3.x/
│       └── site-packages/
└── pyenvv.cfg     # 虚拟环境配置文件（包含 Python 版本、路径信息）
```

7.3 激活与退出

```
# --- 激活虚拟环境 ---

# macOS / Linux (bash/zsh)
source .venv/bin/activate

# 或使用简写（某些 Shell 配置支持）
. .venv/bin/activate

# Windows (Command Prompt)
.venv\Scripts\activate.bat

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# --- 确认虚拟环境已激活 ---
# 激活后，终端提示符前会出现虚拟环境名称，如：
# (.venv) user@macbook project %
# 可以通过以下命令确认当前使用的 Python 路径
which python    # 应指向 .venv/bin/python
which pip       # 应指向 .venv/bin/pip

# --- 退出虚拟环境 ---
deactivate
```

激活虚拟环境后发生了什么：

1. 修改 PATH 环境变量，将虚拟环境的 bin/ 目录置于最前
2. 修改 Shell 提示符，在开头显示虚拟环境名称
3. 设置 VIRTUAL_ENV 环境变量指向虚拟环境根目录
4. 此后所有 python、pip 等命令都指向虚拟环境中的版本

7.4 包管理

```
# --- 安装包 ---

# 安装单个包的最新版本
pip install <包名>

# 安装指定版本
pip install <包名>==2.5.0

# 安装版本范围
pip install <包名>>=1.0,<2.0

# 从 requirements 文件批量安装
pip install -r requirements.txt

# 从当前目录的 setup.py 或 pyproject.toml 安装（开发模式）
pip install -e .

# 安装时不安装依赖（仅安装指定包本身）
pip install --no-deps <包名>

# 从本地 wheel 文件安装
pip install /path/to/package.whl

# 从 Git 仓库安装
pip install git+https://github.com/user/repo.git
pip install git+https://github.com/user/repo.git@branch-name

# --- 卸载包 ---

# 批量卸载（需要确认）
pip uninstall -r requirements.txt

# 不询问确认直接卸载
pip uninstall -y <包名>

# --- 查看已安装的包 ---

# 列出当前环境中所有已安装的包及其版本
pip list

# 列出有更新的包（与 PyPI 上最新版本对比）
pip list --outdated

# 以表格形式显示（需要安装 pipdeptree）
pip install pipdeptree
pipdeptree

# 查看指定包的详细信息（版本、依赖、作者、许可证等）
pip show <包名>

# 查看指定包安装了哪些文件
```

```
pip show --files <包名>

# --- 导出与冻结依赖 ---

# 冻结当前环境的依赖到文件（锁定精确版本）
pip freeze > requirements.txt

# 只冻结项目直接依赖（排除子依赖），需要 pipreqs
pip install pipreqs
pipreqs . --force

# 查看哪些已安装的包与 requirements.txt 不一致
pip check

# --- 升级包 ---

# 升级单个包到最新版本
pip install --upgrade <包名>

# 升级 pip 自身
pip install --upgrade pip

# 查看哪些包需要升级
pip list --outdated --format=columns

# --- 缓存管理 ---

# 查看 pip 缓存目录
pip cache dir

# 查看缓存信息和使用量
pip cache info

# 清理所有缓存
pip cache purge

# 清理特定包的缓存
pip cache remove <包名>

# --- 配置管理 ---

# 查看当前 pip 的所有配置
pip config list

# 设置国内镜像源（清华大学镜像）
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple

# 设置阿里云镜像源
pip config set global.index-url https://mirrors.aliyun.com/pypi/simple/

# 设置额外索引源
pip config set global.extra-index-url https://pypi.org/simple

# 编辑配置文件
pip config edit
```

7.5 常用依赖管理文件

requirements.txt 格式说明：

```
# 精确版本
requests==2.28.0

# 版本范围
numpy>=1.20,<2.0

# 不等于某个版本
click!=8.0.0

# 兼容版本 (>=1.0, <2.0)
flask~=2.0

# 从 Git 仓库安装
git+https://github.com/user/repo.git@v1.0#egg=package_name

# 从本地路径安装
../local_package

# 包含另一个 requirements 文件
-r base-requirements.txt
```

推荐的依赖管理策略：

- requirements.in — 手动维护的直接依赖（不锁定版本或宽松版本）
- requirements.txt — 通过 pip-compile 从 .in 文件生成的锁定版本文件（包含所有传递依赖的精确版本）
- requirements-dev.txt — 仅开发环境需要的依赖（测试框架、代码检查工具等）

7.6 虚拟环境最佳实践

1. 命名规范：始终使用 .venv 或 venv 作为虚拟环境目录名（以点开头的 .venv 在 Finder 中默认隐藏，更为整洁）
2. 添加到 .gitignore：永远不要把虚拟环境目录提交到 Git 仓库。在 .gitignore 中添加 .venv/、venv/、env/
3. 始终激活后操作：养成在执行任何 Python 相关命令前先激活虚拟环境的习惯
4. 使用绝对路径激活：可以从任意目录激活虚拟环境，source /path/to/project/.venv/bin/activate
5. VS Code 集成：VS Code 会自动检测项目中的虚拟环境，在打开终端时自动激活。可以在命令面板中选择 Python 解释器来切换虚拟环境
6. 一个项目一个环境：每个 Python 项目都应该有自己独立的虚拟环境，不要跨项目共用
7. 重建而非修复：如果虚拟环境出现问题（依赖冲突、损坏），直接删除重建比排查修复更快：rm -rf .venv && python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt

7.7 uv——极速 Python 包管理器

uv 是一个用 Rust 编写的极速 Python 包管理器和虚拟环境管理器，由 Astral 团队（Ruff 的开发者）开发，可以作为 pip、pip-tools、virtualenv、poetry 等多工具的替代品。它的核心优势是速度——通常比 pip 快 10-100 倍。

安装：

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# 通过 Homebrew
brew install uv

# 通过 pip
pip install uv

# 更新
uv self update
```

虚拟环境管理：

```
# 创建虚拟环境（默认在 .venv 目录，比 venv 快得多）
uv venv

# 创建指定 Python 版本的虚拟环境（自动下载该 Python 版本）
uv venv --python 3.12

# 创建指定路径的虚拟环境
uv venv /path/to/venv

# 激活虚拟环境（与标准 venv 相同）
source .venv/bin/activate
```

包管理（替代 pip）：

```
# 安装包（比 pip install 快 10-100 倍）
uv pip install <包名>

# 从 requirements 文件安装
uv pip install -r requirements.txt

# 安装到系统 Python（需要指定 --system 或激活环境）
uv pip install --system <包名>

# 编译生成锁定的依赖文件
uv pip compile requirements.in -o requirements.txt

# 同步环境到与 requirements.txt 完全一致（删除多余的包）
uv pip sync requirements.txt

# 列出已安装的包
uv pip list
```

```
# 冻结依赖
uv pip freeze

# 查看包的详细信息
uv pip show <包名>

# 卸载包
uv pip uninstall <包名>

# 查看依赖树
uv pip tree
```

uv 独有的高级功能:

```
# 运行单次 Python 脚本并自动管理依赖 (类似于 npx)
uv run --with <包名> python script.py

# 使用指定的 Python 版本运行
uv run --python 3.11 python script.py

# 安装 Python 解释器版本 (自动管理 Python 版本)
uv python install 3.12
uv python install 3.11 3.10 3.9

# 列出已安装的 Python 版本
uv python list

# 查找系统中可用的 Python 版本
uv python find

# 锁文件管理 (生成跨平台的锁文件)
uv lock
uv sync

# 初始化项目 (创建 pyproject.toml)
uv init

# 添加依赖到项目
uv add <包名>
uv add --dev <开发依赖包名>

# 移除依赖
uv remove <包名>

# 构建项目
uv build
```

uv 与 **pip**/**venv** 的对比:

对比维度	pip + venv	uv
安装速度	慢（纯 Python，逐个下载安装）	极快（Rust 实现，并行下载，增量缓存）
依赖解析	SAT 求解器，可能很慢	高级解析器，通常更快
虚拟环境创建	标准库支持	兼容标准 venv 结构
锁文件支持	需要 pip-tools	内置 uv.lock
Python 版本管理	不内置	内置 uv python install
跨平台锁文件	不内置	原生支持

uv 与 pip 的兼容性：

- uv 使用与 pip 兼容的包格式和依赖解析规则
- uv 创建的虚拟环境与标准 venv 完全兼容
- uv pip 子命令专为与现有 pip 工作流兼容而设计
- 生成的 requirements.txt 与 pip 完全兼容

八、Conda 环境管理完全指南

8.1 Conda 核心概念

Conda 是一个跨平台的包管理器和环境管理器，最初为 Python 科学计算社区设计，但现在已支持任意语言的包管理。Conda 常通过 Anaconda（完整发行版，预装上百个科学计算包）或 Miniconda（最小安装版，仅包含 conda 和 Python）安装。

Conda 与 venv/pip 的本质区别：

- Conda 不仅能管理 Python 包，还能管理 Python 解释器本身的版本。你可以在同一台电脑上用 Conda 创建一个 Python 3.8 环境，再创建一个 Python 3.12 环境，它们完全独立。
- Conda 下载的是预编译的二进制包，而非源码包。这意味着对于依赖 C/C++ 底层库的科学计算包（如 NumPy、SciPy、pandas），Conda 不会遇到 `pip install` 时常出现的「缺少 C++ 编译器」问题。
- Conda 可以安装非 Python 的依赖，包括 C/C++ 库、R 包、甚至系统级工具。
- Conda 拥有自己的包仓库（conda-forge、defaults 等频道），与 PyPI 是独立的生态系统。

Miniconda vs Anaconda：

- **Miniconda：** 仅包含 conda、Python 和少量基础包，体积小、安装快。推荐给有经验的开发者，按需安装所需包。
- **Anaconda：** 包含 conda、Python 和 150+ 科学计算常用包，安装包体积大。适合新手或需要开箱即用科学计算环境的用户。

8.2 安装

在 macOS 上推荐通过 Homebrew 安装 Miniconda：

```
# 通过 Homebrew 安装 Miniconda
brew install --cask miniconda

# 安装完成后初始化 conda（使其可在终端中使用）
conda init zsh

# 重启终端或重新加载配置
source ~/.zshrc

# 验证安装
conda --version
```

也可以从官网下载 .pkg 安装包：

- Miniconda: <https://docs.conda.io/en/latest/miniconda.html>
- Anaconda: <https://www.anaconda.com/download>

8.3 环境管理命令

```
# --- 查看环境 ---

# 列出所有已创建的 Conda 环境（当前激活的环境前有 * 号）
conda env list
# 或
conda info --envs

# 查看当前环境的详细信息
conda info

# 查看当前环境中安装的所有包
conda list

# 查看指定环境中的包
conda list --name <环境名>

# 以可导出的格式列出包（适合生成 environment.yml）
conda list --export

# 查看环境中包的依赖关系
conda list --show-channel-urls

# --- 创建环境 ---

# 创建指定 Python 版本的基础环境
conda create --name <环境名> python=3.11

# 创建环境并同时安装多个包
conda create --name <环境名> python=3.11 numpy pandas matplotlib

# 创建环境到指定路径（而非默认的 envs 目录）
conda create --prefix ./env python=3.11

# 从 environment.yml 文件创建环境（精确复现）
conda env create --file environment.yml

# 从 environment.yml 创建到指定路径
conda env create --prefix ./env --file environment.yml

# 克隆已有环境（创建一个完全相同的副本）
conda create --name <新环境名> --clone <已有环境名>

# 仅创建空环境（不含 Python），之后手动安装所需包
conda create --name <环境名>

# --- 激活与退出 ---

# 激活环境
conda activate <环境名>

# 激活路径环境（使用 prefix 创建的环境）
conda activate ./env
```

```
# 退出当前环境（回到 base 环境）
conda deactivate

# 查看当前激活的环境
conda info --active

# --- 删除环境 ---

# 删除指定环境（包括其中所有包）
conda env remove --name <环境名>

# 删除路径环境
conda env remove --prefix ./env

# 删除环境前先列出将被删除的内容
conda env remove --name <环境名> --dry-run

# --- 更新环境 ---

# 根据 environment.yml 更新已有环境
conda env update --file environment.yml

# 更新路径环境
conda env update --prefix ./env --file environment.yml

# 更新后清理不再需要的包
conda env update --file environment.yml --prune
```

8.4 包管理命令

```
# --- 安装包 ---

# 在当前激活的环境中安装包
conda install <包名>

# 从指定频道安装（conda-forge 是社区维护的最大频道，包更全更新）
conda install --channel conda-forge <包名>
# 或简写
conda install -c conda-forge <包名>

# 安装指定版本
conda install <包名>=1.2.3

# 同时安装多个包
conda install numpy pandas scipy

# 在指定环境中安装包（无需先激活该环境）
conda install --name <环境名> <包名>

# 仅下载包而不安装（用于离线安装）
```

```
conda install --download-only <包名>

# 安装时跳过确认提示
conda install --yes <包名>

# 指定包的构建版本（当有多个版本时）
conda install <包名>=1.0=py37_0

# --- 使用 pip 在 Conda 环境中安装 ---

# 当 conda 仓库中没有某个包时，可以使用 pip 安装
# 但要注意：先用 conda 安装尽可能多的包，最后再用 pip
pip install <包名>

# 建议顺序：conda install 能装的尽量用 conda，conda 没有的才用 pip
# 混合使用 conda 和 pip 可能导致依赖冲突，建议尽量只用一种

# --- 卸载包 ---

# 卸载当前环境中的包
conda remove <包名>

# 从指定环境中卸载包
conda remove --name <环境名> <包名>

# 批量卸载
conda remove <包名1> <包名2> <包名3>

# 卸载包并同时删除其依赖中不再被其他包需要的包
conda remove <包名> --force-remove

# --- 更新包 ---

# 更新指定包到最新版本
conda update <包名>

# 更新当前环境中所有包
conda update --all

# 更新 conda 自身
conda update conda

# 更新 Python 到最新版本（在当前环境内）
conda update python

# 查看哪些包可以更新
conda update --all --dry-run

# --- 搜索与查看 ---

# 搜索可用的包（在所有已配置的频道中）
conda search <包名>

# 搜索包并显示所有可用版本
conda search <包名> --info
```

```
# 查看包的详细信息
conda search <包名> --info <包名>=1.0

# 查看包的依赖关系
conda info <包名>

# 查看指定频道的包列表
conda search --channel conda-forge <包名>
```

8.5 频道管理

Conda 的「频道 (Channel)」相当于包索引仓库。最常用的频道是 `defaults` (Anaconda 官方维护) 和 `conda-forge` (社区维护, 包更丰富、更新更快)。

```
# 查看当前配置的频道列表及其优先级
conda config --show channels

# 添加频道 (默认追加到列表末尾, 优先级最低)
conda config --add channels conda-forge

# 添加频道到最高优先级
conda config --prepend channels conda-forge

# 移除频道
conda config --remove channels <频道名>

# 设置频道为严格优先级 (严格按照 channels 列表顺序查找包)
conda config --set channel_priority strict

# 在安装时临时指定频道 (不修改配置)
conda install -c conda-forge <包名>

# 查看指定频道的 URL
conda config --show channels --show-source-location
```

推荐的频道优先级设置:

```
conda-forge (最高优先级)
defaults
```

8.6 *environment.yml* 文件

`environment.yml` 是 Conda 环境的声明文件, 用于精确复现环境。它比 `requirements.txt` 更强大, 因为它不仅记录了 Python 包, 还记录了 Python 版本和 Conda 包。


```
name: myproject
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - numpy=1.26
  - pandas=2.1
  - scipy=1.11
  - matplotlib=3.8
  - pip
  - pip:
    - requests==2.31.0
    - black==23.10.0
```

导出和导入:

```
# 导出当前环境到文件 (包含精确版本号)
conda env export > environment.yml

# 导出仅明确安装的包 (不包含依赖的依赖, 文件更简洁)
conda env export --from-history > environment.yml

# 仅导出包列表 (不包含环境名和频道信息)
conda list --export > package-list.txt

# 从 environment.yml 创建环境
conda env create --file environment.yml
```

8.7 Conda 配置管理

```
# 查看所有 Conda 配置
conda config --show

# 查看特定配置项
conda config --get <配置项名>

# 设置配置项
conda config --set <配置项名> <值>

# 常见配置项:

# 创建环境时默认使用 conda-forge 频道
conda config --add channels conda-forge

# 设置严格频道优先级
conda config --set channel_priority strict

# 启动时不自动激活 base 环境
conda config --set auto_activate_base false
```

```
# 设置默认环境创建路径
conda config --set envs_dirs /custom/path/envs

# 设置包缓存路径
conda config --set pkgs_dirs /custom/path/pkgs

# 显示包的下载来源 URL
conda config --set show_channel_urls true

# 设置 SSL 验证（在公司网络代理环境下可能需要关闭）
conda config --set ssl_verify false

# 设置代理
conda config --set proxy_servers.http http://proxy:8080
conda config --set proxy_servers.https https://proxy:8080

# 查看配置文件路径
conda config --show-sources
```

8.8 清理与维护

```
# 清理下载缓存的包文件（释放磁盘空间）
conda clean --packages

# 清理所有缓存（包缓存 + 索引缓存 + 临时文件）
conda clean --all

# 仅清理索引缓存
conda clean --index-cache

# 清理临时文件
conda clean --tempfiles

# 预览将被清理的内容而不实际删除
conda clean --all --dry-run

# 清理不再被任何环境使用的包
conda clean --packages --force
```

8.9 Mamba —— Conda 的极速替代品

Mamba 是 Conda 的一个重新实现，使用 C++ 编写，解决了 Conda 在依赖解析时慢的问题。Mamba 与 Conda 生态完全兼容——使用相同的命令语法、频道、包格式和环境结构，只是在依赖解析和下载速度上快得多。

```
# 安装 Mamba（通过 Conda）
conda install mamba -n base -c conda-forge
```

```
# 或在安装 Miniconda 时选择 Mambaforge
# Mambaforge 是预装了 Mamba 的 Miniconda 发行版

# Mamba 的命令与 Conda 几乎完全相同，只需将 conda 替换为 mamba
mamba create --name myenv python=3.11
mamba install numpy pandas
mamba update --all
mamba env list

# Mamba 特有的极速搜索
mamba repoquery search <包名>
mamba repoquery depends <包名>
mamba repoquery whoneeds <包名>

# 导出环境的完整依赖信息（包含构建字符串）
mamba list --explicit > explicit-env.txt
```

Conda vs Mamba 对比：

对比维度	Conda	Mamba
依赖解析速度	慢（Python 实现，SAT 求解器）	极快（C++ 实现，libsolv）
下载速度	单线程下载	并行多线程下载
命令兼容性	标准	与 Conda 命令完全兼容
生态兼容性	标准	完全兼容 Conda 频道、包、环境

建议：安装 Mamba 后，日常操作使用 mamba 替代 conda 可以获得显著的速度提升。在编写脚本时，conda 命令仍然可以正常使用，或全部替换为 mamba。

九、Python 多版本管理 pyenv

9.1 简介

pyenv 是一个 Python 版本管理工具，允许你在同一台机器上安装和切换多个 Python 版本。它通过在 PATH 前插入 shim（垫片）可执行文件来拦截 Python 命令并路由到对应版本。

pyenv 与 venv/Conda 的区别：

- venv 只能隔离包，不能管理 Python 版本
- Conda 可以管理 Python 版本，但较重
- pyenv 专注于 Python 版本管理，轻量级，可以与 venv 或 virtualenv 配合使用

```
# 安装 pyenv (通过 Homebrew)
brew install pyenv

# 将 pyenv 初始化添加到 shell 配置
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.zshrc
echo 'command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.zshrc
echo 'eval "$(pyenv init -)"' >> ~/.zshrc
source ~/.zshrc

# 验证安装
pyenv --version
```

Python 版本管理：

```
# 列出所有可安装的 Python 版本
pyenv install --list

# 安装指定 Python 版本
pyenv install 3.11.8
pyenv install 3.12.2

# 查看已安装的 Python 版本
pyenv versions

# 设置全局默认 Python 版本（影响所有终端）
pyenv global 3.11.8

# 设置当前目录（项目）的 Python 版本（优先于全局设置）
# 会在当前目录下创建 .python-version 文件
pyenv local 3.12.2

# 设置当前 Shell 会话的 Python 版本（仅对当前终端窗口有效）
pyenv shell 3.11.8

# 查看当前生效的 Python 版本和来源
pyenv version
```

```
# 卸载 Python 版本
pyenv uninstall 3.11.8

# 查看 pyenv 在解析 Python 命令时的执行流程
pyenv which python
pyenv which pip

# 重新生成 shim (安装新包后可能需要)
pyenv rehash
```

pyenv-virtualenv 插件:

pyenv 的最佳搭档是 pyenv-virtualenv 插件，可以在 pyenv 管理的 Python 版本上创建虚拟环境:

```
# 安装插件
brew install pyenv-virtualenv

# 并在 .zshrc 中添加
echo 'eval "$(pyenv virtualenv-init -)"' >> ~/.zshrc

# 创建虚拟环境 (基于当前 pyenv 的 Python 版本)
pyenv virtualenv 3.11.8 myproject

# 列出所有虚拟环境
pyenv virtualenvs

# 激活虚拟环境 (自动方式—进入目录时自动激活)
# 在项目目录下设置本地环境
pyenv local myproject

# 手动激活/停用
pyenv activate myproject
pyenv deactivate

# 删除虚拟环境
pyenv virtualenv-delete myproject
```

pyenv 工作机制:

1. pyenv 在 PATH 的最前面插入 ~/.pyenv/shims/ 目录
 2. 该目录包含 Python、pip 等命令的垫片可执行文件
 3. 当执行 python 时，shim 根据优先级规则 (pyenv shell > pyenv local > pyenv global) 决定使用哪个版本的 Python
 4. 将命令转发给实际对应版本的 Python 可执行文件
-

十、Node.js 版本管理 nvm

10.1 简介

nvm (Node Version Manager) 是 Node.js 生态中最流行的版本管理工具。它允许你安装和切换多个 Node.js 版本，每个版本有独立的全局 npm 包空间。

```
# 安装 nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash

# 安装后重启终端，或手动加载
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"

# 验证安装
nvm --version
```

版本管理：

```
# 查看所有可安装的 Node.js 版本 (LTS 版本和最新版本)
nvm ls-remote

# 只查看 LTS 版本
nvm ls-remote --lts

# 安装指定版本
nvm install 20.11.0
nvm install 18.19.0

# 安装最新的 LTS 版本
nvm install --lts

# 安装最新版本
nvm install node

# 查看已安装的版本
nvm ls

# 切换版本 (仅当前终端有效)
nvm use 20

# 设置默认版本 (新终端自动使用)
nvm alias default 20

# 查看当前使用的版本
nvm current

# 卸载指定版本
nvm uninstall 18.19.0
```

```
# 指定版本下运行命令
nvm run 18 --version
nvm exec 18 npm install

# 为指定版本安装全局包
nvm install 20 --reinstall-packages-from=18

# 查看特定版本的安装路径
nvm which 20
```

nvm 的替代方案:

- **fnm (Fast Node Manager):** Rust 编写的极速 Node.js 版本管理器，兼容 `.nvmrc` 和 `.node-version` 文件

```
brew install fnm
fnm install 20
fnm use 20
fnm default 20
```

- **Volta:** Rust 编写的跨平台 Node.js 工具链管理器，项目级自动版本切换

```
brew install volta
volta install node@20
volta pin node@20 # 在项目中固定版本
```

.nvmrc 文件:

在项目根目录创建 `.nvmrc` 文件，写入所需的 Node.js 版本号（如 `20` 或 `20.11.0`）。在项目目录下运行 `nvm use` 即可自动切换到该版本。大多数 CI/CD 系统和部署平台都支持通过 `.nvmrc` 自动选择 Node.js 版本。

十一、Docker 容器化完全指南

11.1 Docker 核心概念

Docker 是一种操作系统级别的虚拟化技术（容器化技术），它不同于传统的虚拟机（如 VMware、VirtualBox）。Docker 容器共享宿主机的操作系统内核，但在用户空间拥有独立的文件系统、网络栈和进程空间，因此启动速度极快（秒级）且资源开销极小。

Docker 架构中的核心组件：

- **Docker Daemon (dockerd)：** 在后台运行的守护进程，负责管理镜像、容器、网络 and 存储卷。接收来自 Docker Client 的 API 请求并执行相应操作。
- **Docker Client (docker)：** 命令行工具，用户通过它与 Docker Daemon 交互。每次输入 docker 命令时，实际上是在向 Docker Daemon 的 REST API 发送请求。
- **Docker Registry：** 用于存储和分发 Docker 镜像的仓库服务。Docker Hub 是最著名的公共 Registry，你也可以搭建私有 Registry。
- **Docker Compose：** 用于定义和运行多容器 Docker 应用的工具。通过一个 YAML 文件配置应用的服务、网络 and 卷，然后用一条命令启动所有服务。

核心概念：

- **Image (镜像)：** 一个只读的模板，包含了运行应用所需的操作系统、依赖库、代码和配置。镜像由多层文件系统叠加而成（Union File System），每一层都是只读的。镜像是容器的「蓝图」。
- **Container (容器)：** 镜像的一个运行实例。容器在镜像的基础上添加了一个可写层，所有运行时修改都发生在这个可写层中。容器之间相互隔离，就像一台独立的微型服务器。容器可以被启动、停止、删除、迁移。
- **Dockerfile：** 一个纯文本文件，包含了一系列指令，用于自动化构建 Docker 镜像。每条指令都会创建一个新的镜像层。
- **Volume (卷)：** Docker 管理的数据持久化机制。容器本身是无状态的——删除容器时，容器可写层中的所有数据都会丢失。Volume 将数据存储在宿主机上，独立于容器的生命周期。
- **Network (网络)：** Docker 的虚拟网络系统，允许容器之间、容器与宿主机之间、容器与外部网络之间进行通信。
- **Bind Mount (绑定挂载)：** 将宿主机上的一个目录或文件直接映射到容器内部。与 Volume 不同，Bind Mount 的路径由用户指定，Docker 不会自动管理。

Docker vs 虚拟机：

对比维度	Docker 容器	传统虚拟机
启动速度	秒级	分钟级
资源占用	MB 级内存，共享宿主机内核	GB 级内存，需要完整 Guest OS
隔离级别	进程级别隔离	操作系统级别隔离
镜像大小	通常几十 MB 到几百 MB	通常几 GB
性能	接近原生性能（无 Hypervisor 损耗）	有一定的性能损耗
跨平台	共享宿主机内核，Linux 容器只能在 Linux 上原生运行	完全跨平台

11.2 在 macOS 上安装 Docker

macOS 上使用 Docker 需要通过轻量级 Linux 虚拟机来运行 Docker 引擎（因为 Docker 依赖 Linux 内核特性）。有以下几种安装方式：

```
# 方式一：通过 Homebrew 安装 Docker Desktop (推荐, GUI + CLI)
brew install --cask docker

# 方式二：仅安装命令行工具 (无 GUI, 适合服务器或轻量使用)
brew install docker docker-compose

# 方式三：使用 Colima (轻量级 Docker 运行时替代方案, 资源占用更低)
brew install colima docker
colima start

# 方式四：使用 OrbStack (macOS 原生 Docker 替代品, 性能更好)
brew install --cask orbstack

# 验证安装
docker --version
docker compose version

# 测试 Docker 是否正常工作
docker run hello-world
```

11.3 镜像管理

```
# --- 搜索与下载镜像 ---

# 在 Docker Hub 上搜索镜像
docker search <关键词>
docker search --filter=stars=100 <关键词>    # 只显示 100 星以上的
```

```
docker search --limit 10 <关键词> # 限制结果数

# 从 Registry 拉取 (下载) 镜像
docker pull <镜像名>:<标签>
# 如果不指定标签, 默认拉取 latest 标签
docker pull python:3.11

# 拉取指定摘要 (digest) 的镜像 (精确到某个具体构建)
docker pull <镜像名>@sha256:<hash>

# 从私有 Registry 拉取
docker pull <registry地址>/<镜像名>:<标签>

# 拉取镜像的所有标签 (不常用, 会下载大量数据)
docker pull --all-tags <镜像名>

# --- 查看镜像 ---

# 列出本地所有镜像
docker images
# 或
docker image list

# 仅显示镜像 ID
docker images --quiet

# 显示虚悬镜像 (dangling images, 标签为 <none> 的中间层镜像)
docker images --filter dangling=true

# 显示特定仓库的镜像
docker images <仓库名>

# 格式化输出 (只显示仓库名和标签)
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"

# 查看镜像的详细元数据 (层信息、环境变量、暴露端口等)
docker image inspect <镜像名>:<标签>

# 查看镜像的构建历史 (每一层的大小和创建命令)
docker image history <镜像名>:<标签>

# --- 构建镜像 ---

# 从当前目录的 Dockerfile 构建镜像
docker build --tag <镜像名>:<标签> .

# 指定 Dockerfile 路径
docker build --tag <镜像名>:<标签> --file /path/to/Dockerfile .

# 构建时不使用缓存 (确保获取最新依赖)
docker build --no-cache --tag <镜像名>:<标签> .

# 为镜像打多个标签
docker build --tag <镜像名>:latest --tag <镜像名>:v1.0 .
```

```
# 设置构建参数
docker build --build-arg <参数名>=<值> --tag <镜像名>:<标签> .

# 指定构建平台 (如为 ARM 架构构建)
docker build --platform linux/arm64 --tag <镜像名>:<标签> .

# --- 标记与推送 ---

# 为已有镜像添加新标签 (相当于创建一个引用副本)
docker tag <源镜像>:<源标签> <目标镜像>:<目标标签>

# 推送到 Docker Hub
docker push <用户名>/<镜像名>:<标签>

# 推送到其他 Registry
docker push <registry地址>/<镜像名>:<标签>

# 推送所有标签
docker push --all-tags <用户名>/<镜像名>

# --- 删除镜像 ---

# 删除镜像
docker rmi <镜像名>:<标签>
# 或
docker image rm <镜像名>:<标签>

# 强制删除 (即使有容器在使用)
docker rmi --force <镜像名>:<标签>

# 删除所有未被使用的镜像
docker image prune

# 删除所有镜像 (包括有标签的)
docker image prune --all

# 删除虚悬镜像 (标签为 <none> 的)
docker image prune --filter "dangling=true"

# --- 保存与加载 ---

# 将镜像导出为 tar 文件
docker save --output <文件名>.tar <镜像名>:<标签>

# 从 tar 文件加载镜像
docker load --input <文件名>.tar

# 导出镜像并通过管道压缩
docker save <镜像名>:<标签> | gzip > <文件名>.tar.gz

# 从压缩文件加载
gunzip -c <文件名>.tar.gz | docker load

# --- 清理空间 ---
```

```
# 一次性清理所有未使用的资源（镜像、容器、卷、网络）
docker system prune

# 清理所有未使用的资源（包括未使用的镜像，更彻底）
docker system prune --all --volumes

# 查看 Docker 磁盘使用情况
docker system df

# 以详细模式查看
docker system df --verbose
```

11.4 容器管理

```
# --- 运行容器 ---

# 最基本的前台运行（终端会阻塞，Ctrl+C 停止容器）
docker run <镜像名>

# 后台运行（detached mode，返回容器 ID）
docker run --detach <镜像名>
# 简写
docker run -d <镜像名>

# 命名容器（方便后续管理，而非使用随机生成的名称）
docker run --name <容器名> <镜像名>

# 端口映射（将宿主机端口映射到容器端口）
docker run --publish 8080:80 <镜像名>
# 简写
docker run -p 8080:80 <镜像名>
# 映射到宿主机随机端口
docker run -P <镜像名>

# 挂载卷或目录
docker run --volume /宿主机路径:/容器路径 <镜像名>
# 简写
docker run -v /宿主机路径:/容器路径 <镜像名>
# 挂载命名卷
docker run -v <卷名>:/容器路径 <镜像名>
# 只读挂载
docker run -v /宿主机路径:/容器路径:ro <镜像名>

# 设置环境变量
docker run --env <变量名>=<值> <镜像名>
# 简写
docker run -e <变量名>=<值> <镜像名>
# 从文件加载多个环境变量
docker run --env-file <文件名> <镜像名>

# 容器退出后自动删除（适合一次性任务）
```

```
docker run --rm <镜像名>
```

以交互模式运行并分配伪终端（适合需要手动操作的情况）

```
docker run --interactive --tty <镜像名> /bin/bash
```

简写

```
docker run -it <镜像名> /bin/bash
```

限制资源使用

```
docker run --memory="512m" --cpus="1.5" <镜像名>
```

```
docker run --memory="1g" --memory-swap="2g" <镜像名>
```

设置重启策略

```
docker run --restart=always <镜像名>
```

始终自动重启

```
docker run --restart=unless-stopped <镜像名>
```

除非手动停止

```
docker run --restart=on-failure:3 <镜像名>
```

仅失败时重启，最多 3 次

设置工作目录

```
docker run --workdir /app <镜像名>
```

简写

```
docker run -w /app <镜像名>
```

添加主机名映射

```
docker run --add-host <主机名>:<IP地址> <镜像名>
```

--- 查看容器 ---

列出正在运行的容器

```
docker ps
```

或

```
docker container list
```

列出所有容器（包括已停止的）

```
docker ps --all
```

简写

```
docker ps -a
```

仅显示容器 ID

```
docker ps --quiet
```

显示最近创建的 N 个容器

```
docker ps --last 5
```

显示容器大小（包括可写层）

```
docker ps --size
```

格式化输出

```
docker ps --format "table {{.ID}}\t{{.Names}}\t{{.Status}}\t{{.Ports}}"
```

查看容器详细信息（完整 JSON 格式）

```
docker inspect <容器名或ID>
```

查看容器日志

```
docker logs <容器名或ID>
```

持续跟踪日志（类似 tail -f）

```
docker logs --follow <容器名或ID>
# 简写
docker logs -f <容器名或ID>

# 显示最近 N 行日志
docker logs --tail 50 <容器名或ID>

# 显示时间戳
docker logs --timestamps <容器名或ID>

# 查看容器内进程
docker top <容器名或ID>

# 查看容器资源使用统计（CPU、内存、网络、磁盘）
docker stats
docker stats <容器名或ID>

# --- 容器生命周期管理 ---

# 停止容器（发送 SIGTERM 信号，等待优雅退出）
docker stop <容器名或ID>

# 立即强制停止（发送 SIGKILL 信号）
docker kill <容器名或ID>

# 启动已停止的容器
docker start <容器名或ID>

# 重启容器
docker restart <容器名或ID>

# 暂停容器进程（冻结进程，不释放内存）
docker pause <容器名或ID>

# 恢复暂停的容器
docker unpause <容器名或ID>

# 删除已停止的容器
docker rm <容器名或ID>

# 强制删除运行中的容器
docker rm --force <容器名或ID>

# 删除所有已停止的容器
docker container prune

# 删除容器及其关联的匿名卷
docker rm --volumes <容器名或ID>

# --- 容器交互 ---

# 在运行中的容器内执行命令
docker exec <容器名或ID> <命令>

# 进入容器的交互式 Shell
```

```
docker exec --interactive --tty <容器名或ID> /bin/bash
# 简写
docker exec -it <容器名或ID> /bin/bash
# 如果 bash 不可用, 尝试
docker exec -it <容器名或ID> /bin/sh

# 以指定用户身份执行命令
docker exec --user root <容器名或ID> <命令>

# 设置环境变量后执行
docker exec -e <变量名>=<值> <容器名或ID> <命令>

# 将文件从宿主机复制到容器
docker cp <宿主机路径> <容器名或ID>:<容器路径>

# 将文件从容器复制到宿主机
docker cp <容器名或ID>:<容器路径> <宿主机路径>
```

11.5 Dockerfile 编写指南

Dockerfile 是 Docker 镜像的构建脚本，每一条指令都会在基础镜像之上创建一个新的文件系统层。

常用指令说明：

指令	说明	用法
FROM	指定基础镜像，必须是 Dockerfile 的第一条指令	FROM python:3.11-slim
LABEL	添加元数据标签（作者、版本、描述等）	LABEL maintainer="name@example.com"
RUN	在镜像构建时执行命令（用于安装依赖、编译代码等）	RUN apt-get update && apt-get install -y curl
COPY	将宿主机文件复制到镜像中（推荐使用 COPY 而非 ADD）	COPY . /app
ADD	功能比 COPY 更强大（支持 URL 下载和自动解压 tar 文件），但行为不透明	ADD https://example.com/file.tar.gz /tmp/
WORKDIR	设置后续指令的工作目录（相当于 cd）	WORKDIR /app
ENV	设置环境变量（在构建和运行时都生效）	ENV NODE_ENV=production
ARG	定义构建参数（仅在构建时可用，通过 --build-arg 传入）	ARG VERSION=1.0
EXPOSE	声明容器运行时监听的端口（仅文档作用，不实际发布端口）	EXPOSE 8080
CMD	容器启动时的默认命令（可被 docker run 后面的参数覆盖）	CMD ["python", "app.py"]
ENTRYPOINT	容器入口点（不能被 docker run 后的参数覆盖，参数作为追加）	ENTRYPOINT ["python"]
VOLUME	创建匿名卷挂载点	VOLUME ["/data"]
USER	切换运行用户	USER nobody
HEALTHCHECK	定义容器的健康检查命令	HEALTHCHECK CMD curl -f http://localhost/ exit 1

Dockerfile 最佳实践：

1. 使用尽可能小的基础镜像：如 python:3.11-slim（基于 Debian 精简版）或 python:3.11-alpine（基于 Alpine Linux，体积极小但可能缺少某些库）
2. 合并 RUN 指令：将多个 RUN 合并为一个，减少镜像层数。用 && 串联命令，用 \ 换行
3. 清理缓存：在同一个 RUN 指令中安装依赖后立即清理包管理器缓存
4. COPY 优于 ADD：除非需要自动解压 tar 文件，否则始终使用 COPY
5. 利用多阶段构建：在第一个阶段编译代码，在第二个阶段仅复制编译产物，大幅减小最终镜像体积
6. 不要以 root 用户运行：通过 USER 指令切换到非特权用户
7. 使用 .dockerignore：类似于 .gitignore，排除不必要的文件（node_modules/、.git/ 等）送入构建上下文

11.6 Docker Compose

Docker Compose 用于定义和管理多容器应用。通过一个 `compose.yml` (或 `docker-compose.yml`) YAML 文件, 你可以定义多个服务、网络和卷, 并用一条命令统一管理它们的生命周期。

```
# --- 基本命令 ---

# 启动所有服务 (后台运行, 如果镜像不存在则先构建)
docker compose up --detach

# 简写
docker compose up -d

# 启动并强制重新构建镜像
docker compose up --build

# 前台启动 (查看所有日志输出, Ctrl+C 停止)
docker compose up

# 停止所有服务 (不删除容器)
docker compose stop

# 启动已停止的服务
docker compose start

# 停止并删除所有服务容器和网络
docker compose down

# 停止并删除所有内容 (包括卷和镜像)
docker compose down --volumes --rmi all

# 重启服务
docker compose restart

# 查看所有服务的运行状态
docker compose ps

# 查看所有服务的日志
docker compose logs

# 跟踪日志输出
docker compose logs --follow

# 查看指定服务的日志
docker compose logs <服务名>

# 在运行中的服务中执行命令
docker compose exec <服务名> <命令>
docker compose exec <服务名> /bin/bash

# 列出所有服务使用的镜像
docker compose images

# 暂停/恢复服务
docker compose pause
```

```
docker compose unpause

# 查看服务运行的端口映射
docker compose port <服务名> <容器内部端口>

# 查看资源使用
docker compose top

# 拉取服务镜像（不启动）
docker compose pull

# 构建服务镜像（不启动）
docker compose build

# 强制重新构建（不使用缓存）
docker compose build --no-cache

# 验证和查看合并后的 Compose 配置
docker compose config

# 缩放指定服务的实例数量
docker compose up --scale <服务名>=3

# --- 运行一次性任务 ---
# 启动一个新容器运行一次性命令（区别于 exec 是在已有容器中运行）
docker compose run <服务名> <命令>
docker compose run --rm <服务名> <命令>      # 命令结束后自动删除容器
```

11.7 卷 (Volume) 管理

```
# 列出所有卷
docker volume list

# 创建卷
docker volume create <卷名>

# 查看卷详细信息（在宿主机上的实际路径等）
docker volume inspect <卷名>

# 删除卷
docker volume rm <卷名>

# 删除所有未使用的卷
docker volume prune

# 备份卷（将卷内容导出为 tar）
docker run --rm -v <卷名>:/data -v $(pwd):/backup alpine tar czf /backup/backup.tar.gz -C /data .

# 恢复卷（从 tar 文件导入）
```

```
docker run --rm -v <卷名>:/data -v $(pwd):/backup alpine tar xzf /backup/backup.tar.gz -C /data
```

11.8 网络管理

列出所有网络

```
docker network list
```

创建自定义网络

```
docker network create <网络名>
```

创建可与其他 Compose 项目通信的网络

```
docker network create --driver bridge <网络名>
```

查看网络详细信息（连接到该网络的容器等）

```
docker network inspect <网络名>
```

将运行中的容器连接到网络

```
docker network connect <网络名> <容器名>
```

断开容器与网络的连接

```
docker network disconnect <网络名> <容器名>
```

删除网络

```
docker network rm <网络名>
```

清理未使用的网络

```
docker network prune
```

十二、Make 与 Makefile 构建自动化

12.1 简介

Make 是一个经典的构建自动化工具，最初用于 C/C++ 程序的编译构建，但现在广泛用于各种项目的任务自动化——从编译代码到运行测试、部署应用、管理环境等。Make 通过读取 Makefile（或 makefile）中定义的目标、依赖和规则来自动化执行重复性的命令序列。

Make 的核心优势：

- 增量构建：Make 通过比较文件的时间戳，只重新构建那些依赖发生了变化的文件，而非每次全量构建
- 声明式依赖管理：在 Makefile 中定义目标之间的依赖关系，Make 自动确定执行顺序
- 无处不在：Make 预装在几乎所有的 macOS 和 Linux 系统中，无需额外安装
- 语言无关：虽然最初为 C 而生，但 Make 可以用于任何有命令行工具的项目

12.2 Makefile 基本语法

```
# 基本规则结构：
# 目标：依赖1 依赖2 ...
#   命令1
#   命令2
# 注意：命令前的缩进必须是 Tab 字符（不是空格！）

# 变量定义
PYTHON := python3
PIP := pip3
VENV := .venv
SRC_DIR := src
TEST_DIR := tests

# 伪目标（不生成同名文件的目标，如 install、clean、test）
.PHONY: help install build test clean lint format

# 默认目标（make 不带参数时执行的目标，通常是第一个定义的目标）
.DEFAULT_GOAL := help

help:
    @echo "可用命令："
    @echo "  make install  - 创建虚拟环境并安装依赖"
    @echo "  make test     - 运行测试"
    @echo "  make lint     - 代码检查"
    @echo "  make format   - 代码格式化"
    @echo "  make clean    - 清理构建产物"
    @echo "  make build    - 构建项目"

install: $(VENV)/bin/activate
```

```

$(VENV)/bin/activate: requirements.txt
    $(PYTHON) -m venv $(VENV)
    $(VENV)/bin/$(PIP) install --upgrade pip
    $(VENV)/bin/$(PIP) install -r requirements.txt
    @touch $(VENV)/bin/activate

build:
    $(VENV)/bin/$(PYTHON) -m build

test:
    $(VENV)/bin/$(PYTHON) -m pytest $(TEST_DIR) -v --cov=$(SRC_DIR)

lint:
    $(VENV)/bin/$(PYTHON) -m ruff check $(SRC_DIR)
    $(VENV)/bin/$(PYTHON) -m mypy $(SRC_DIR)

format:
    $(VENV)/bin/$(PYTHON) -m ruff format $(SRC_DIR)

clean:
    find . -type d -name "__pycache__" -exec rm -rf {} + 2>/dev/null || true
    find . -type f -name "*.pyc" -delete
    rm -rf $(VENV) dist build *.egg-info .pytest_cache .mypy_cache .coverage htmlcov

```

12.3 Makefile 高级特性

自动变量:

变量	含义
\$@	当前目标名称
\$<	第一个依赖的名称
\$^	所有依赖的名称（去重）
\$?	所有比目标更新的依赖名称
\$*	目标名称的主干部分（不含扩展名）

模式规则:

```

# 将所有的 .c 文件编译为对应的 .o 文件
%.o: %.c
    $(CC) -c $(CFLAGS) $< -o $@

```

条件判断:

```
ifeq ($(OS),Darwin)    # macOS
    PLATFORM := macos
else
    PLATFORM := linux
endif
```

函数:

```
# 通配符函数
SOURCES := $(wildcard src/*.py)

# 字符串替换函数
OBJECTS := $(SOURCES:.py=.pyc)

# Shell 命令执行
GIT_HASH := $(shell git rev-parse --short HEAD)
```

12.4 常用 Make 命令行用法

```
# 执行默认目标 (Makefile 中的第一个目标)
make

# 执行指定目标
make test
make clean

# 并行构建 (-j 指定并行任务数, -j 不指定则无限制)
make -j4          # 4 个并行任务
make -j           # 不限制并行数 (注意: 可能耗尽资源)

# 模拟运行 (只显示将执行的命令, 不实际执行, 用于验证 Makefile)
make -n
make --dry-run

# 在指定目录下执行 Makefile
make -C /path/to/project
make -C build

# 指定 Makefile 文件 (而非默认的 Makefile 或 makefile)
make -f custom.mk

# 安静模式 (不打印执行的命令, 只显示输出)
make -s
make --silent

# 始终执行 (忽略文件时间戳, 强制重新构建所有目标)
make -B
make --always-make

# 传入变量覆盖 Makefile 中的默认值
```

```

make PREFIX=/usr/local
make VERSION=2.0.0 build
make CFLAGS="-O2 -Wall"

# 忽略构建过程中的错误，继续执行后续目标
make -i
make --ignore-errors

# 出错时立即停止（默认行为，与 -i 相反）
make -k # 出错后跳过当前目标，继续其他不依赖它的目标
make --keep-going

# 显示 Makefile 中定义的所有变量和目标（调试利器）
make -p # 打印所有变量和规则
make -p | grep "^[a-zA-Z]" # 粗略列出所有目标

# 打印变量值（无需自定义目标）
make -p | grep "^VARIABLE_NAME"

# 查看 make 版本信息
make --version

```

12.5 just —— 现代命令运行器

just 是一个用 Rust 编写的命令运行器，定位与 Make 不同——just 不做增量构建（不比较文件时间戳），它纯粹是一个项目命令的快捷入口，类似于 npm scripts 但语言无关。

just 与 Make 的核心区别：

维度	Make	just
增量构建	✅ 核心功能（比较时间戳）	❌ 不支持
语法	复杂的 Makefile 语法	简洁直观
错误处理	默认遇到错误停止	可逐条控制
跨平台	依赖 Unix Shell	原生跨平台（Win/Mac/Linux）
学习成本	高（自动变量、模式规则）	低（直觉式语法）
适用场景	C/C++ 编译、需要增量构建	跨语言项目的通用任务管理

安装：

```
brew install just
```

基本语法（justfile）：

```

# 变量定义
python := "python3"
venv := ".venv"

# 默认命令 (just 不带参数时执行)
default:
    @just --list

# 基本命令
install:
    {{python}} -m venv {{venv}}
    {{venv}}/bin/pip install -r requirements.txt

# 带依赖的命令
test: lint
    {{venv}}/bin/pytest -v

lint:
    {{venv}}/bin/ruff check .

# 带参数的命令
release VERSION:
    @echo "Releasing version {{VERSION}}..."
    git tag -a "v{{VERSION}}" -m "Release {{VERSION}}"

# 可选参数 (使用默认值)
deploy BRANCH="main":
    @echo "Deploying branch {{BRANCH}}"
    git push origin {{BRANCH}}

# 多行命令
build:
    #!/bin/bash
    set -e
    echo "Building..."
    {{python}} -m build

# 在不同目录执行
format:
    cd src && black .
    cd tests && black .

# [private] 标记的 recipe 不会出现在 just --list 中
[private]
_secret-task:
    @echo "This is hidden"

```

常用命令:

```

# 列出所有可用命令 (recipe)
just --list
just -l

# 执行指定命令

```



```
just test
just build release

# 传入参数
just release 1.5.0
just deploy main

# 选择 justfile 位置
just --justfile /path/to/justfile

# 在工作目录的父目录链中搜索 justfile (默认行为)
just --working-directory /path/to/project

# 初始化 justfile
just --init

# 模拟运行 (打印命令但不执行)
just --dry-run

# 显示 justfile 的完整内容 (展开所有变量)
just --dump

# 使用不同的 Shell
just --shell bash
```

just vs Make 选择建议:

- 如果你的项目需要 C/C++ 编译且有增量构建需求 → **Make**
 - 如果你只是想要一个跨语言、跨平台的项目命令入口 → **just**
 - 两者可以共存: Make 管编译, just 管日常命令
-

十三、常用系统工具

13.1 *rsync* —— 文件同步利器

rsync 是一个高效的文件同步和传输工具，通过增量传输算法只传输文件的变化部分，非常适合目录同步、备份和远程文件拷贝。

```
# 基本语法
rsync [选项] <源路径> <目标路径>

# 常用同步方式

# 将本地目录同步到远程（归档模式，保留权限、时间戳、软链接等）
rsync -avz /local/path/ user@hostname:/remote/path/

# 从远程同步到本地
rsync -avz user@hostname:/remote/path/ /local/path/

# 模拟运行（不实际传输，只显示将要传输的文件）
rsync -avzn /source/ /destination/

# 删除目标端多余的文件（使目标成为源的精确镜像）
rsync -avz --delete /source/ /destination/

# 排除特定文件或目录
rsync -avz --exclude '*.tmp' --exclude 'node_modules/' --exclude '.git/' /source/
/destination/

# 从文件读取排除列表
rsync -avz --exclude-from='exclude-list.txt' /source/ /destination/

# 限制传输带宽（KB/s）
rsync -avz --bwlimit=1000 /source/ user@hostname:/remote/path/

# 断点续传（传输中断后可以从断点继续）
rsync -avz --partial --progress /source/ user@hostname:/remote/path/

# 显示传输进度
rsync -avz --progress /source/ /destination/

# 仅传输大小或时间不同的文件（默认行为是比较大小和时间）
rsync -avz --size-only /source/ /destination/
rsync -avz --checksum /source/ /destination/ # 用校验和比较（更准确但更慢）

# 压缩传输（在慢网络下加速）
rsync -avz /source/ user@hostname:/remote/path/

# 通过 SSH 和指定端口
rsync -avz -e "ssh -p 2222" /source/ user@hostname:/remote/path/
```

rsync 核心选项速查：

选项	说明
-a	归档模式（相当于 -rlptgoD，保留几乎所有文件属性）
-v	详细输出模式
-z	传输时压缩数据
-n	模拟运行（dry-run）
-P	等同于 --partial --progress，支持断点续传并显示进度
--delete	删除目标端多余文件
--exclude	排除指定模式的文件/目录
--bwlimit	限制带宽（KB/s）
-h	人类可读的数字格式

路径末尾斜杠的区别（极易出错）：

- /source/ → 将 source 目录的内容复制到目标目录
- /source → 将 source 目录本身复制到目标目录内

13.2 sed —— 流编辑器

sed (Stream Editor) 是一个非交互式的文本流编辑器，适合在管道中对文本进行查找、替换、删除、插入等操作。它逐行读取输入，执行编辑命令，然后输出结果。

```
# --- 替换操作 ---

# 替换每行第一个匹配项
sed 's/old/new/' file.txt

# 替换每行所有匹配项
sed 's/old/new/g' file.txt

# 直接修改文件（原地编辑）
sed -i '' 's/old/new/g' file.txt      # macOS
sed -i 's/old/new/g' file.txt        # Linux

# 先备份再原地编辑
sed -i '.bak' 's/old/new/g' file.txt  # macOS, 备份为 file.txt.bak

# 只替换第 N 次出现的匹配项
sed 's/old/new/3' file.txt            # 替换每行第 3 次出现的 old
```

```

# 使用正则表达式替换
sed 's/[0-9]\+/NUMBER/g' file.txt
sed -E 's/[0-9]+\+/NUMBER/g' file.txt      # macOS (-E 启用扩展正则)

# 替换时忽略大小写
sed 's/old/new/gI' file.txt                # GNU sed
sed -E 's/[0o][Ll][Dd]/new/g' file.txt    # macOS 替代写法

# 使用不同的分隔符 (当搜索模式中包含 / 时)
sed 's|usr/local/bin|opt/bin|g' file.txt
sed 's#http://old.com#https://new.com#g' file.txt

# --- 行操作 ---

# 打印指定行
sed -n '5p' file.txt                       # 打印第 5 行
sed -n '5,10p' file.txt                   # 打印第 5-10 行
sed -n '/pattern/p' file.txt              # 打印匹配行

# 删除行
sed '5d' file.txt                         # 删除第 5 行
sed '5,10d' file.txt                     # 删除第 5-10 行
sed '/pattern/d' file.txt                 # 删除匹配行
sed '/^$/d' file.txt                     # 删除空行
sed '/^#/d' file.txt                     # 删除注释行

# 在行前/后插入
sed '5i\新行内容' file.txt               # 在第 5 行前插入
sed '5a\新行内容' file.txt               # 在第 5 行后追加

# --- 多命令 ---

# 执行多个 sed 命令
sed -e 's/foo/bar/g' -e 's/baz/qux/g' file.txt
# 或用分号分隔
sed 's/foo/bar/g; s/baz/qux/g' file.txt

# 只操作匹配特定模式的行
sed '/pattern/s/old/new/g' file.txt      # 只在匹配 pattern 的行中替换

# 打印被替换的行 (结合 -n 和 p 标志)
sed -n 's/old/new/gp' file.txt

```

macOS 与 Linux 的 sed 差异:

- macOS 使用 BSD sed, Linux 使用 GNU sed
- 原地编辑: macOS 必须提供备份后缀 (-i '' 表示不备份), Linux 可以直接 -i
- 扩展正则: macOS 用 -E, Linux 用 -r (新版也支持 -E)
- 一些高级 GNU sed 特性在 macOS sed 中不可用
- 建议: 安装 GNU sed 以获得跨平台一致性: `brew install gnu-sed`, 然后使用 `gsed` 命令

13.3 awk——文本处理语言

awk 是一个功能强大的文本处理语言和命令行工具，特别适合处理结构化文本数据（CSV、日志、表格等）。它将每行拆分为字段，允许你基于字段进行过滤、转换和统计。

```
# 基本语法
awk '条件 { 动作 }' 文件

# ---- 字段处理 ----

# 打印指定列 ($0=整行, $1=第一列, $2=第二列, $NF=最后一列)
awk '{ print $1 }' file.txt           # 打印第一列
awk '{ print $1, $3 }' file.txt       # 打印第一列和第三列
awk '{ print $NF }' file.txt          # 打印最后一列
awk '{ print $(NF-1) }' file.txt      # 打印倒数第二列

# 指定分隔符 (默认是空白字符)
awk -F ',' '{ print $2 }' data.csv    # 逗号分隔
awk -F ':' '{ print $1 }' /etc/passwd # 冒号分隔
awk -F '\t' '{ print $1 }' data.tsv   # 制表符分隔

# ---- 过滤与条件 ----

# 按字段值过滤
awk '$3 > 100 { print $0 }' data.txt  # 第三列大于 100
awk '$1 == "ERROR" { print $0 }' log.txt # 第一列等于 ERROR
awk '$2 ~ /pattern/ { print $0 }' file.txt # 第二列匹配正则
awk 'length($0) > 80 { print $0 }' file.txt # 行长度大于 80

# ---- 内置变量 ----

# NR = 行号, NF = 字段数, FILENAME = 当前文件名
awk '{ print NR, $0 }' file.txt        # 打印带行号的内容
awk '{ print NF, $0 }' file.txt        # 打印每行字段数
awk 'NR >= 5 && NR <= 10 { print $0 }' file.txt # 打印第 5-10 行

# ---- BEGIN 和 END ----

# BEGIN 块在读取任何输入之前执行
# END 块在所有输入处理完成后执行
awk 'BEGIN { print "Start" } { print $0 } END { print "End" }' file.txt

# 统计
awk '{ sum += $1 } END { print "Sum:", sum }' data.txt
awk '{ sum += $1; count++ } END { print "Average:", sum/count }' data.txt

# ---- 格式化输出 ----

# 使用 printf 格式化
awk '{ printf "%-20s %10d\n", $1, $2 }' data.txt # 左对齐字符串, 右对齐数字

# 设置输出字段分隔符
awk 'BEGIN { OFS="\t" } { print $1, $2, $3 }' data.txt
```

```
# --- 常用实战 ---
```

```
# 分析访问日志：统计每个 IP 的访问次数
```

```
awk '{ count[$1]++ } END { for (ip in count) print count[ip], ip }' access.log | sort -rn
```

```
# 合并两个文件（基于相同字段）
```

```
awk 'NR==FNR { map[$1]=$2; next } { print $0, map[$1] }' file1.txt file2.txt
```

13.4 cron / launchd —— 定时任务

cron（适用于 macOS 和 Linux）：

```
# 编辑当前用户的定时任务
```

```
crontab -e
```

```
# 查看当前用户的定时任务
```

```
crontab -l
```

```
# 删除所有定时任务
```

```
crontab -r
```

```
# 编辑其他用户的定时任务（需要权限）
```

```
crontab -u <用户名> -e
```

crontab 时间格式（分 时 日 月 周）：

```
* * * * * 命令
```

```
| | | | |  
| | | | └─ 星期（0-7，0 和 7 都表示周日）  
| | | └─ 月份（1-12）  
| | └─ 日期（1-31）  
| └─ 小时（0-23）  
└─ 分钟（0-59）
```

```
# 常见时间表达式
```

```
0 2 * * * # 每天凌晨 2 点
```

```
*/5 * * * * # 每 5 分钟
```

```
0 9-17 * * 1-5 # 工作日（周一至周五）每小时的整点
```

```
0 0 1 * * # 每月 1 号午夜
```

```
@daily # 每天午夜（等同于 0 0 * * *）
```

```
@hourly # 每小时
```

```
@reboot # 系统启动时
```

macOS launchd（推荐在 macOS 上使用，更可靠）：

```
# launchd 是 macOS 的原生任务调度系统，比 cron 更适合 macOS
```

```
# 配置文件路径（用户级别）
```

```
~/Library/LaunchAgents/
```

```
# 列出当前用户的定时任务
launchctl list

# 加载配置
launchctl load ~/Library/LaunchAgents/com.user.mytask.plist

# 卸载配置
launchctl unload ~/Library/LaunchAgents/com.user.mytask.plist

# 手动启动任务
launchctl start com.user.mytask

# 查看任务状态
launchctl list | grep com.user.mytask
```

launchd plist 文件结构 (com.user.mytask.plist) :

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>
  <string>com.user.mytask</string>
  <key>ProgramArguments</key>
  <array>
    <string>/path/to/script.sh</string>
  </array>
  <key>StartCalendarInterval</key>
  <dict>
    <key>Hour</key>
    <integer>2</integer>
    <key>Minute</key>
    <integer>0</integer>
  </dict>
  <key>StandardOutPath</key>
  <string>/tmp/mytask.stdout.log</string>
  <key>StandardErrorPath</key>
  <string>/tmp/mytask.stderr.log</string>
</dict>
</plist>
```

13.5 进程与端口管理

```
# --- 进程查看 ---

# 列出当前 Shell 的后台任务
jobs
jobs -l # 同时显示 PID
```

```
# 查看系统所有进程（动态刷新，按 q 退出）
top
htop # 更友好的交互式进程查看器 (brew install htop)

# 查看进程树
pstree # 如果不可用: brew install pstree

# 按名称查找进程 PID
pgrep <进程名>
pgrep -f <完整命令关键词> # 匹配完整命令行
pgrep -l <进程名> # 同时显示进程名

# 查看进程详细信息
ps aux | grep <进程名>
ps -ef # 完整格式

# --- 终止进程 ---

# 按 PID 终止进程
kill <PID> # 发送 SIGTERM（优雅终止）
kill -9 <PID> # 发送 SIGKILL（强制终止，不推荐首选）
kill -15 <PID> # SIGTERM，同默认 kill

# 按名称终止
pkill <进程名> # 终止匹配的所有进程
pkill -f <完整命令匹配> # 按完整命令行匹配
killall <进程名> # 终止所有同名进程 (macOS)

# 终止挂起的进程
kill -STOP <PID> # 暂停进程 (SIGSTOP)
kill -CONT <PID> # 恢复暂停的进程 (SIGCONT)

# --- 后台任务管理 ---

# 在命令末尾加 & 使其在后台运行
<耗时命令> &
sleep 300 &

# 将当前前台任务暂停并放入后台
# 在任务运行中按 Ctrl + Z (发送 SIGTSTP 暂停进程)

# 查看后台任务
jobs

# 将后台任务调到前台
fg # 调出最近的后台任务
fg %1 # 调出编号为 1 的后台任务

# 让暂停的后台任务继续在后台运行
bg %1

# 使任务脱离终端（终端关闭后继续运行）
nohup <命令> &
nohup <命令> > output.log 2>&1 &
```



```

# 将已在运行的任务脱离终端
disown %1          # 从作业列表中移除
disown -h %1       # 移除但不发送 SIGHUP

# --- 端口与网络 ---

# 查找占用指定端口的进程
lsof -i :<端口号>
lsof -i :8080
lsof -i :3000-4000  # 端口范围

# 查看所有网络监听端口及对应进程
lsof -i -P -n | grep LISTEN
netstat -an | grep LISTEN

# 查看特定进程的网络连接
lsof -i -a -p <PID>

# 查看本机所有网络接口
ifconfig
ipconfig getifaddr en0  # 本机局域网 IP

# --- 系统资源 ---

# 查看磁盘使用
df -h          # 各挂载点使用情况
du -sh <目录>  # 目录总大小
du -sh * | sort -hr  # 当前目录下子目录大小排序
du -d 1 -h | sort -hr  # 一级子目录大小

# 查找大文件 (>100MB)
find . -type f -size +100M -exec ls -lh {} \; 2>/dev/null

# 查找大目录
du -sh */ | sort -hr | head -10

# 查找最近修改的文件
find . -type f -mtime -1  # 24 小时内修改的文件

# 查看系统运行时间和负载
uptime

```

13.6 curl——HTTP 请求与 API 测试

curl 是命令行下的 HTTP 客户端，macOS 自带，用于调试 API、下载文件、测试网络服务。是后端开发者的必备工具。

```

# --- 基本请求 ---

# GET 请求（默认）

```

```
curl https://api.example.com/users

# 仅显示响应体（不显示进度条等额外信息）
curl --silent https://api.example.com/users

# 显示完整的请求-响应头信息
curl --verbose https://api.example.com/users
curl -v https://api.example.com/users      # 简写

# 仅显示响应头
curl --head https://api.example.com
curl -I https://api.example.com            # 简写

# 跟随重定向
curl --location https://short.link
curl -L https://short.link                # 简写

# --- POST / PUT / DELETE ---

# POST JSON 数据
curl -X POST https://api.example.com/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com"}'

# POST 表单数据
curl -X POST https://api.example.com/login \
  -d "username=alice" \
  -d "password=secret"

# PUT 请求（更新）
curl -X PUT https://api.example.com/users/1 \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Updated"}'

# DELETE 请求
curl -X DELETE https://api.example.com/users/1

# PATCH 请求（部分更新）
curl -X PATCH https://api.example.com/users/1 \
  -H "Content-Type: application/json" \
  -d '{"email": "new@example.com"}'

# --- 请求头与认证 ---

# 自定义请求头
curl -H "Authorization: Bearer <token>" \
  -H "X-Custom-Header: value" \
  https://api.example.com

# Basic Auth
curl --user username:password https://api.example.com
curl -u username:password https://api.example.com

# Bearer Token
curl -H "Authorization: Bearer <token>" https://api.example.com
```

从文件读取请求体

```
curl -X POST https://api.example.com/data \  
  -H "Content-Type: application/json" \  
  -d @data.json
```

--- 文件操作 ---

下载文件 (保存到指定文件名)

```
curl --output filename.zip https://example.com/file.zip  
curl -o filename.zip https://example.com/file.zip
```

下载文件 (使用远程文件名)

```
curl --remote-name https://example.com/file.zip  
curl -O https://example.com/file.zip
```

断点续传

```
curl --continue-at - -O https://example.com/large-file.zip  
curl -C - -O https://example.com/large-file.zip
```

上传文件 (multipart form)

```
curl -X POST https://api.example.com/upload \  
  -F "file=@/path/to/local/file.pdf" \  
  -F "description=my document"
```

上传多个文件

```
curl -F "file1=@/path/to/file1.jpg" \  
  -F "file2=@/path/to/file2.jpg" \  
  https://api.example.com/upload
```

--- 调试与性能 ---

显示响应状态码

```
curl --write-out '%{http_code}' --silent --output /dev/null https://example.com  
curl -w '%{http_code}\n' -s -o /dev/null https://example.com
```

显示请求耗时详情

```
curl -w '\nDNS: %{time_namelookup}s\nConnect: %{time_connect}s\nTLS: %  
{time_appconnect}s\nTTFB: %{time_starttransfer}s\nTotal: %{time_total}s\n' \  
  -s -o /dev/null https://example.com
```

将响应保存到文件同时显示到终端

```
curl https://api.example.com/data | tee response.json
```

格式化 JSON 响应 (需要安装 jq)

```
curl -s https://api.example.com/users | jq .  
curl -s https://api.example.com/users | jq '.[0].name'
```

使用 HTTP/2

```
curl --http2 https://example.com
```

--- HTTPS 与证书 ---

忽略 SSL 证书验证 (仅用于测试环境)

```
curl --insecure https://self-signed.example.com
```

```
curl -k https://self-signed.example.com

# 指定 CA 证书
curl --cacert /path/to/ca-cert.pem https://example.com

# 指定客户端证书
curl --cert client.pem --key client-key.pem https://example.com

# --- Cookie 管理 ---

# 发送 Cookie
curl -b "session_id=abc123" https://example.com

# 从文件读取 Cookie
curl -b cookies.txt https://example.com

# 保存响应 Cookie 到文件
curl -c cookies.txt https://example.com/login

# 使用 Cookie jar (先登录, 再用 Cookie 访问)
curl -c cookies.txt -X POST https://example.com/login -d "user=alice&pass=secret"
curl -b cookies.txt https://example.com/dashboard
```

HTTPIe——更友好的 curl 替代品:

```
# 安装
brew install httpie

# GET 请求 (语法更简洁, 自动格式化 JSON)
http https://api.example.com/users

# POST JSON
http POST https://api.example.com/users name=Alice email=alice@example.com

# 认证
http -a username:password https://api.example.com
http https://api.example.com Authorization:"Bearer <token>"

# 下载文件
http --download https://example.com/file.zip
```

WebSocket 测试——websocat:

```
# 安装
brew install websocat

# 连接到 WebSocket 服务
websocat ws://echo.websocket.org

# 连接并发送消息
echo "hello" | websocat ws://localhost:8080/ws

# 连接到安全的 WebSocket (wss://)
websocat wss://example.com/socket
```

```
# 带自定义 Header
websocat -H "Authorization: Bearer <token>" ws://localhost:8080/ws

# 作为 WebSocket 服务器（监听端口）
websocat -s 8080

# 双向交互模式
websocat ws://localhost:8080/ws --linemode-strip-newlines
```

gRPC 测试——grpcurl:

```
# 安装
brew install grpcurl

# 列出服务器上的所有 gRPC 服务
grpcurl -plaintext localhost:50051 list

# 查看服务的详细定义
grpcurl -plaintext localhost:50051 describe <服务名>

# 查看服务方法的输入输出类型
grpcurl -plaintext localhost:50051 describe <服务名>.<方法名>

# 调用 gRPC 方法
grpcurl -plaintext -d '{"name": "Alice"}' localhost:50051 <服务名>/<方法名>

# 使用 protoset 文件（不需要服务器反射）
grpcurl -protoset myfile.protoset -d '{"id": 1}' localhost:50051 <服务名>/<方法名>

# 带 TLS 调用
grpcurl -authority example.com example.com:443 list

# 带 Metadata（类似 HTTP Headers）
grpcurl -H "Authorization: Bearer <token>" -plaintext localhost:50051 list
```

GraphQL 测试:

```
# graphql (GraphQL CLI 客户端)
npm install -g graphql
# 或 brew install graphql

# 执行查询
gq https://api.example.com/graphql -q 'query { users { id name } }'

# 带变量的查询
gq https://api.example.com/graphql -q 'query($id: ID!) { user(id: $id) { name } }' -v '{"id": "1"}'

# Mutation
gq https://api.example.com/graphql -q 'mutation { createUser(name: "Alice") { id } }'

# 设置 Header
gq https://api.example.com/graphql -H "Authorization: Bearer <token>" -q '...'
```

```
# 使用 curl 发送 GraphQL 查询也是一种常用方式
curl -X POST https://api.example.com/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "query { users { id name email } }"}'

# 自省查询 (查看 Schema)
gq https://api.example.com/graphql --introspect
```

快速 Mock REST API——json-server:

```
# 安装
npm install -g json-server

# 创建 db.json 文件
cat > db.json << 'EOF'
{
  "users": [
    { "id": 1, "name": "Alice", "email": "alice@example.com" },
    { "id": 2, "name": "Bob", "email": "bob@example.com" }
  ],
  "posts": [
    { "id": 1, "title": "Hello World", "authorId": 1 }
  ]
}
EOF

# 启动 REST API 服务器 (自动生成 CRUD 路由)
json-server --watch db.json --port 3000
# 现在 GET/POST/PUT/DELETE http://localhost:3000/users 等接口就可用了
```

13.7 日志查看与文本处理实战

日常开发中大量操作围绕日志和文本展开——查看应用日志、搜索错误信息、统计接口调用次数、筛选字段。以下命令的组合运用能覆盖绝大多数场景。

```
# --- 日志查看 ---

# 查看文件内容
cat file.log          # 全部输出 (小文件)
less file.log         # 分页浏览 (大文件, 推荐)
# less 内快捷键: Space=下一页, b=上一页, /=搜索, n=下一个匹配, q=退出, g=文件头, G=文件尾

# 查看文件前/后 N 行
head -20 file.log     # 前 20 行
tail -20 file.log     # 最后 20 行

# 实时跟踪日志 (类似 Console.app)
tail -f app.log       # 持续显示新增内容 (Ctrl+C 退出)
tail -f *.log         # 同时跟踪多个日志文件
```

```
tail -n 100 -f app.log      # 先显示最后 100 行，再持续跟踪

# --- 搜索与过滤 ---

# 基本搜索
grep "ERROR" app.log        # 查找包含 ERROR 的行

# 忽略大小写
grep -i "error" app.log

# 递归搜索目录
grep -r "TODO" src/         # 在 src 目录下所有文件中搜索
grep -rl "TODO" src/        # 只显示文件名，不显示内容

# 显示匹配行的前后文
grep -C 3 "ERROR" app.log   # 匹配行 + 上下各 3 行 (Context)
grep -B 5 "ERROR" app.log   # 匹配行 + 前面 5 行 (Before)
grep -A 5 "ERROR" app.log   # 匹配行 + 后面 5 行 (After)

# 反向匹配 (排除行)
grep -v "DEBUG" app.log     # 不包含 DEBUG 的行
grep -v -e "DEBUG" -e "TRACE" app.log # 同时排除多个模式

# 正则表达式搜索
grep -E "ERROR|FATAL" app.log # 匹配 ERROR 或 FATAL
grep -E "^2024-05-[0-9]{2}" app.log # 匹配特定日期格式

# 统计匹配行数
grep -c "ERROR" app.log     # 有多少行包含 ERROR

# 仅显示匹配部分 (而非整行)
grep -o "[0-9]\+\.[0-9]\+\.[0-9]\+\.[0-9]\+" access.log # 提取所有 IP

# --- 列操作 ---

# 按分隔符提取列
cut -d ',' -f 1,3 data.csv   # 提取第 1 和第 3 列 (逗号分隔)
cut -d ':' -f 1 /etc/passwd  # 提取第 1 列 (冒号分隔)
cut -c 1-10 file.txt         # 提取每行第 1-10 个字符

# --- 排序与去重 ---

# 排序
sort file.txt                # 字母升序
sort -n numbers.txt          # 数字排序
sort -r file.txt             # 倒序
sort -t ',' -k 2 data.csv    # 按第 2 列排序 (逗号分隔)
sort -t ',' -k 2 -n data.csv # 按第 2 列数字排序

# 去重 (只对相邻重复行有效，通常先 sort 再 uniq)
sort file.txt | uniq         # 排序后去重
sort file.txt | uniq -c      # 去重并统计出现次数
sort file.txt | uniq -d      # 只显示重复的行

# --- 统计 ---
```

```

# 行数、词数、字符数
wc -l file.txt          # 行数
wc -w file.txt          # 词数
wc -c file.txt          # 字符/字节数

# 统计当前目录下文件数
ls | wc -l
find . -type f | wc -l

# --- 实战组合 ---

# 分析日志中 TOP 10 高频 IP
awk '{ print $1 }' access.log | sort | uniq -c | sort -rn | head -10

# 统计日志中各 HTTP 状态码的数量
awk '{ print $9 }' access.log | sort | uniq -c | sort -rn

# 查找所有 Python 文件的总行数
find . -name "*.py" -exec cat {} + | wc -l
find . -name "*.py" | xargs wc -l | tail -1

# 实时监控日志中的错误
tail -f app.log | grep --color=always "ERROR"

# 查看占用空间最大的 10 个文件/目录
du -sh * | sort -hr | head -10

# 批量重命名 (将所有 .txt 改为 .md)
for f in *.txt; do mv "$f" "${f%.txt}.md"; done
# 或使用 rename (brew install rename)
rename 's/\.txt$/.md/' *.txt

```

13.8 管道、重定向与环境变量

Shell 的管道和重定向是组合命令、串联处理流程的核心机制。理解这些概念是从「会用命令」到「高效使用命令行」的分水岭。

标准输入/输出/错误 (stdin/stdout/stderr) :

描述符	符号	说明
0	stdin	标准输入，命令读取数据的地方
1	stdout	标准输出，命令正常输出的地方
2	stderr	标准错误，命令输出错误信息的地方

```

# --- 重定向 ---

```



```
# 覆盖写入文件 (>)
echo "hello" > file.txt          # stdout 写入文件
ls /nonexistent 2> error.log    # stderr 写入文件

# 追加写入文件 (>>)
echo "another line" >> file.txt  # stdout 追加到文件末尾
ls /nonexistent 2>> error.log    # stderr 追加

# 同时重定向 stdout 和 stderr 到同一个文件
command > output.log 2>&1        # 覆盖模式
command >> output.log 2>&1       # 追加模式
command &> output.log           # bash/zsh 简写 (覆盖)
command &>> output.log           # bash/zsh 简写 (追加)

# 丢弃输出 (>/dev/null)
command > /dev/null 2>&1         # 同时丢弃 stdout 和 stderr

# --- 管道 (Pipe) ---

# 管道将前一个命令的 stdout 传给后一个命令的 stdin
cat file.txt | grep "pattern" | sort | uniq

# 同时将 stdout 传给管道和文件
command | tee output.log        # 输出到终端 + 写入文件
command | tee -a output.log     # 追加模式

# 同时捕获 stdout 和 stderr 通过管道
command 2>&1 | grep "ERROR"

# --- Here Document 与 Here String ---

# Here Document: 将多行文本作为命令的输入
cat << EOF > config.txt
line1
line2
EOF

# Here String: 将字符串作为命令的输入
grep "pattern" <<< "this is a test string"

# --- 命令替换 ---

# $( ) 将命令的输出作为另一个命令的参数
echo "Current date: $(date)"
cd $(dirname $(which python))

# 旧式反引号语法 (不推荐, 难以嵌套)
echo "Current date: `date`"

# --- 子 Shell ---

# ( ) 在子 Shell 中执行 (不影响当前 Shell 的状态)
(cd /tmp && ls)                # 在子 Shell 中切换目录
echo $PWD                      # 当前目录不变
```

```
# --- 环境变量 ---

# 查看所有环境变量
env
printenv

# 查看特定变量
echo $HOME
echo $PATH
echo $SHELL

# 临时设置 (仅当前命令生效)
VAR=value command
DEBUG=true python app.py

# 当前会话设置 (仅在当前终端窗口有效)
export VAR=value
export NODE_ENV=development

# 永久设置 (写入 Shell 配置文件)
# 在 ~/.zshrc 中添加:
# export JAVA_HOME=/opt/homebrew/opt/openjdk
# export PATH="$JAVA_HOME/bin:$PATH"

# 添加到 PATH (临时)
export PATH="/custom/bin:$PATH"
# 添加到 PATH (永久, 在 ~/.zshrc 中)

# 查看 PATH 的所有目录
echo $PATH | tr ':' '\n'

# 常用预定义变量
echo $HOME          # 用户主目录
echo $USER          # 当前用户名
echo $PWD           # 当前工作目录
echo $OLDPWD        # 上一个工作目录
echo $SHELL         # 当前 Shell
echo $?            # 上一条命令的退出状态码 (0=成功, 非 0=失败)
echo $$            # 当前 Shell 的 PID
echo $0            # 当前 Shell 的名称

# --- 退出状态码 ---
# 0    = 成功
# 1    = 一般错误
# 2    = 误用 Shell 命令
# 126  = 命令不可执行
# 127  = 命令未找到
# 128+N = 进程被信号 N 终止 (128+9=137 表示被 kill -9 终止)

# 检查退出状态码
command && echo "Success" || echo "Failed"
if command; then echo "OK"; else echo "FAIL"; fi
```

13.9 SQLite 命令行速查

SQLite 是 macOS 自带的轻量级关系型数据库（sqlite3 命令预装），无需安装服务器、无需配置，数据存储在一个单文件中。适合本地开发、原型验证、数据分析和嵌入式场景。

```
# 打开/创建数据库文件
sqlite3 database.db

# 在命令行直接执行 SQL 而不进入交互模式
sqlite3 database.db "SELECT * FROM users;"

# 从文件执行 SQL 脚本
sqlite3 database.db < schema.sql
sqlite3 database.db ".read schema.sql"    # 交互模式下也可用

# 将查询结果导出为 CSV
sqlite3 -csv database.db "SELECT * FROM users;" > users.csv
sqlite3 -header -csv database.db "SELECT * FROM users;" > users.csv
```

sqlite3 交互模式命令（以 . 开头的元命令）：

.help	显示所有元命令帮助
.tables	列出所有表
.schema	显示所有表的 CREATE 语句
.schema users	显示指定表的 CREATE 语句
.mode csv	设置输出为 CSV 格式
.mode column	设置输出为列对齐格式（最常用）
.mode json	设置输出为 JSON 格式
.mode line	每列一行显示
.mode table	表格格式（ASCII 边框）
.headers on	显示列名
.headers off	隐藏列名
.output file.txt	将输出重定向到文件
.output stdout	恢复输出到终端
.import file.csv table_name	从 CSV 导入数据
.dump	导出整个数据库为 SQL 文本
.dump users	导出指定表为 SQL 文本
.read file.sql	执行 SQL 文件
.quit	退出
.exit	退出

在交互模式中常用 SQL：

```
-- 创建表
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT UNIQUE,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 插入数据
INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com');
```

```

-- 查询
SELECT * FROM users;
SELECT id, name FROM users WHERE email LIKE '%@example.com';
SELECT COUNT(*) FROM users;

-- 更新
UPDATE users SET name = 'Alice Updated' WHERE id = 1;

-- 删除
DELETE FROM users WHERE id = 1;

-- 创建索引
CREATE INDEX idx_users_email ON users(email);

-- 查看表结构
PRAGMA table_info(users);

```

一键将 SQLite 查询结果导入 Python/Pandas:

```

# 导出 CSV 后直接用 pandas 读取
sqlite3 -header -csv database.db "SELECT * FROM users;" > users.csv
python -c "import pandas; print(pandas.read_csv('users.csv'))"

```

13.10 命令行基础命令速查

以下命令是开发者每天都会用到的基础操作，涵盖文件管理、权限、压缩、查找、帮助等核心场景。

文件与目录操作

```

# ---- 列出文件 (ls) ----
ls                # 列出当前目录文件（不包含隐藏文件）
ls -l             # 长格式：权限、大小、修改时间
ls -la           # 包括隐藏文件（. 开头）
ls -lh           # 人类可读的文件大小（K/M/G）
ls -lt           # 按修改时间排序（最新在前）
ls -ltr          # 按修改时间排序（最旧在前）
ls -R            # 递归列出子目录
ls -l            # 每行一个文件（适合管道处理）
ls -d */         # 只列出目录

# ---- 切换目录 (cd) ----
cd /path/to/dir  # 切换到指定目录
cd ..            # 上级目录
cd ../..         # 上两级目录
cd -             # 切换到上一个所在目录

```

```

cd ~                # 用户主目录
cd                  # 同上，回家目录

# --- 目录栈 (pushd / popd / dirs) ---
pushd /some/dir     # 压入当前目录并切换到新目录
popd                # 弹出栈顶目录并切换过去
dirs                # 查看目录栈
dirs -v             # 带编号显示目录栈

# --- 复制 (cp) ---
cp source dest      # 复制文件
cp -r src/ dest/    # 递归复制目录
cp -p source dest   # 保留权限、时间戳等属性
cp -a src/ dest/    # 归档复制 (等同于 cp -rpP)
cp -i source dest   # 覆盖前询问确认
cp -n source dest   # 不覆盖已存在的文件
cp -v source dest   # 显示复制过程

# --- 移动/重命名 (mv) ---
mv old new          # 重命名文件或目录
mv file /target/    # 移动文件到目标目录
mv -i source dest   # 覆盖前询问确认
mv -n source dest   # 不覆盖已存在的文件

# --- 删除 (rm) ---
rm file             # 删除文件 (不可恢复!)
rm -i file          # 删除前逐个确认
rm -f file          # 强制删除，不提示
rm -r directory     # 递归删除目录及其中所有内容
rm -rf directory    # 强制递归删除 (极其危险，慎用!)

# --- 创建目录 (mkdir) ---
mkdir dirname       # 创建目录
mkdir -p a/b/c      # 递归创建目录树 (父目录不存在时自动创建)
mkdir -m 755 dir     # 创建并指定权限

# --- 创建文件/更新时间戳 (touch) ---
touch file.txt      # 创建空文件，或更新已有文件的时间戳为当前时间

# --- 符号链接 (ln) ---
ln -s target link   # 创建软链接 (符号链接)，类似 macOS 的"替身"
ln target link       # 创建硬链接 (同一文件的不同入口，不能跨文件系统)
readlink link        # 查看软链接指向的目标路径

```

文件查看与搜索

```

# --- 查看文件内容 ---
cat file            # 一次性输出全部内容
cat -n file         # 带行号输出
less file           # 分页浏览 (大文件友好)
# less 快捷键: Space 下一页 | b 上一页 | / 搜索 | n 下一个 | q 退出 | g 开头 | G 结尾
head -20 file       # 显示前 20 行 (默认 10 行)
tail -20 file       # 显示最后 20 行 (默认 10 行)
tail -f file        # 实时跟踪文件新增内容

```

```

tail -F file          # 同 -f 但文件被删除重建后自动重新跟踪

# --- 文件类型识别 ---
file unknown.bin      # 判断文件类型 (文本/二进制/图片/压缩包等)
file -I file.txt       # 显示 MIME 类型

# --- 查找文件位置 ---
which python           # 在 PATH 中查找可执行文件的位置
which -a python         # 显示所有匹配的可执行文件路径
whereis python          # 查找二进制、源码和 man 手册位置
type python            # 判断命令是内置/别名/外部命令
command -v python       # POSIX 兼容的 which 替代

# --- 查找文件 (find) ---
find . -name "*.py"      # 按文件名查找
find . -iname "*.py"     # 不区分大小写
find . -type f -name "*.log" # 只查找文件
find . -type d -name "test*" # 只查找目录
find . -type f -mtime -7  # 7 天内修改过的文件
find . -type f -size +10M # 大于 10MB 的文件
find . -type f -empty     # 空文件
find . -name "*.tmp" -delete # 查找并删除
find . -name "*.py" -exec wc -l {} + # 对找到的文件执行命令

# --- 定位文件 (mdfind - macOS Spotlight 命令行) ---
mdfind "kind:python"      # 查找所有 Python 文件
mdfind -name "README.md"  # 按文件名搜索
mdfind -onlyin ~/Documents "关键词" # 限定目录搜索

```

权限管理

```

# --- 查看权限 ---
ls -l file          # 显示文件权限 (r=读 w=写 x=执行)
stat file           # 显示文件详细信息

# --- 修改权限 (chmod) ---
# 符号模式:
chmod +x script.sh  # 给所有用户添加执行权限
chmod u+x file       # 仅所有者添加执行权限
chmod g+w file       # 组添加写权限
chmod o-r file       # 其他人移除读权限
chmod a+r file       # 所有人添加读权限 (a=all)

# 数字模式 (推荐, 精确且简洁):
# 4=读(r) 2=写(w) 1=执行(x), 相加组合
chmod 755 file       # rwxr-xr-x (所有者全权限, 组和其他人读+执行)
chmod 644 file       # rw-r--r-- (常用文件权限)
chmod 600 file       # rw----- (私密文件, 如 SSH 密钥)
chmod 700 dir        # rwx----- (私密目录)
chmod 777 file       # rwxrwxrwx (所有人全权限, 安全风险!)
chmod -R 755 dir/    # 递归修改目录

# --- 修改所有者 (chown) ---
chown user file      # 修改文件所有者

```

```
chown user:group file      # 同时修改所有者和组
chown -R user:group dir/   # 递归修改目录
```

压缩与归档

```
# --- tar (归档 + 压缩) ---
tar -czvf archive.tar.gz dir/      # 创建 tar.gz (最常用)
tar -cjvf archive.tar.bz2 dir/     # 创建 tar.bz2 (更高压缩率)
tar -cJvf archive.tar.xz dir/      # 创建 tar.xz (压缩率最高)
tar -xzvf archive.tar.gz           # 解压到当前目录
tar -xzvf archive.tar.gz -C /target/dir/ # 解压到指定目录
tar -tzvf archive.tar.gz           # 查看内容不提取

# --- gzip / gunzip (单文件压缩) ---
gzip file                          # 压缩为 file.gz (原文件被替换)
gunzip file.gz                     # 解压
gzip -k file                        # 压缩但保留原文件
gzip -9 file                       # 最高压缩率 (1-9, 默认 6)

# --- zip / unzip (跨平台兼容) ---
zip -r archive.zip dir/            # 递归压缩目录
zip -e archive.zip file             # 加密压缩 (需要输入密码)
unzip archive.zip                  # 解压
unzip -l archive.zip               # 查看内容不提取
unzip archive.zip -d /target/dir/  # 解压到指定目录
```

文本处理

```
# --- 比较文件 (diff) ---
diff file1 file2                  # 逐行比较两个文件
diff -u file1 file2               # 统一格式 (类似 git diff)
diff -r dir1/ dir2/               # 递归比较目录
diff -q file1 file2               # 仅报告是否不同

# --- 字符转换 (tr) ---
echo "HELLO" | tr 'A-Z' 'a-z'     # 大写转小写
cat file | tr -d '\r'              # 删除 Windows 换行符
echo "a b c" | tr -s ' '          # 压缩连续空格

# --- 分割与合并 (split / paste) ---
split -l 1000 large.txt chunk_     # 按行数分割文件
split -b 50m large.zip chunk_      # 按文件大小分割
cat chunk_* > reassembled.zip      # 合并分割的文件
paste file1.txt file2.txt          # 按列合并两个文件

# --- 路径操作 ---
basename /path/to/file.txt         # 提取文件名
basename /path/to/file.txt .txt    # 提取文件名并去掉后缀
dirname /path/to/file.txt          # 提取目录路径
realpath file.txt                  # 解析为绝对路径

# --- 校验和 ---
md5 file                           # MD5 校验和
```

```
shasum -a 256 file          # SHA-256 校验和
openssl dgst -sha256 file    # OpenSSL SHA-256
```

系统与帮助

```
# --- 帮助系统 ---
man ls          # 查看命令手册 (q 退出, / 搜索)
man -k keyword   # 搜索手册页关键词
whatis ls        # 显示命令的简要说明
tldr ls          # 命令的常用示例 (需 brew install tldr)
command --help   # 大多数命令支持此选项

# --- 时间与日期 ---
date            # 显示当前日期和时间
date "+%Y-%m-%d %H:%M:%S" # 自定义格式
cal             # 显示当月日历

# --- 计时 (time) ---
time command    # 测量命令执行时间
# real = 墙钟时间, user = CPU用户态时间, sys = CPU内核态时间

# --- 延迟与等待 ---
sleep 5         # 等待 5 秒
sleep 5m        # 等待 5 分钟
wait            # 等待所有后台任务完成

# --- 别名 ---
alias           # 列出所有别名
alias ll="ls -la" # 创建临时别名
unalias ll      # 删除别名
# 永久别名应写入 ~/.zshrc

# --- 环境变量 ---
printenv        # 列出所有环境变量
echo $HOME      # 查看特定变量
export VAR=value # 设置环境变量
unset VAR       # 删除变量
env VAR=value command # 临时设置变量并执行命令

# --- 磁盘与内存 ---
df -h           # 磁盘使用概览
du -sh * | sort -hr # 各子目录大小排序
vm_stat         # macOS 虚拟内存统计
```

13.11 现代化 CLI 工具集

以下工具是用 Rust/Go 等现代语言重写的经典命令行工具替代品，速度快、用户体验更好。在 2020 年代，这些工具已经成为许多开发者的默认选择。

bat——语法高亮的 cat:


```

# 安装
brew install bat

# 基本使用（自动语法高亮、行号、Git 修改标记）
bat file.py
bat README.md
bat config.json

# 纯文本输出（管道友好，去掉行号和边框）
bat --style=plain file.txt
bat -p file.txt

# 显示指定行范围
bat --line-range :100 file.txt
bat -r 50:100 file.txt

# 显示所有支持的语言
bat --list-languages

# 自定义主题
bat --theme="TwoDark" file.py

# 管道中使用（bat 会自动检测并适应管道）
curl -s https://api.example.com/data | bat

# 作为 man 命令的 pager（在 ~/.zshrc 中设置）
export MANPAGER="sh -c 'col -bx | bat -l man -p'"

```

fd——现代化的 find 深入使用：

```

# 安装
brew install fd

# 按文件名模式搜索
fd '\.py$'          # 正则表达式搜索 Python 文件
fd pattern          # 简单的子串匹配

# 在指定目录搜索
fd pattern /path/to/dir

# 仅搜索文件或目录
fd --type file pattern
fd --type directory pattern
fd -t f pattern      # 简写

# 搜索并执行命令
fd '\.py$' --exec wc -l {} # 对找到的每个文件执行 wc -l

# 排除特定目录
fd --exclude node_modules --exclude .git pattern

# 按扩展名搜索
fd --extension py
fd -e py -e js        # 多个扩展名

```

```
# 大小写敏感搜索
fd --case-sensitive pattern
fd -s pattern

# 搜索隐藏文件和忽略文件（包括 .gitignore 中的）
fd --hidden --no-ignore pattern
fd -H -I pattern

# 限制搜索深度
fd --max-depth 2 pattern
```

zoxide——智能目录跳转（cd 替代品）：

```
# 安装
brew install zoxide

# 在 ~/.zshrc 中添加（Oh My Zsh 用户直接在 plugins 中添加 zoxide）
eval "$(zoxide init zsh)"

# 跳转到最匹配的目录（基于访问频率）
z project          # 跳转到包含 "project" 的目录

# 交互式选择（fzf 风格）
zi                # 打开交互式目录选择器

# 查看数据库中匹配的目录
z query           # 搜索但不跳转

# 删除当前目录的记录
z -               # 从数据库中移除

# zoxide 与 fzf 集成获得最佳体验
# zoxide 会自动记住你访问过的目录，并根据频率和最近访问时间排序
```

fzf——模糊搜索深入使用：

```
# 安装
brew install fzf

# 安装 shell 集成（启用 Ctrl+T、Ctrl+R、** 触发器）
$(brew --prefix)/opt/fzf/install

# 基础用法
fzf              # 在当前目录下交互式搜索文件
ls | fzf         # 从管道中搜索
find . -type f | fzf  # 搜索所有文件

# Ctrl+T - 在任何命令行中搜索文件并粘贴路径
# 输入 `code` 然后按 Ctrl+T，选择文件后直接打开

# Ctrl+R - 模糊搜索命令历史
# 比传统的 Ctrl+R 更直观
```

```

# ** 触发器 - 在命令中触发 fzf
# 输入 `vim **` 然后按 Tab, 自动打开 fzf 选择文件

# 预览窗口
fzf --preview 'bat --style=numbers --color=always {}'
# 在 ~/.zshrc 中设置默认预览
export FZF_DEFAULT_OPTS='--preview "bat --style=numbers --color=always {}"'

# 快捷键绑定
export FZF_DEFAULT_OPTS="
  --bind 'ctrl-/:toggle-preview'
  --bind 'ctrl-y:execute-silent(echo {} | pbcopy)'
"

# 搜索 Git 文件
git ls-files | fzf          # 仅搜索 Git 追踪的文件

# 搜索并直接打开 VS Code
code $(fzf)

```

direnv——目录级环境变量自动加载:

```

# 安装
brew install direnv

# 在 ~/.zshrc 末尾添加 (必须在最后一行)
eval "$(direnv hook zsh)"

# 在项目目录创建 .envrc 文件
echo 'export DATABASE_URL=postgresql://localhost/mydb' > .envrc
echo 'export API_KEY=sk-xxxxx' >> .envrc

# 允许 .envrc 生效 (安全机制, 每次修改后需要重新允许)
direnv allow

# 进入目录时自动加载环境变量, 离开目录时自动卸载
# 非常适合: 每个项目的 API 密钥、数据库连接、不同 Node/Python 版本

```

starship——跨 Shell 美化提示符:

```

# 安装
brew install starship

# 在 ~/.zshrc 末尾添加
eval "$(starship init zsh)"

# 配置文件 ~/.config/starship.toml
mkdir -p ~/.config && touch ~/.config/starship.toml

# 示例配置
cat > ~/.config/starship.toml << 'EOF'
# 不显示前导换行
add_newline = false

```

```

[character]
success_symbol = "[→](bold green)"
error_symbol = "[→](bold red)"

[directory]
truncation_length = 3
truncate_to_repo = true

[git_branch]
symbol = "🌱 "

[python]
symbol = "🐍 "

[nodejs]
symbol = "🟡 "

[docker_context]
symbol = "🐳 "
EOF

```

lsd——美化版 ls:

```

# 安装
brew install lsd

# 别名 (替换 ls)
alias ls="lsd"
alias ll="lsd -la"
alias la="lsd -A"
alias lt="lsd --tree"

# 常用选项
lsd --tree           # 树状显示
lsd --tree --depth 2 # 限制树深度
lsd --icon never     # 不显示图标 (适合管道)
lsd --date relative  # 相对日期显示

```

bottom (btm)——图形化系统监控:

```

# 安装
brew install bottom

# 运行
btm           # 图形化 CPU、内存、网络、磁盘、进程监控

# 基本操作 (在 btm 界面中)
# q - 退出
# h - 帮助
# ↑/↓ - 选择进程
# F1-F5 - 切换视图标签
# / - 搜索进程
# Esc - 取消

```

其他好用工具速查：

```
# duf - 美化版 df (磁盘使用概览)
brew install duf
duf                                # 彩色磁盘使用展示

# gdu - 美化版 du (磁盘使用分析, 类似 ncdu 但更快)
brew install gdu
gdu /path/to/dir                  # 交互式磁盘使用分析

# tokei - 代码统计 (比 cloc 更快, 支持更多语言)
brew install tokei
tokei                             # 统计当前目录代码行数
tokei --sort lines                # 按行数排序

# hyperfine - 命令行基准测试
brew install hyperfine
hyperfine 'sleep 0.5' 'sleep 0.3' # 比较命令执行时间
hyperfine --warmup 5 'python script.py'

# watchexec - 文件变更时自动执行命令
brew install watchexec
watchexec -e py,js -- pytest      # .py/.js 文件变化时自动运行 pytest
watchexec -w src/ -- make build   # src/ 目录变化时自动构建
watchexec --restart "python app.py" # 文件变化时重启服务 (适合开发)
```

13.12 Nginx 本地开发环境

Nginx 是一个高性能的 HTTP 服务器和反向代理服务器。在本地开发中，它可以用来托管静态文件、做反向代理转发 API 请求、测试负载均衡等。

```
# 安装
brew install nginx

# 启动/停止/重启
brew services start nginx      # 启动并设置开机自启
brew services stop nginx       # 停止
brew services restart nginx    # 重启
nginx                           # 前台运行 (测试用)
nginx -s reload                 # 优雅重启 (重新加载配置)

# 配置文件位置 (Homebrew 安装)
# /opt/homebrew/etc/nginx/nginx.conf    - 主配置文件
# /opt/homebrew/etc/nginx/servers/      - 虚拟主机配置目录
# /opt/homebrew/var/www/                - 默认 Web 根目录

# 测试配置文件语法
nginx -t

# 查看 Nginx 版本和编译参数
nginx -V
```

常用本地开发配置示例：

```
# 示例 1: 静态文件服务器
# 在 /opt/homebrew/etc/nginx/servers/static.conf
server {
    listen 8080;
    server_name localhost;
    root /Users/yourname/projects/myapp/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;    # SPA 路由支持
    }

    # 启用 gzip 压缩
    gzip on;
    gzip_types text/plain text/css application/json application/javascript;
}

# 示例 2: 反向代理到本地 API 服务
server {
    listen 8080;
    server_name localhost;

    # 前端静态文件
    location / {
        root /Users/yourname/projects/myapp/frontend/dist;
        try_files $uri /index.html;
    }

    # API 请求转发到后端
    location /api/ {
        proxy_pass http://127.0.0.1:3000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # WebSocket 代理
    location /ws/ {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

# 示例 3: HTTPS 本地开发 (自签证书)
# 生成自签证书
# openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
# -keyout localhost.key -out localhost.crt \
# -subj "/CN=localhost"
```

```
server {
    listen 443 ssl;
    server_name localhost;
    ssl_certificate /path/to/localhost.crt;
    ssl_certificate_key /path/to/localhost.key;

    location / {
        proxy_pass http://127.0.0.1:3000;
    }
}
```

13.13 CI/CD 本地测试 (act)

act 允许你在本地运行 GitHub Actions workflow，无需将代码推送到 GitHub 就能测试 CI 配置。

```
# 安装
brew install act

# 前置条件：需要安装 Docker Desktop 或 Colima
# 首次运行需要选择镜像大小（推荐 Medium）

# 在 Git 仓库中本地运行 push 触发的 workflow
act

# 运行指定事件触发的 workflow
act pull_request
act push

# 运行指定 job
act -j <job名>
act -j test

# 运行指定文件中的 workflow
act -W .github/workflows/ci.yml

# 模拟特定事件的数据
act pull_request -e pull_request.json

# 列出所有可用的 job
act -l

# 指定 Docker 镜像（解决某些 action 的兼容性问题）
act --container-architecture linux/amd64

# 传入 Secrets
act -s SECRET_NAME=value
act --secret-file .env

# 传入环境变量
act --env CI=true
```

常用限制

- 不支持某些 GitHub 特有的功能 (如 GitHub Packages Registry 的认证)

- 大型 workflow 可能运行较慢

- 某些 Actions (如 deploy 类) 可能需要 GitHub 上下文, 在本地无法完全模拟

十四、数据库客户端与工具

14.1 PostgreSQL (psql)

PostgreSQL 是开源界最强大的关系型数据库，psql 是其命令行客户端。

```
# 安装
brew install postgresql@16

# 连接到本地数据库
psql -U <用户名> -d <数据库名>
psql postgres # 连接默认数据库

# 连接到远程数据库
psql -h <主机> -p <端口> -U <用户名> -d <数据库名>
psql "postgresql://user:pass@host:5432/dbname"

# 从文件执行 SQL
psql -d mydb -f schema.sql

# 导出数据库 (pg_dump)
pg_dump -U user -d mydb > backup.sql
pg_dump -U user -d mydb --schema-only > schema.sql # 仅导出结构
pg_dump -U user -d mydb --data-only > data.sql # 仅导出数据
pg_dump -U user -d mydb -t users > users_backup.sql # 仅导出指定表

# 导入数据库 (pg_restore 用于 .dump 格式, psql 用于 .sql 格式)
psql -U user -d mydb < backup.sql
pg_restore -U user -d mydb backup.dump
```

psql 元命令速查表（在 psql 交互模式中使用）：

命令	功能
\l	列出所有数据库
\c <dbname>	切换数据库
\dt	列出当前数据库的所有表
\d <表名>	查看表结构（列、类型、约束）
\di	列出所有索引
\dv	列出所有视图
\du	列出所有用户/角色
\dn	列出所有 Schema
\df	列出所有函数
\x	切换扩展显示模式（宽表友好）
\e	在外部编辑器中编辑上次查询
\i <文件>	执行 SQL 文件
\o <文件>	将输出重定向到文件
\timing	切换查询计时显示
\copy	导入/导出 CSV
\?	查看所有元命令帮助
\q	退出

```
# 在 psql 中导出/导入 CSV
\copy (SELECT * FROM users) TO '/tmp/users.csv' WITH CSV HEADER;
\copy users FROM '/tmp/users.csv' WITH CSV HEADER;

# 查询性能分析
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'alice@example.com';
```

14.2 MySQL

```
# 安装客户端
brew install mysql-client

# 连接
mysql -h <主机> -P <端口> -u <用户名> -p
```

```
mysql -u root -p
```

常用命令

SHOW DATABASES;	# 列出数据库
USE <dbname>;	# 切换数据库
SHOW TABLES;	# 列出表
DESCRIBE <表名>;	# 查看表结构
SHOW CREATE TABLE <表名>;	# 查看建表语句
SHOW INDEX FROM <表名>;	# 查看索引

导出/导入

```
mysqldump -u root -p <dbname> > backup.sql
mysql -u root -p <dbname> < backup.sql
mysqldump -u root -p <dbname> --no-data > schema.sql # 仅结构
```

14.3 Redis (redis-cli)

安装

```
brew install redis
```

启动 Redis 服务

```
brew services start redis
```

连接

```
redis-cli
redis-cli -h <主机> -p <端口> -a <密码>
```

--- 五种数据类型基本操作 ---

String (字符串)

SET key value	# 设置
GET key	# 获取
SETEX key 60 value	# 设置 60 秒过期
INCR counter	# 自增
MSET k1 v1 k2 v2	# 批量设置

Hash (哈希表)

HSET user:1 name "Alice" email "a@example.com"	
HGET user:1 name	# 获取单个字段
HGETALL user:1	# 获取所有字段
HDEL user:1 email	# 删除字段
HKEYS user:1	# 所有字段名

List (列表)

LPUSH queue "task1"	# 左侧推入
RPUSH queue "task2"	# 右侧推入
LPOP queue	# 左侧弹出
LRANGE queue 0 -1	# 查看全部元素
LLEN queue	# 列表长度

Set (集合)

```

SADD tags "python" "redis"          # 添加
SMEMBERS tags                        # 所有成员
SISMEMBER tags "python"              # 判断是否存在
SINTER set1 set2                     # 交集
SUNION set1 set2                     # 并集

# Sorted Set (有序集合)
ZADD scores 100 "Alice" 85 "Bob"    # 添加
ZRANGE scores 0 -1                  # 按分数升序
ZREVRANGE scores 0 -1 WITHSCORES    # 按分数降序 (带分数)
ZRANK scores "Alice"                 # 排名

# --- 监控与调试 ---
INFO                                # 服务器信息与统计
INFO memory                         # 内存使用详情
MONITOR                             # 实时监控所有命令 (生产慎用)
SLOWLOG GET 10                      # 最近 10 条慢查询
DBSIZE                              # 当前数据库 key 数量
KEYS pattern                        # 匹配 key (生产慎用, 用 SCAN 代替)
SCAN 0 MATCH user:* COUNT 100      # 安全遍历 key (游标迭代)
FLUSHDB                             # 清空当前数据库 (危险!)
FLUSHALL                            # 清空所有数据库 (非常危险!)

# 管道模式 (批量操作)
echo "SET k1 v1\nSET k2 v2" | redis-cli --pipe

# 监控延迟
redis-cli --latency
redis-cli --latency-history

```

14.4 MongoDB (mongosh)

```

# 安装
brew install mongosh

# 连接
mongosh                                # 连接本地默认端口
mongosh "mongodb://localhost:27017/mydb"
mongosh --host <主机> --port <端口> -u <用户> -p <密码>

# --- 基本 CRUD ---

# 切换/创建数据库
use mydb

# 插入
db.users.insertOne({ name: "Alice", email: "alice@example.com", age: 30 })
db.users.insertMany([
  { name: "Bob", email: "bob@example.com", age: 25 },
  { name: "Charlie", email: "charlie@example.com", age: 35 }
])

```

```

# 查询
db.users.find() # 查询所有
db.users.find({ age: { $gt: 25 } }) # age > 25
db.users.find({ name: /ali/i }) # 正则匹配
db.users.findOne({ email: "alice@example.com" }) # 查询一条
db.users.find().sort({ age: -1 }).limit(5) # 排序+限制
db.users.find({}, { name: 1, email: 1 }) # 投影（只返回特定字段）
db.users.countDocuments({ age: { $gt: 25 } }) # 计数

# 更新
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 31 } }
)
db.users.updateMany(
  { age: { $lt: 30 } },
  { $inc: { age: 1 } } # 自增
)

# 删除
db.users.deleteOne({ name: "Charlie" })
db.users.deleteMany({ age: { $lt: 25 } })

# --- 聚合管道示例 ---
db.users.aggregate([
  { $match: { age: { $gte: 25 } } },
  { $group: { _id: null, avgAge: { $avg: "$age" } } }
])

# --- 索引 ---
db.users.createIndex({ email: 1 }, { unique: true })
db.users.getIndexes()
db.users.dropIndex("email_1")

```

14.5 SQLite 图形化工具

除命令行 `sqlite3`（见 13.9）外，以下工具让 SQLite 操作更直观：

- **DB Browser for SQLite**: `brew install --cask db-browser-for-sqlite`，图形化查看和编辑 SQLite 数据库
- **VS Code SQLite 扩展**：搜索 SQLite 扩展，直接在 VS Code 中浏览表、执行查询

十五、深度学习开发环境

15.1 开发范式：本地编码 → 云端训练

深度学习开发的典型 workflow 是「本地开发 + 云端 GPU 训练」。以下说明基于这个范式：

- **macOS 本地**：代码编写、小规模调试、模型架构验证（使用 MPS 或 CPU）
- **Linux 云端 GPU**：全量训练、大规模实验、模型部署（NVIDIA GPU + CUDA）

15.2 PyTorch 安装

本地 macOS（MPS 加速 — 仅用于调试和小规模实验）：

```
# pip 安装 (带 MPS 支持)
pip install torch torchvision torchaudio

# Conda 安装
conda install pytorch torchvision torchaudio -c pytorch

# 验证 MPS 是否可用
python -c "import torch; print(torch.backends.mps.is_available())"
# 输出 True 表示 MPS 可用

# 在代码中使用 MPS
device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
model.to(device)
```

Linux 云端（CUDA GPU — 主力训练环境）：

```
# 安装前先确认 CUDA 版本
nvidia-smi                                # 查看 CUDA 版本

# pip 安装 (指定 CUDA 版本, 例如 CUDA 12.1)
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121

# Conda 安装 (指定 CUDA 版本)
conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia

# 验证 CUDA 是否可用
python -c "import torch; print(torch.cuda.is_available());
print(torch.cuda.get_device_name(0))"
```

PyTorch 与 CUDA 兼容性速查：

PyTorch 版本	支持的 CUDA 版本
2.5.x	CUDA 11.8 / 12.1 / 12.4
2.4.x	CUDA 11.8 / 12.1 / 12.4
2.3.x	CUDA 11.8 / 12.1
2.2.x	CUDA 11.8 / 12.1

始终从 pytorch.org 获取最新安装命令。

15.3 TensorFlow / Keras

```
# macOS (Apple Silicon – 使用 tensorflow-metal 插件)
pip install tensorflow-macos tensorflow-metal

# Linux (CUDA GPU)
pip install tensorflow[and-cuda]      # TensorFlow 2.15+

# 验证 GPU
python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
```

15.4 JAX

```
# CPU 版本
pip install jax jaxlib

# Linux CUDA GPU
pip install jax[cuda12] jaxlib

# macOS Metal (实验性)
pip install jax-metal
```

15.5 Jupyter Notebook/Lab 完整配置

```
# 安装
brew install jupyterlab
# 或
pip install jupyterlab jupyter

# 启动 (本地)
```

```
jupyter lab                # JupyterLab (推荐, 现代界面)
jupyter notebook           # 经典 Notebook

# 指定端口和 IP
jupyter lab --port 8888 --ip 0.0.0.0

# 生成配置文件
jupyter lab --generate-config
# 配置文件位置: ~/.jupyter/jupyter_lab_config.py

# 设置密码 (用于远程访问)
jupyter lab password

# 无浏览器模式启动 (远程服务器上使用)
jupyter lab --no-browser --port=8888
```

远程服务器 Jupyter 访问 (SSH 端口转发) :

```
# 在本地终端执行 (将远程 8888 端口映射到本地 8888)
ssh -L 8888:localhost:8888 user@remote-server

# 然后在远程服务器启动 Jupyter
jupyter lab --no-browser --port=8888

# 在本地浏览器打开 http://localhost:8888
```

VS Code Jupyter 集成:

安装 VS Code 扩展 `ms-toolsai.jupyter`, 直接在 VS Code 中打开 `.ipynb` 文件或使用 `# %%` 魔法注释在 `.py` 文件中运行单元格。

常用 IPython/Jupyter 魔法命令:

命令	功能
<code>%timeit func()</code>	测量代码执行时间（多次运行取平均）
<code>%time func()</code>	测量单次执行时间
<code>%%timeit</code>	测量整个 Cell 的执行时间
<code>%matplotlib inline</code>	在 Notebook 内嵌显示图表
<code>%%writefile script.py</code>	将 Cell 内容写入文件
<code>%load script.py</code>	加载文件内容到 Cell
<code>%run script.py</code>	运行 Python 脚本
<code>%env VAR=value</code>	设置环境变量
<code>!command</code>	执行 Shell 命令
<code>%debug</code>	进入调试模式
<code>%pdb</code>	自动进入调试模式（出错时）
<code>%lsmagic</code>	列出所有魔法命令

15.6 实验追踪

TensorBoard:

```
# 安装
pip install tensorboard

# 在 PyTorch 中使用 (torch.utils.tensorboard)
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter('runs/experiment_1')
writer.add_scalar('Loss/train', loss, epoch)
writer.add_scalar('Accuracy/val', acc, epoch)
writer.close()

# 启动 TensorBoard
tensorboard --logdir=runs
# 浏览器打开 http://localhost:6006

# 远程服务器 TensorBoard (SSH 端口转发)
ssh -L 6006:localhost:6006 user@remote-server
# 远程端启动: tensorboard --logdir=runs --bind_all
```

Weights & Biases (wandb):

```
# 安装
pip install wandb

# 登录
wandb login

# 在代码中集成 (最小示例)
import wandb
wandb.init(project="my-project", name="experiment-1", config={"lr": 0.001, "epochs": 10})
wandb.log({"loss": 0.5, "accuracy": 0.85})
wandb.finish()
```

MLflow:

```
# 安装
pip install mlflow

# 启动 Tracking Server (本地)
mlflow ui --port 5000
# 浏览器打开 http://localhost:5000

# 在代码中记录
import mlflow
mlflow.set_experiment("my-experiment")
with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.001)
    mlflow.log_metric("accuracy", 0.85)
    mlflow.pytorch.log_model(model, "model")
```

15.7 Hugging Face 生态系统

```
# 安装核心库
pip install transformers datasets accelerate peft

# 模型下载 (自动缓存到 ~/.cache/huggingface/hub/)
from transformers import AutoModel, AutoTokenizer
model = AutoModel.from_pretrained("bert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# 国内镜像加速 (设置环境变量)
export HF_ENDPOINT=https://hf-mirror.com
# 或在 Python 中
import os
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"

# 离线模式 (使用已缓存的模型, 不尝试下载)
export HF_HUB_OFFLINE=1

# 管理缓存
```

```
huggingface-cli scan-cache          # 查看缓存
huggingface-cli delete-cache        # 交互式清理缓存
```

Hugging Face 核心库简介:

库	用途
transformers	预训练模型加载、微调、推理 (BERT, GPT, LLaMA 等)
datasets	高效数据集加载和处理, 支持流式读取大文件
accelerate	简化多 GPU/TPU/混合精度训练
peft	参数高效微调 (LoRA, QLoRA, Adapter 等)
diffusers	扩散模型 (Stable Diffusion 等图像生成)
trl	Transformer 强化学习 (RLHF, DPO 等对齐训练)

15.8 ONNX 模型导出与推理

```
# 安装
pip install onnx onnxruntime

# PyTorch -> ONNX 导出
import torch.onnx
dummy_input = torch.randn(1, 3, 224, 224)
torch.onnx.export(model, dummy_input, "model.onnx",
                  input_names=["input"],
                  output_names=["output"],
                  dynamic_axes={"input": {0: "batch_size"}})

# ONNX Runtime 推理
import onnxruntime as ort
session = ort.InferenceSession("model.onnx")
outputs = session.run(None, {"input": input_array.numpy()})

# 验证 ONNX 模型
import onnx
onnx_model = onnx.load("model.onnx")
onnx.checker.check_model(onnx_model)      # 验证模型结构
```

十六、CUDA 与并行计算基础

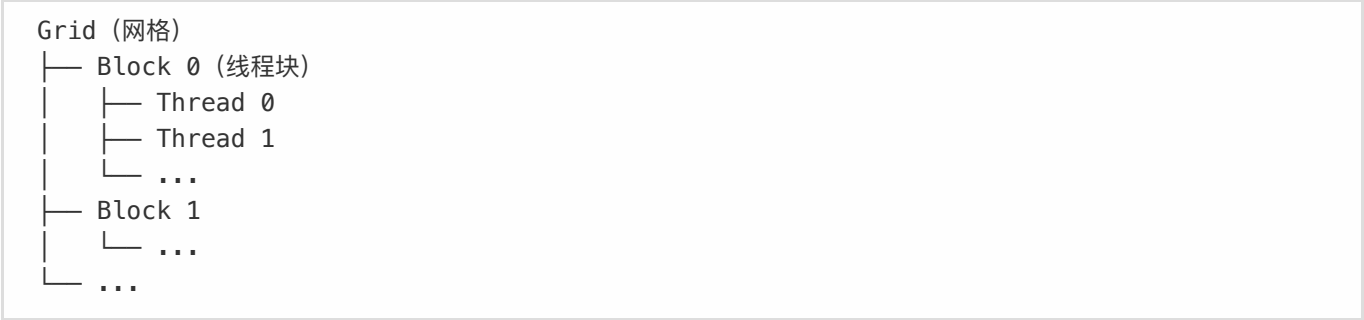
16.1 CUDA 编程模型概述

CUDA (Compute Unified Device Architecture) 是 NVIDIA 的并行计算平台和编程模型。理解以下核心概念是进行 GPU 编程和性能优化的前提。

主机 (Host) 与设备 (Device):

- Host = CPU + 系统内存
- Device = GPU + 显存 (VRAM)
- 数据必须在 Host 和 Device 之间显式传输 (cudaMemcpy)

线程层次结构:



- Thread: 最小执行单元
- Block: 一组线程，可以通过共享内存协作
- Grid: 一组 Block，构成一个 Kernel 的全部线程
- Warp: 硬件调度单位 = 32 个线程 (NVIDIA 特有概念)

内存层次结构:

内存类型	作用域	速度	大小
Global Memory	所有线程	慢 (~500 cycles)	大 (GB 级 = 显存)
Shared Memory	Block 内线程	快 (~20 cycles)	小 (通常 48-164 KB / Block)
Local Memory	单个线程	慢 (寄存器溢出时)	—
Constant Memory	所有线程 (只读)	快 (有缓存)	64 KB
Register	单个线程	最快	有限 (通常 255/thread)

基本 CUDA C/C++ 示例 (向量加法) :

```
// Kernel 函数 (在 GPU 上执行)
__global__ void vectorAdd(float *a, float *b, float *c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}

// Host 端调用
int n = 1000000;
int blockSize = 256;
int numBlocks = (n + blockSize - 1) / blockSize;
vectorAdd<<<numBlocks, blockSize>>>(d_a, d_b, d_c, n);
```

16.2 常用 CUDA 库

库	用途	示例用途
cuBLAS	基本线性代数子程序（矩阵乘法等）	GEMM 操作，PyTorch/TF 底层依赖
cuDNN	深度神经网络原语（卷积、池化、归一化等）	CNN/RNN/Transformer 加速
NCCL	多 GPU 集合通信（AllReduce, Broadcast 等）	分布式训练 GPU 间通信
cuFFT	快速傅里叶变换	信号处理、频谱分析
cuSPARSE	稀疏矩阵运算	稀疏网络、图计算
Thrust	C++ 模板并行算法库（类似 STL）	sort, reduce, scan 等
CUTLASS	高性能 GEMM 模板库	自定义矩阵乘法优化

16.3 nvcc 编译器与编译选项

```
# 编译 CUDA 源文件
nvcc -o program program.cu

# 指定计算能力 (Compute Capability)
nvcc -arch=sm_80 -o program program.cu      # A100
nvcc -arch=sm_89 -o program program.cu      # RTX 4090
nvcc -arch=sm_75 -o program program.cu      # RTX 2080 Ti

# 常用 GPU 计算能力对照
# sm_70: V100
# sm_75: RTX 2080 Ti / T4
# sm_80: A100
```

```

# sm_86: RTX 3090
# sm_89: RTX 4090
# sm_90: H100

# 优化选项
nvcc -O3 -use_fast_math -o program program.cu
nvcc -g -G -o program program.cu      # 调试模式 (-G 生成 GPU 调试信息)

# 查看 CUDA 版本
nvcc --version

# 查看 GPU 详细信息
nvidia-smi -q      # 完整信息
nvidia-smi --query-gpu=name,memory.total,compute_cap --format=csv

# 查看 Kernel 资源使用
nvcc --ptxas-options=-v program.cu    # 查看寄存器使用、shared memory 等

```

16.4 CUDA 性能分析工具

```

# nvidia-smi - 实时 GPU 监控
nvidia-smi      # 一次性查看
watch -n 1 nvidia-smi      # 每秒刷新
nvidia-smi dmon -s pucv    # 持续监控 (功耗/利用率/时钟/显存)

# nvidia-smi - 图形化 GPU 监控 (推荐, 仅 Linux)
# 安装: sudo apt install nvidia-smi (Ubuntu/Debian)
nvidia-smi

# gpustat - 简洁 GPU 状态
pip install gpustat
gpustat
gpustat -i 1      # 每秒刷新

# NVIDIA Nsight Systems - 系统级性能分析
nsys profile python train.py
nsys-ui report.qdrep      # 打开 GUI 查看时间线

# NVIDIA Nsight Compute - Kernel 级性能分析
ncu --set full -o profile python train.py
ncu-ui profile.ncu-rep

```

16.5 OpenMP 基础

OpenMP (Open Multi-Processing) 是一个面向共享内存多处理器系统的并行编程 API，主要用于 CPU 多核并行化。

```
# 编译 (gcc/g++)
g++ -fopenmp -o program program.cpp

# 查看 CPU 核心数
sysctl -n hw.ncpu          # macOS
nproc                      # Linux
```

核心指令速查:

```
// 并行 for 循环
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}

// 带 reduction 的并行 (求和、乘积等归约操作)
double sum = 0.0;
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < N; i++) {
    sum += a[i];
}

// 并行区域 (更灵活的控制)
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int nthreads = omp_get_num_threads();
    printf("Thread %d of %d\n", tid, nthreads);
}

// 临界区 (互斥访问)
#pragma omp critical
{
    // 一次只有一个线程执行此代码块
    shared_counter++;
}

// 设置线程数
omp_set_num_threads(8);
// 或通过环境变量
export OMP_NUM_THREADS=8
```

常用编译选项与技巧:

```
# 查看 OpenMP 版本支持
echo | cpp -fopenmp -dM | grep _OPENMP

# 性能调优
export OMP_NUM_THREADS=4           # 设置线程数
export OMP_PROC_BIND=true          # 绑定线程到核心
export OMP_PLACES=cores             # 线程放置策略

# 结合 CUDA 的使用场景
# GPU 负责大规模并行计算（矩阵乘法）
# OpenMP 负责 CPU 端的数据预处理和后处理
```

16.6 学习资源

- **CUDA C++ Programming Guide:** NVIDIA 官方编程指南，最权威的 CUDA 参考
 - **NVIDIA Deep Learning Institute (DLI):** 免费的 GPU 编程和深度学习实践课程
 - **CUDA by Example / Programming Massively Parallel Processors:** 经典入门书籍
 - **NVIDIA Developer Blog:** 最新的优化技巧和案例分享
 - **PyTorch CUDA Extension:** 学习如何编写自定义 CUDA Kernel 并与 PyTorch 集成
-

十七、数据科学与 MLOps 工具

17.1 数据处理框架

pandas — 传统王者:

```
pip install pandas
```

- 最成熟的 Python 数据分析库
- DataFrame 为核心数据结构
- 适合中小规模数据（能装入内存）

polars — Rust 实现的极速替代品:

```
pip install polars

# API 对比
# pandas
import pandas as pd
df = pd.read_csv("data.csv")
result = df.groupby("category").agg({"value": "mean"}).sort_values("value",
ascending=False)

# polars (惰性求值 + 表达式 API)
import polars as pl
result = (
    pl.scan_csv("data.csv")                # 惰性读取
    .group_by("category")
    .agg(pl.col("value").mean())
    .sort("value", descending=True)
    .collect()                             # 触发执行
)
```

pandas vs polars 对比:

维度	pandas	polars
实现语言	Python + C	Rust
内存效率	一般	高（零拷贝 / Arrow）
速度	基准	5-10x 更快
多线程	有限	原生支持
惰性求值	✗	✓
生态成熟度	极成熟	快速增长中

Apache Arrow 与 Parquet 格式:

```
pip install pyarrow

# Parquet – 列式存储格式, 压缩率高、读取快
df.to_parquet("data.parquet")
df = pd.read_parquet("data.parquet")

# Feather – Arrow 原生格式, 读写极快 (适合中间数据交换)
df.to_feather("data.feather")
df = pd.read_feather("data.feather")
```

17.2 数据版本控制 (DVC)

DVC (Data Version Control) 为 ML 项目提供类似 Git 的数据和模型版本管理, 但不将大文件存储在 Git 中。

```
# 安装
brew install dvc
# 或 pip install dvc

# 初始化 (在 Git 仓库中)
dvc init
git commit -m "Initialize DVC"

# 追踪数据文件
dvc add data/train.csv          # 生成 data/train.csv.dvc (元数据)
git add data/train.csv.dvc data/.gitignore
git commit -m "Add training data"

# 配置远程存储 (S3 示例)
dvc remote add -d myremote s3://my-bucket/dvc-store
dvc push                      # 推送数据到远程

# 拉取数据 (类似 git pull 但针对数据)
dvc pull

# 定义 ML Pipeline (dvc.yaml)
# stages:
#   preprocess:
#     cmd: python preprocess.py
#     deps: [data/raw/]
#     outs: [data/processed/]
#   train:
#     cmd: python train.py
#     deps: [data/processed/, src/train.py]
#     outs: [models/model.pth]
#     metrics: [metrics.json]

# 执行并追踪 Pipeline
dvc repro                      # 自动执行并追踪所有阶段
```

```
# 查看 Pipeline
```

```
dvc dag
```

```
# 可视化依赖图
```

17.3 实验与模型管理 (MLflow)

```
# 安装
```

```
brew install mlflow
```

```
# 或 pip install mlflow
```

```
# 启动 Tracking Server
```

```
mlflow ui --port 5000
```

```
# 或启动完整服务器 (含模型注册)
```

```
mlflow server --host 0.0.0.0 --port 5000 --backend-store-uri sqlite:///mlflow.db
```

```
# Python 集成示例
```

```
import mlflow
```

```
import mlflow.pytorch
```

```
mlflow.set_tracking_uri("http://localhost:5000")
```

```
mlflow.set_experiment("sentiment-classification")
```

```
with mlflow.start_run(run_name="bert-base-lr-2e-5"):
```

```
    # 记录超参数
```

```
    mlflow.log_params({"lr": 2e-5, "batch_size": 32, "epochs": 3})
```

```
    # 训练代码...
```

```
    # 记录指标
```

```
    mlflow.log_metric("train_loss", 0.35, step=1)
```

```
    mlflow.log_metric("val_accuracy", 0.92, step=1)
```

```
    # 保存模型
```

```
    mlflow.pytorch.log_model(model, "model")
```

```
    # 保存工件 (任意文件)
```

```
    mlflow.log_artifact("confusion_matrix.png")
```

```
# 加载已注册的模型
```

```
model = mlflow.pytorch.load_model("models:/sentiment-classifier/Production")
```

17.4 模型部署与推理服务

FastAPI 快速搭建模型 API:

```
pip install fastapi uvicorn
```

```
# app.py
```

```
from fastapi import FastAPI
```

```

from pydantic import BaseModel
import torch

app = FastAPI()
model = torch.jit.load("model.pt")
model.eval()

class PredictRequest(BaseModel):
    text: str

class PredictResponse(BaseModel):
    label: str
    confidence: float

@app.post("/predict", response_model=PredictResponse)
async def predict(req: PredictRequest):
    # 推理逻辑
    with torch.no_grad():
        output = model(req.text)
    return PredictResponse(label="positive", confidence=0.95)

# 启动: uvicorn app:app --host 0.0.0.0 --port 8000

```

Docker 容器化部署:

```

FROM pytorch/pytorch:2.2.0-cuda12.1-cudnn8-runtime
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

```

17.5 Triton Inference Server 简介

NVIDIA Triton Inference Server 是一个企业级模型推理服务器，支持多框架（PyTorch, TF, ONNX, TensorRT 等）、动态批处理、模型版本管理、多 GPU 并发推理。

```

# 快速体验 (Docker)
docker run --gpus all -p 8000:8000 -p 8001:8001 -p 8002:8002 \
  -v /path/to/models:/models \
  nvcr.io/nvidia/tritonserver:24.01-py3 \
  tritonserver --model-repository=/models

# 模型仓库结构
# models/
# └─ my_model/
#     └─ config.pbtxt          # 模型配置
#         └─ 1/                # 版本 1
#             └─ model.pt      # PyTorch 模型文件

```

客户端调用

```
pip install tritonclient
import tritonclient.http as httpclient
client = httpclient.InferenceServerClient("localhost:8000")
```

十八、Prompt Engineering 提示词工程

18.1 核心概念

Prompt Engineering（提示词工程）是设计和优化与大语言模型（LLM）交互的输入文本的技术，目的是引导模型生成高质量、准确、符合预期的输出。一个精心设计的 Prompt 可以显著提升模型在特定任务上的表现，甚至让模型完成其原本难以完成的任务。

为什么 Prompt Engineering 很重要：

- 大语言模型是概率性的——同样的 Prompt 可能得到不同的结果，好的 Prompt 能提高结果的一致性和可靠性
- 模型不会主动询问澄清信息——你需要在 Prompt 中提供足够的上下文和约束
- 模型没有真实世界的知识更新——你需要提供必要的背景信息和上下文
- 不同任务需要不同的 Prompt 策略——总结、推理、生成、分类各有最佳实践的写法

18.2 Prompt 的基本结构

一个设计良好的 Prompt 通常包含以下要素：

1. 角色 (Role/Persona)：定义模型应扮演的角色或身份，设定专业领域和行为方式
2. 任务描述 (Task Description)：清晰、精确地描述需要完成的任务
3. 上下文 (Context)：提供任务相关的背景信息、约束条件和前提假设
4. 格式要求 (Output Format)：指定输出的格式（JSON、Markdown 表格、列表、代码块等）
5. 示例 (Few-shot Examples)：提供一至多个输入-输出示例，帮助模型理解期望的模式
6. 约束与边界 (Constraints)：明确说明不能做什么、不需要包含什么、需要遵循什么规则
7. 评估标准 (Evaluation Criteria)：让模型知道什么样的输出被认为是好的

18.3 核心技巧

1. 角色设定 (Role Prompting)：

通过赋予模型一个特定的专家角色来引导其知识和语言风格：

- 「你是一位拥有 20 年经验的 Python 后端架构师」
- 「你是一位严格的代码审查员，专注于安全漏洞和性能问题」
- 「你是一位擅长将复杂技术概念解释给非技术人员的科普作家」

2. 思维链 (Chain-of-Thought, CoT)：

要求模型在给出最终答案之前展示推理过程，将复杂问题分解为多个步骤：

- 「让我们一步一步地思考这个问题」
- 「在给出答案前，请先分析问题的各个组成部分」
- 「首先识别问题的核心约束，然后列举可能的方案，最后选择最优方案并说明理由」

3. 少样本学习 (Few-shot Learning):

提供几个输入-输出的示例，让模型通过类比学习期望的模式和格式。示例应该：

- 覆盖不同但相关的情况
- 展示期望的输出格式和详细程度
- 与当前任务有相似的结构

4. 结构化输出 (Structured Output):

明确指定输出的格式，减少歧义和后续解析的成本：

- 「请以 JSON 格式返回结果，包含以下字段： ...」
- 「用 Markdown 表格展示，包含以下列： ...」
- 「输出格式为：一行总结，然后一个项目符号列表」

5. 分步指令 (Step-by-Step Instructions):

将复杂的任务分解为清晰的顺序步骤：

- 「第一步：阅读并理解以下代码。第二步：识别潜在的性能瓶颈。第三步：为每个瓶颈提出优化方案。第四步：评估各方案的优缺点。」

6. 约束与边界设定 (Constraints):

明确告诉模型什么不能做、什么不要输出：

- 「请不要提供任何示例代码」
- 「只使用 Python 标准库中的模块」
- 「不要解释你的推理过程，只给出最终答案」
- 「如果信息不足，请明确指出不确定的地方，而不是猜测」

7. 自我一致性 (Self-Consistency):

要求模型从多个角度验证自己的答案：

- 「请检查你的回答是否在逻辑上自洽」
- 「请从反面论证你的结论——有什么理由证明你的结论是错误的？」
- 「请审视你的代码中是否存在边界条件未处理的情况」

8. 思维树 (Tree of Thoughts):

让模型同时探索多个推理路径，比较后选择最优路径：

- 「针对这个问题，请提出三种不同的解决方案思路。对每种思路进行简要的可行性分析，最后推荐最佳方案并说明理由」

9. 负面提示 (Negative Prompting):

明确告诉模型应该避免的模式：

- 「不要使用过于技术化的术语」

- 「不要在回答中包含任何关于安装过程的说明」
- 「避免使用被动语态」

10. 温度与采样策略（API 层面）：

- `temperature=0` → 确定性输出，适合代码生成、数学、事实性问答
- `temperature=0.7-1.0` → 创造性输出，适合头脑风暴、创意写作
- `top_p=0.9` → 核采样，截断低概率的词汇，保持输出质量

18.4 常见应用场景的 Prompt 模式

代码生成：

- 明确编程语言和版本
- 指定输入和输出的数据结构
- 说明期望的时间/空间复杂度
- 要求包含错误处理和边界条件
- 要求包含类型注解和文档字符串

代码审查：

- 指定审查的重点维度（安全性、性能、可读性、可维护性）
- 要求按严重程度分类问题
- 要求对每个问题给出具体的修复建议
- 遵循的编码标准和最佳实践

代码调试：

- 提供完整的错误信息和堆栈跟踪
- 描述预期行为和实际行为的差异
- 提供相关环境信息（操作系统、语言版本、依赖版本）
- 说明已尝试过的解决方案

文档生成：

- 指定文档的目标读者
- 指定文档格式（README、API 文档、用户手册、内联注释）
- 要求包含哪些章节
- 技术深度和专业程度

技术解释：

- 指定目标受众的技术水平

- 要求使用类比和比喻帮助理解
- 由浅入深，先讲概念再讲原理最后讲实践
- 从「是什么 → 为什么需要 → 怎么用 → 什么时候用 → 常见误区」的结构组织

18.5 调试与优化 Prompt 的策略

1. 从简单开始：先写一个基础版的 Prompt 验证可行性，再逐步添加细节
 2. **AB 测试**：对同一个问题使用不同的 Prompt 变体，比较输出质量
 3. 系统性变化：每次只改一个变量（角色、格式、示例数量等），观察其对输出的影响
 4. 记录有效模式：维护一个 Prompt 模板库，记录在不同场景下效果好的 Prompt 模式
 5. 过度约束的风险：过多的约束可能限制模型的推理能力或导致输出僵化
 6. 长上下文的管理：将最重要的信息放在 Prompt 的开头和结尾（首尾效应），中间部分容易被模型忽略
 7. 迭代优化：如果初次输出不理想，不要放弃——基于不理想的输出调整 Prompt 并重试
-

十九、AI Agent 与 AI 辅助开发

19.1 核心概念

AI Agent (AI 智能体) 是一种能够自主感知环境、做出决策并采取行动以达成目标的 AI 系统。与传统的「单轮问答」LLM 交互不同, Agent 具有持续的自主性, 能够规划多步骤的任务, 使用外部工具, 并从反馈中学习和调整策略。

Agent 与传统 LLM 调用的关键区别:

- **自主性:** Agent 可以自己决定下一步做什么, 而不是被动等待指令
- **工具使用:** Agent 可以调用外部工具 (搜索引擎、代码执行器、API、数据库、文件系统等)
- **记忆与状态:** Agent 维护短期和长期记忆, 在多次交互中保持上下文
- **规划与反思:** Agent 可以将大任务分解为小步骤, 在执行过程中评估进展并调整计划
- **多模态:** 高级 Agent 能处理文本、图像、代码、结构化数据等多种输入形式

19.2 Agent 的核心架构

一个典型的 AI Agent 包含以下核心组件:

1. 推理引擎 (Reasoning Engine):

- 核心 LLM, 负责理解任务、制定计划、做出决策
- 可以是单个模型, 也可以是多模型协作 (如一个模型负责规划、另一个负责执行)
- 选择合适的模型对 Agent 性能至关重要——需要强大的推理能力和指令遵循能力

2. 工具集 (Tool Set):

Agent 可以调用的外部功能集合, 每个工具都有明确的输入输出规范:

- 代码执行工具 (Python 解释器、Shell 执行器)
- 信息检索工具 (搜索引擎、文档查询、数据库查询)
- 文件操作工具 (读取、写入、编辑文件)
- API 调用工具 (GitHub API、天气 API、日历 API 等)
- 浏览器工具 (网页浏览、表单填充、数据提取)

3. 记忆系统 (Memory System):

- 短期记忆 (工作记忆): 当前对话的上下文窗口, 包含最近的交互历史
- 长期记忆 (持久记忆): 跨会话的信息存储, 如用户偏好、项目知识库、经验教训
- 记忆的检索与更新机制: 如何高效地从大量记忆中找出与当前任务相关的信息

4. 规划模块 (Planning Module):

- 任务分解：将复杂任务拆解为可执行的具体步骤
- 依赖关系分析：识别步骤之间的依赖和顺序
- 资源分配：决定每个步骤使用哪些工具
- 应急计划：为可能的失败情况准备备选方案

5. 反思与纠错模块 (Reflection & Error Correction):

- 输出验证：检查执行结果是否符合预期
- 错误检测：识别执行失败或结果异常的步骤
- 调整策略：根据失败原因重新规划或使用替代方案
- 学习机制：从错误中提取经验以改进未来表现

19.3 主流 Agent 框架

框架	语言	核心特点
LangChain	Python / JS	最流行的 LLM 应用开发框架。提供链式调用、Agent 抽象、工具集成、记忆管理、RAG 支持。生态最丰富，文档最全面，但抽象层级多导致调试困难。
LangGraph	Python	LangChain 生态中的 Agent 编排框架。基于有向图的状态机模型，适合构建复杂的多步骤 Agent 工作流。提供可视化图编辑和调试。
AutoGen	Python	微软推出的多 Agent 对话框架。核心理念是多个专业 Agent 之间通过对话协作完成任务。内置代码执行沙箱和人类反馈循环。
CrewAI	Python	基于角色的多 Agent 协作框架。灵感来自团队协作模式，每个 Agent 有明确的角色、目标和背景故事。适合构建模拟团队工作流的应用。
Semantic Kernel	C# / Python / Java	微软推出的企业级 AI 编排框架。与 Azure 和 Microsoft 365 生态深度集成。使用「插件」而非「工具」的概念，支持原生函数调用。
OpenAI Agents SDK	Python	OpenAI 官方推出的 Agent 开发框架。轻量级，核心概念包括 Agent、Handoff（交接）、Guardrails（护栏）、Tracing（追踪）。与 OpenAI 模型原生集成最佳。
Dify	Python（平台）	开源的 LLM 应用开发平台。提供可视化拖拽界面，支持 Agent、工作流、RAG 管道。低代码，适合快速搭建原型。
Coze / 扣子	平台（字节跳动）	低代码 AI Bot 构建平台。提供插件市场、知识库、工作流编排。适合快速构建面向终端用户的 AI 助手。
VS Code Copilot Agent Mode	集成开发环境	VS Code 内置的 Agent 模式。可以在编辑器中通过自然语言指令让 Agent 自动浏览代码库、编辑文件、运行终端命令、安装依赖。目前最贴近开发者日常工作的 Agent 体验。

19.4 工具定义与调用

Agent 与外界交互的主要方式是通过「工具」或「函数」。定义工具时需要考虑以下要素：

工具定义的核心要素：

- 名称 (Name): 简洁且描述清晰的工具名称，便于模型理解和使用
- 描述 (Description): 详细说明工具的功能、适用场景和使用限制
- 参数模式 (Schema): 使用 JSON Schema 定义输入参数的类型、是否必填、默认值、取值范围
- 返回格式 (Return Format): 输出结果的格式规范

工具设计原则：

- 单一职责：每个工具只做一件事，做好一件事
- 自描述：工具的描述足够清晰，模型无需额外上下文就能判断何时使用
- 错误友好：返回有意义的错误信息，而不是原始异常，帮助 Agent 自我纠正
- 幂等性：同一个操作重复执行不会产生副作用（特别是写操作）
- 安全边界：在工具层面实施权限控制，不对 Agent 暴露危险操作

函数调用机制：

Agent 通常通过 LLM 的 Function Calling（函数调用）能力来选择和执行工具。LLM 不会实际执行函数——它只输出「应该调用哪个函数以及用什么参数」。Agent 框架负责实际执行函数，并将结果返回给 LLM 以继续推理。

19.5 常见应用模式

1. 检索增强生成 (RAG, Retrieval-Augmented Generation):

Agent 先从知识库（向量数据库、文档、网页等）中检索相关信息，再基于检索到的内容生成回答。解决了 LLM 知识截止日期和幻觉问题。

- 文档 QA：基于企业内部文档回答用户问题
- 代码库 QA：基于项目代码回答技术问题
- 客服系统：基于产品文档和工单历史回答问题

2. ReAct 模式 (Reasoning + Acting):

将推理（思考下一步做什么）和行动（执行操作）交织进行。Agent 在每一步都先思考再采取行动，然后观察结果，决定下一步。

- 循环：Think → Act → Observe → Think → Act → Observe → ...
- 适合需要与环境动态交互的任务（网页浏览、代码调试、数据分析）

3. 规划-执行模式 (Plan-and-Execute):

Agent 首先制定完整的行动计划，然后逐步执行，执行过程中根据反馈调整计划。

- 适用场景：任务步骤明确且相对稳定的场景
- 优势：整体规划比逐步试错更高效

- 劣势：初始计划可能不完美，需要灵活调整

4. 多 Agent 协作 (Multi-Agent Collaboration):

多个专业化的 Agent 通过通信和协商共同完成复杂任务。

- 角色分工：每个 Agent 负责不同领域（设计、开发、测试、部署）
- 通信方式：消息传递、共享黑板、事件驱动
- 挑战：协调成本、信息同步、冲突解决

5. Human-in-the-Loop（人在回路）：

在 Agent 执行关键操作（如代码提交、生产环境部署、大额交易）前暂停并请求人类审批。

- 关键决策点由人工确认
- Agent 遇到无法解决的歧义时请求人工协助
- 人类反馈用于微调和改进 Agent 行为

19.6 构建 Agent 的实践建议

1. 从简单开始，逐步增加复杂性：先用最基本的工具和流程搭建 MVP，验证核心逻辑后再添加更多工具和复杂推理
2. 充分测试工具描述：模型对工具的理解完全依赖于工具描述的质量。测试不同描述对模型选择正确工具的影响
3. 实施防护栏 (Guardrails)：在关键操作上设置硬性约束——限制文件操作范围、禁止执行特定命令、限制 API 调用频率
4. 全面日志与追踪：记录 Agent 的每一步推理、工具调用和结果。这在调试时是至关重要的一手信息，能让你理解 Agent 为什么做了某个决策
5. 设置最大步数限制：防止 Agent 陷入无限循环或过度迭代
6. 设计优雅的降级策略：当 Agent 无法完成任务时，应该给出有帮助的部分结果或明确的失败信息，而非无限尝试
7. 成本管理：每次推理、每个工具调用都有 API 成本。合理设计 Agent 的反思和验证频率，避免不必要的模型调用
8. 上下文窗口管理：长期任务会产生大量历史消息。需要实现滑动窗口、摘要压缩或外部记忆来避免超出模型的上下文限制

19.7 GitHub Copilot 完整使用指南

GitHub Copilot 是目前最广泛使用的 AI 编程助手，深度集成在 VS Code 中。

代码补全：

- 在编辑器中正常编写代码或注释，Copilot 会以灰色文字提供建议

- Tab — 接受当前建议
- Option +] (macOS) — 查看下一个建议
- Option + [(macOS) — 查看上一个建议
- Esc — 拒绝建议

写好注释引导 Copilot 的最佳实践：

```
# 好的注释（具体、包含约束）
# 读取 CSV 文件，将日期列解析为 datetime，按 user_id 分组计算每用户的总金额
# 返回一个 dict: {user_id: total_amount}

# 不好的注释（太模糊）
# 处理数据
```

Copilot Chat（聊天）：

- Cmd + I — 内联聊天（在编辑器中直接与 Copilot 对话，可引用选中代码）
- Cmd + Shift + I — 打开聊天面板（侧边栏聊天窗口）
- Ctrl + Enter — 在聊天中引用当前打开的文件

Copilot Agent Mode（代理模式）：

VS Code 中的 Agent 模式让 Copilot 从「建议者」升级为「执行者」：

- 自动浏览代码库、搜索相关文件
- 自动编辑多个文件
- 自动运行终端命令（需要用户审批）
- 自动安装依赖（需要用户审批）
- 迭代式修复——运行 → 观察错误 → 修复 → 再次运行

上下文管理（提升 Copilot 准确度）：

- **Open Relevant Files（打开相关文件）：** 保持当前任务相关的文件在编辑器中打开，Copilot 会将其作为上下文
- **.github/copilot-instructions.md：** 仓库级别的 Copilot 行为指令文件，定义项目约定、代码风格、框架偏好
- **.cursorrules / AGENTS.md：** 类似功能的配置文件（其他 AI 工具）

限制与注意事项：

- Copilot 生成的代码可能有安全漏洞，始终进行代码审查
- 对于框架/库的过时 API，Copilot 可能使用旧版本语法
- 复杂业务逻辑的生成可靠性不如简单 CRUD 代码
- 敏感信息（密钥、内部架构）可能通过提示词泄露给云端

Cursor:

Cursor 是基于 VS Code 的 AI-Native 编辑器，对 AI 功能的集成更深：

- Tab — 智能补全（比 Copilot 更激进，会预测多行修改）
- Cmd + K — 选中代码后，通过自然语言指令修改代码（内联编辑）
- Cmd + L — 打开 AI Chat 面板（侧边栏对话）
- Cmd + I — 打开 Composer（多文件编辑工作区）
- @符号引用 — @file, @folder, @web, @docs 等上下文引用
- 内置 .cursorrules 规则文件支持

与 VS Code + Copilot 的对比：

维度	VS Code + Copilot	Cursor
代码补全	单行/多行建议	多行跳转式编辑
上下文控制	有限	精确（@ 引用）
多文件编辑	Agent Mode	Composer 原生
规则/指令	.github/copilot-instructions.md	.cursorrules
生态兼容性	最全的扩展生态	兼容 VS Code 扩展
定价	Copilot 订阅	免费/Pro 订阅

Windsurf (Codeium):

- 另一种 AI-Native IDE，侧重点在「流式」Agent 体验
- Cascade 自动推理模式，适合探索性编程

19.9 Claude Code / Codex CLI — 终端中的 AI 编程助手

Claude Code (Anthropic):

```
# 安装
npm install -g @anthropic-ai/claude-code

# 在项目目录启动
claude

# 典型使用场景
# - "分析这个项目的代码结构，找出性能瓶颈"
# - "为 src/auth.py 写单元测试"
# - "帮我重构这个函数，提高可读性"
```

Claude Code 可以读取项目文件、执行终端命令、创建/编辑文件，是终端中的全能 AI Agent。

OpenAI CLI (通过 Python 包) :

```
# 安装 OpenAI Python CLI
pip install openai

# 使用 API 交互
openai api chat.completions.create -m gpt-4o -g user "创建一个 Python FastAPI 项目"
```

注：旧版 codex CLI 已废弃，推荐使用 Claude Code 或 GitHub Copilot CLI 替代终端 AI 编程需求。

19.10 Prompt Engineering 在代码场景中的实践

代码生成的 Prompt 要素：

1. 明确技术栈：「使用 Python 3.11 + FastAPI + SQLAlchemy」
2. 指定设计模式：「遵循 Repository 模式」
3. 给出输入/输出结构：提供 JSON Schema 或类型定义
4. 说明约束：「不使用第三方库」「需要类型注解」「需要错误处理」
5. 提供参考代码：粘贴项目中已有的类似实现作为风格参考

规则/指令文件（给 AI 的项目级说明书）：

.github/copilot-instructions.md 示例：

```
# 项目代码规范
- 所有 Python 代码必须使用类型注解
- 使用 Ruff 进行代码格式化和检查
- 数据库操作必须使用 SQLAlchemy 2.0 风格
- API 路由使用 FastAPI, Pydantic 模型定义在 schemas.py

# 项目结构约定
- 模型文件: app/models/
- 路由文件: app/routes/
- 测试文件: tests/ 对应模块名_test.py
- 禁止使用 print(), 使用 logging 模块
```

常见反模式：

- ❌ 一个问题拆成多个模糊的短对话
- ❌ 让 AI 猜测而不给上下文
- ❌ 不接受不完美的输出就直接放弃——应该迭代
- ❌ 不检查 AI 生成的代码就直接使用

19.11 AI 辅助代码审查

- 将 PR diff 粘贴到 Copilot Chat / ChatGPT，请求：「请审查这段代码，重点检查安全漏洞、性能问题和边界条件处理」
- 使用 CodeRabbit、Sweep 等自动化 AI 代码审查工具
- 对关键模块（认证、支付、数据加密）进行人工深度审查——AI 审查不能替代人工

19.12 AI 工具选择建议

场景	推荐工具
日常编码补全	GitHub Copilot
复杂多文件重构	Cursor Composer / VS Code Copilot Agent Mode
全项目分析与修改	Claude Code / Codex CLI
快速原型搭建	Cursor + Claude Code
代码审查	Copilot Chat + 人工检查
企业/合规环境	GitHub Copilot Enterprise + 私有化部署模型

二十、云 GPU 训练完整 workflow

20.1 环境快速复现

在新 GPU 服务器上快速搭建训练环境：

```
# 方案 1: 使用 NGC Docker 镜像 (推荐, 环境最一致)
docker pull nvcr.io/nvidia/pytorch:24.01-py3
docker run --gpus all -it -v $(pwd):/workspace nvcr.io/nvidia/pytorch:24.01-py3

# 方案 2: 使用 Conda environment.yml
conda env create -f environment.yml
conda activate myenv

# 方案 3: 使用 pip + venv
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 验证环境
python -c "import torch; print(f'CUDA: {torch.cuda.is_available()}'); print(f'Device: {torch.cuda.get_device_name(0)}')"
```

20.2 远程连接与进程保持

```
# SSH 端口转发 (Jupyter + TensorBoard)
ssh -L 8888:localhost:8888 -L 6006:localhost:6006 user@gpu-server

# tmux 保持训练进程 (最推荐)
tmux new -s training
python train.py
# Ctrl+B, D 分离
# tmux attach -t training 恢复

# nohup 替代方案 (无需 tmux)
nohup python train.py > train.log 2>&1 &
# 查看日志
tail -f train.log

# screen 替代方案 (比 tmux 更老但广泛可用)
screen -S training
python train.py
# Ctrl+A, D 分离
# screen -r training 恢复
```

20.3 代码与数据同步

```
# rsync 增量同步 (最常用)
rsync -avz --progress --exclude '.venv/' --exclude '__pycache__/' \
    /local/project/ user@gpu-server:/remote/project/

# 从远程同步结果/模型到本地
rsync -avz user@gpu-server:/remote/project/checkpoints/ /local/project/checkpoints/

# Git + DVC 协作
git push                # 推送代码
dvc push                # 推送数据到远程存储
# 远程服务器
git pull && dvc pull    # 拉取代码和数据

# 大数据集传输策略
# - <10GB: rsync 直接传输
# - 10-100GB: 压缩后 rsync 或使用云存储中转 (S3/GCS)
# - >100GB: 直接在云存储上下载, 避免通过本地中转
```

20.4 GPU 资源监控与管理

```
# 持续监控 GPU 状态
nvidia-smi -l 1          # 每秒刷新
watch -n 1 nvidia-smi    # 同上

# 更友好的 GPU 监控
nvidia-smi                # 图形化实时监控
gpustat -i 1              # 彩色终端输出

# GPU 显存优化技巧
# 1. 混合精度训练 (FP16/BF16)
from torch.cuda.amp import autocast, GradScaler
scaler = GradScaler()
with autocast():
    loss = model(input)

# 2. 梯度检查点 (Gradient Checkpointing) - 用计算换显存
# HuggingFace Transformers 模型:
model.gradient_checkpointing_enable()
# 纯 PyTorch 使用:
# from torch.utils.checkpoint import checkpoint

# 3. 梯度累积 - 模拟大 batch size
accumulation_steps = 4
for i, batch in enumerate(dataloader):
    loss = model(batch) / accumulation_steps
    loss.backward()
    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
```

```
optimizer.zero_grad()

# 4. 及时释放不需要的张量
del intermediate_tensor
torch.cuda.empty_cache()

# 检查显存使用
print(torch.cuda.memory_summary())
```

20.5 训练最佳实践

```
# 命令行参数管理 (推荐使用 argparse + dataclass)
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--lr", type=float, default=1e-3)
parser.add_argument("--batch_size", type=int, default=32)
parser.add_argument("--epochs", type=int, default=100)
args = parser.parse_args()

# 或使用 Hydra (更强大的配置管理)
# pip install hydra-core
# @hydra.main(config_path="configs", config_name="train")
```

```
# Checkpoint 保存与恢复模板
import os
import torch

def save_checkpoint(model, optimizer, epoch, loss, path="checkpoints"):
    os.makedirs(path, exist_ok=True)
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'loss': loss,
    }, f"{path}/checkpoint_epoch_{epoch}.pt")

def load_checkpoint(model, optimizer, path):
    checkpoint = torch.load(path)
    model.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
    return checkpoint['epoch'], checkpoint['loss']
```

```
# 分布式训练入门 (单机多卡 - torchrun / DDP)
torchrun --nproc_per_node=4 train.py
# 在 train.py 中使用 DistributedDataParallel (DDP)

# 多机多卡 (需要 NCCL + 网络配置)
torchrun --nnodes=2 --nproc_per_node=8 --rdzv_backend=c10d train.py
```

20.6 云平台成本优化

- 竞价/抢占实例：价格通常为按需实例的 30-50%，适合可中断的训练任务（配合 checkpoint 恢复）
 - 快速验证再全量：先在 CPU 或单 GPU 上跑小规模实验验证代码正确性，确认无误后再提交多卡全量训练
 - 合理选择 GPU 型号：小模型用 T4/L4（便宜），大模型训练用 A100/H100，推理用 L40S
 - 存储成本：定期清理旧的 checkpoint（只保留最佳的 N 个），不用的数据集及时从云盘删除
 - 关机不释放磁盘：大部分云平台停止实例后磁盘仍计费，不需要的环境及时释放
-

文档编写日期：2026年5月27日

适用平台：macOS (Apple Silicon / Intel) + Linux GPU 服务器

持续更新：本档会根据工具的版本更新和最佳实践的变化持续维护